

Informed Search (Heuristic Search)

Boris Velichkov

Exercises by Week

1. Introduction
2. **Uninformed (*Blind*) Search**
3. **Informed (*Heuristic*) Search**
4. **Constraint Satisfaction Problems**
5. **Genetic Algorithms**
6. **Games**
7. Introduction to Machine Learning
8. ***k*-Nearest Neighbors**
9. **Naïve Bayes Classifier**
10. **Decision Tree**
11. ***k*Means**
12. **Neural Networks**
13. *Additional Topics, Questions and Homeworks' Presentations*
14. *Additional Topics, Questions and Homeworks' Presentations*
15. *Additional Topics, Questions and Homeworks' Presentations*

Uninformed (*Blind*) Search vs Informed (*Heuristic*) Search

- **Uninformed Search:** Uninformed strategies use only the information available in the problem definition.
 - *Examples: DFS, BFS, UCS, DLS, IDS.*
- **Informed Search:** Informed strategies have information on the *goal state* which helps in more efficient searching. This information is obtained by a function (*heuristic*) that estimates how close a state is to the *goal state*.
 - *Examples: Greedy Best-First Search, A*, Beam Search, Hill Climbing.*

Informed (*Heuristic*) Search

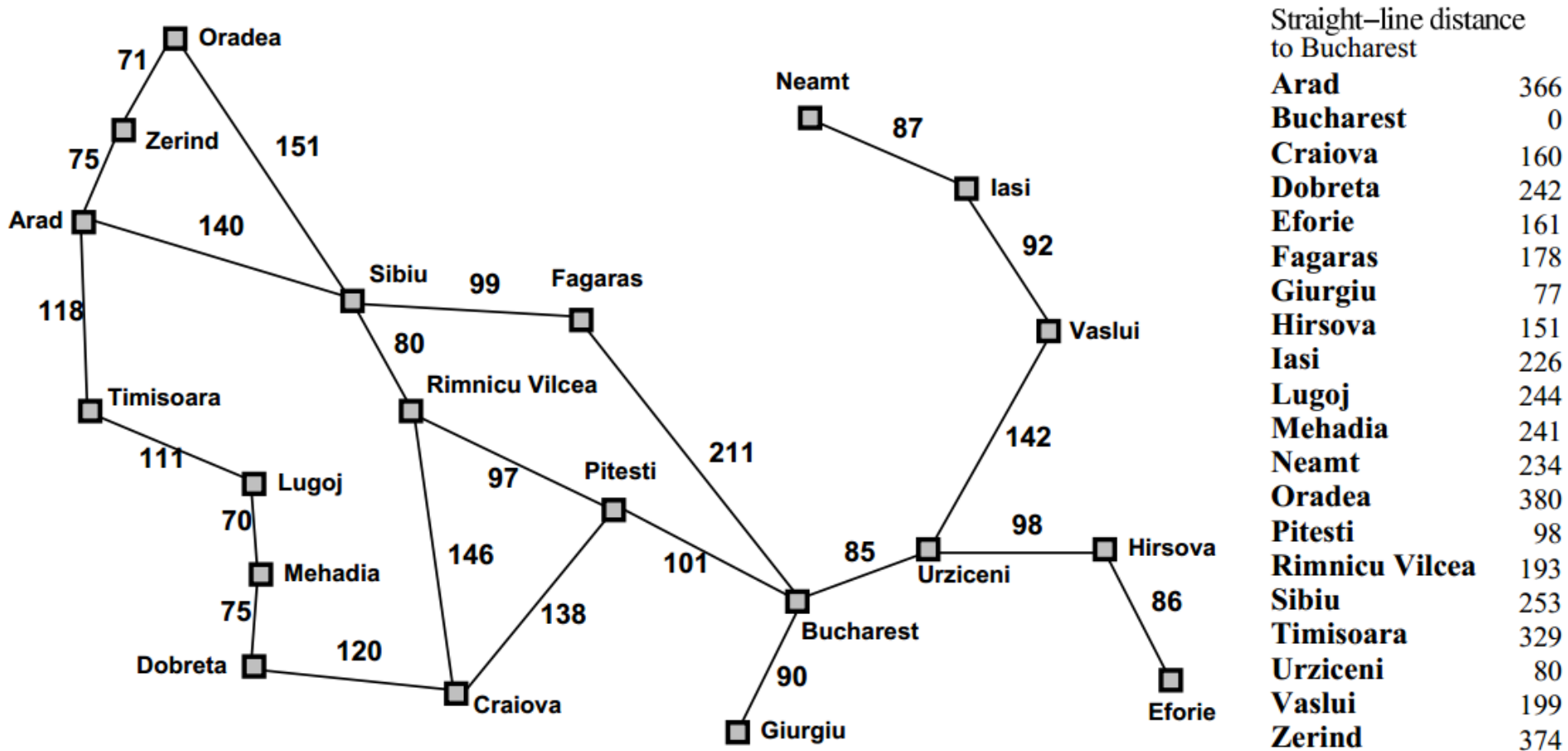
- *“Informed strategies have information on the goal state which helps in more efficient searching. This information is obtained by a **function (heuristic)** that estimates how close a state is to the goal state.”*
- Informed Search Algorithms:
 - Best-First Search
 - Greedy
 - Beam Search
 - Hill Climbing
 - A*
 - Memory-Bounded A*
 - Iterative Deepening A* (IDA*)

Best-First Search

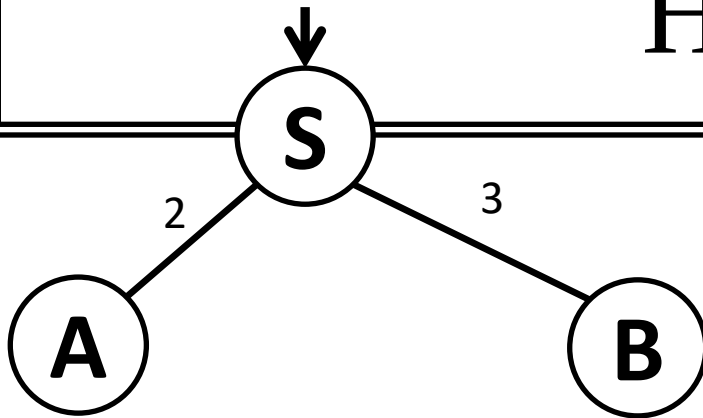
- **Idea:** use an evaluation function for each node
 - estimate of “desirability”
⇒ Expand most desirable unexpanded node
- **Implementation:**
 - ***fringe*** is a queue sorted in decreasing order of desirability
 - Special cases:
 - Greedy search
 - A* search

Best-First Search

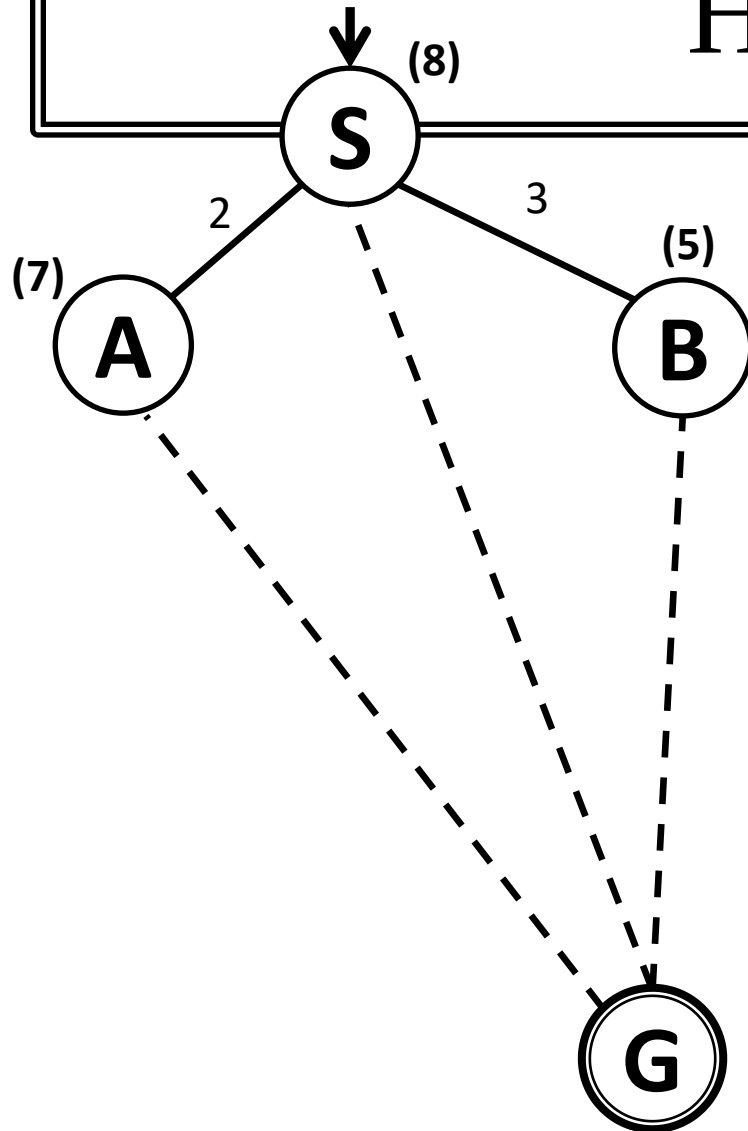
Romania with step costs in km



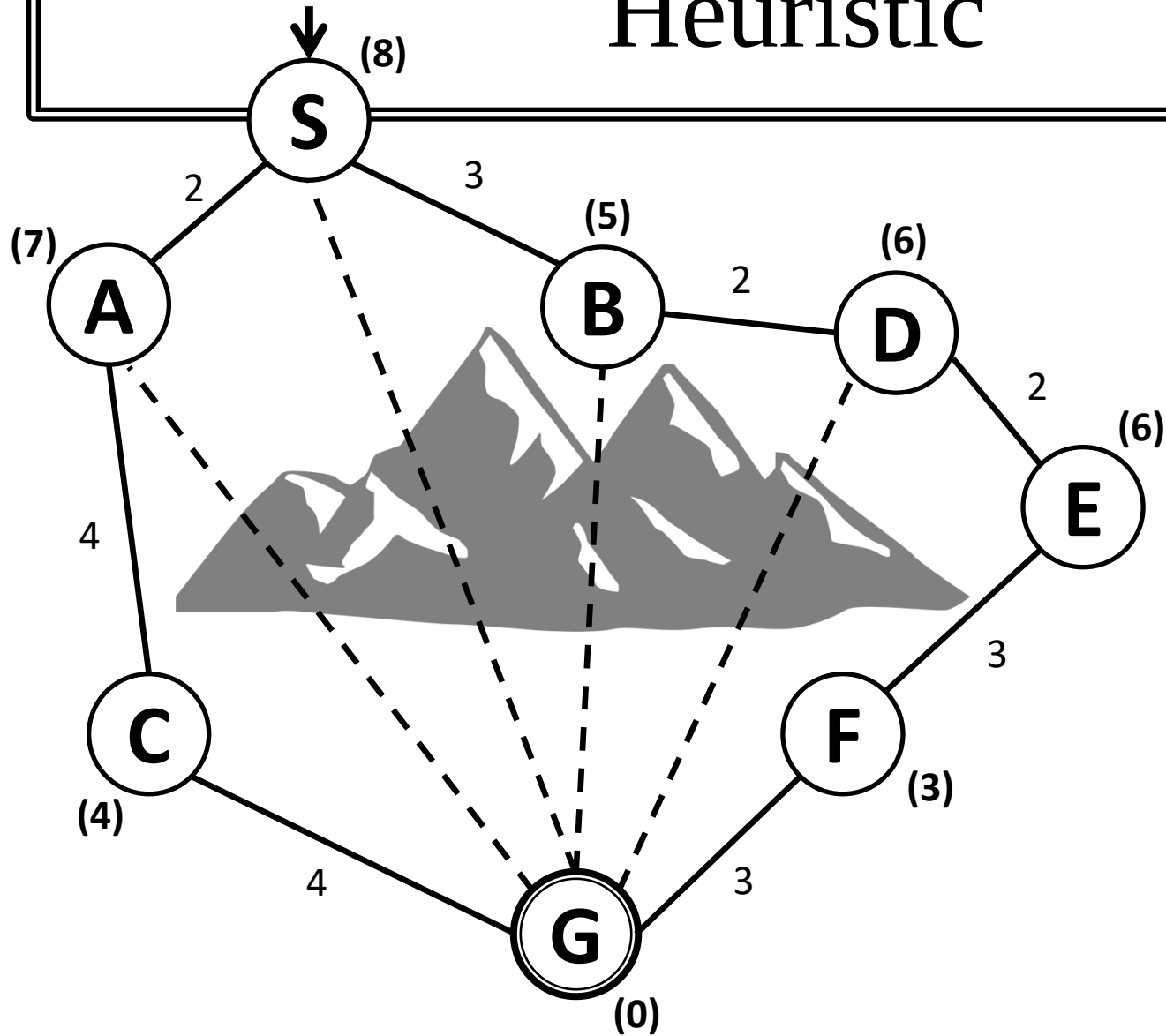
Heuristic



Heuristic



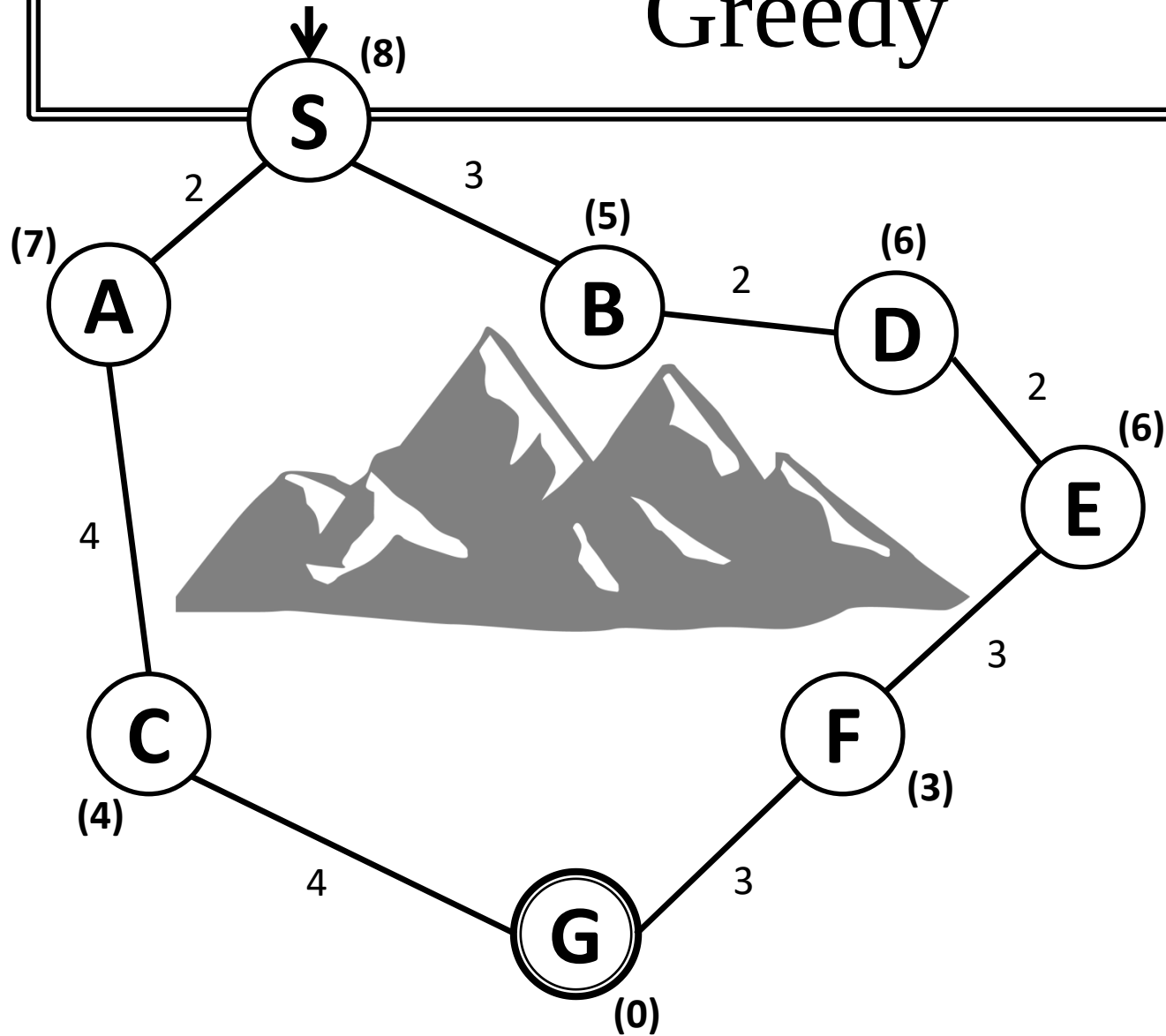
Heuristic

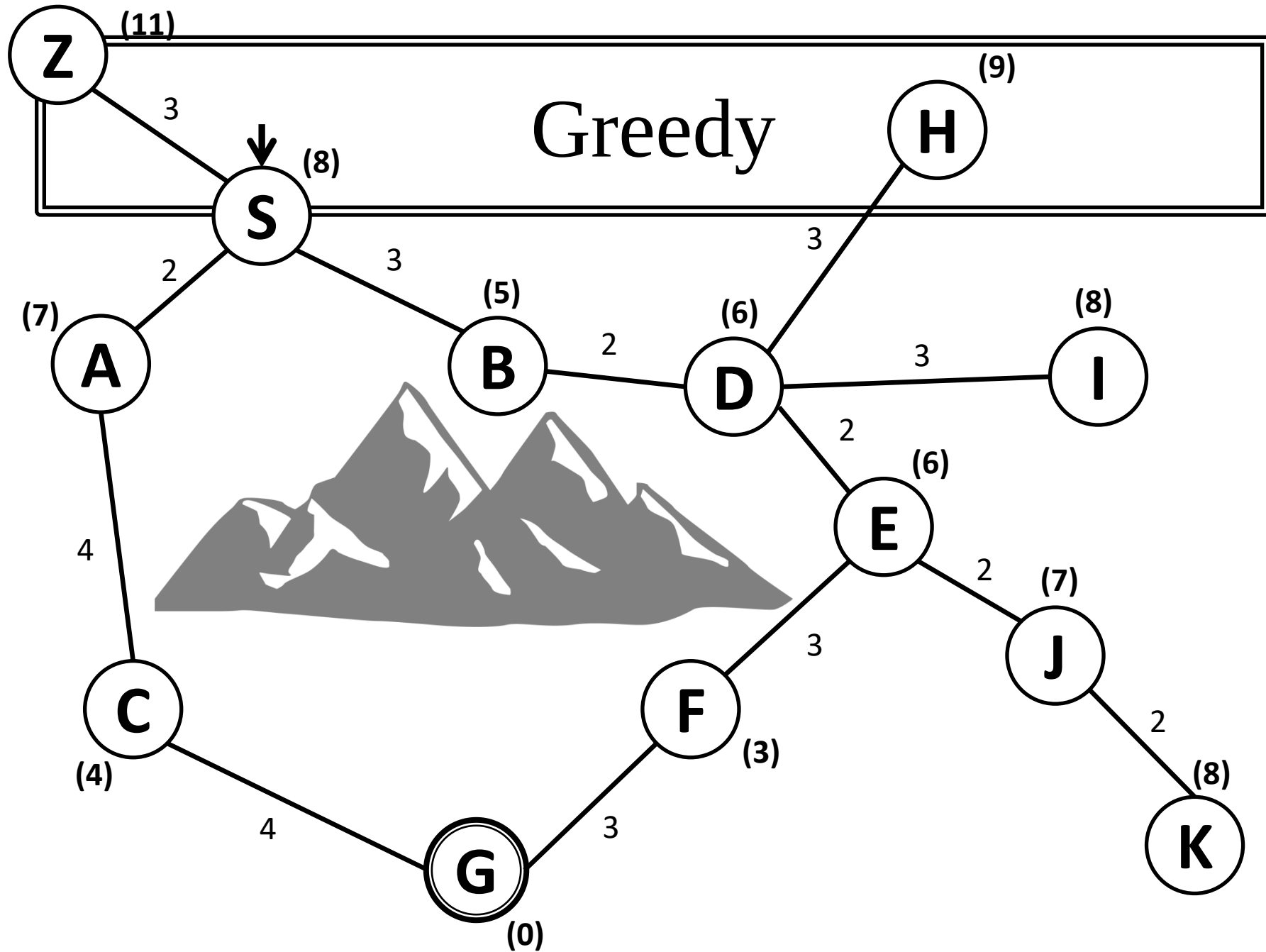


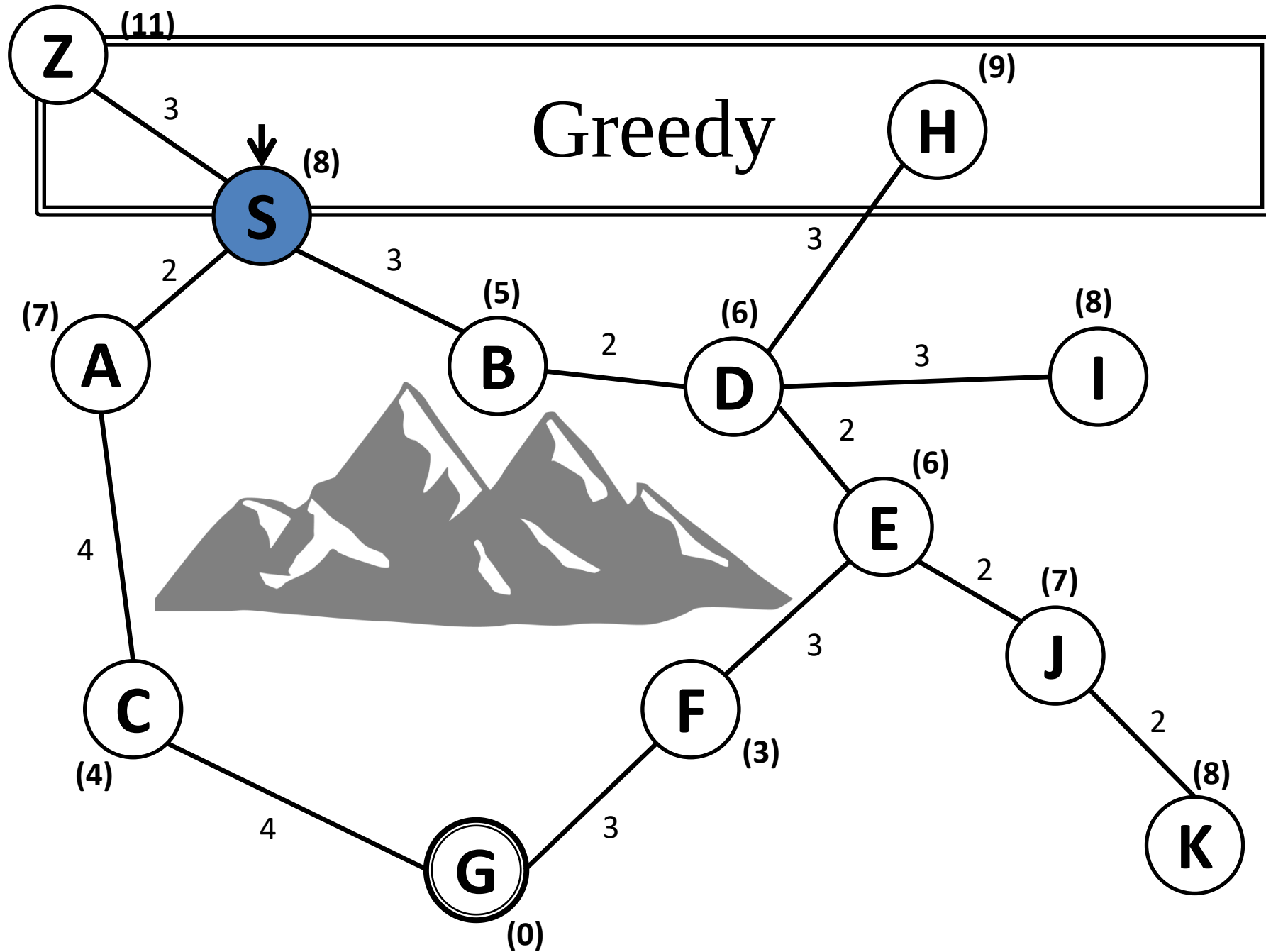
Greedy

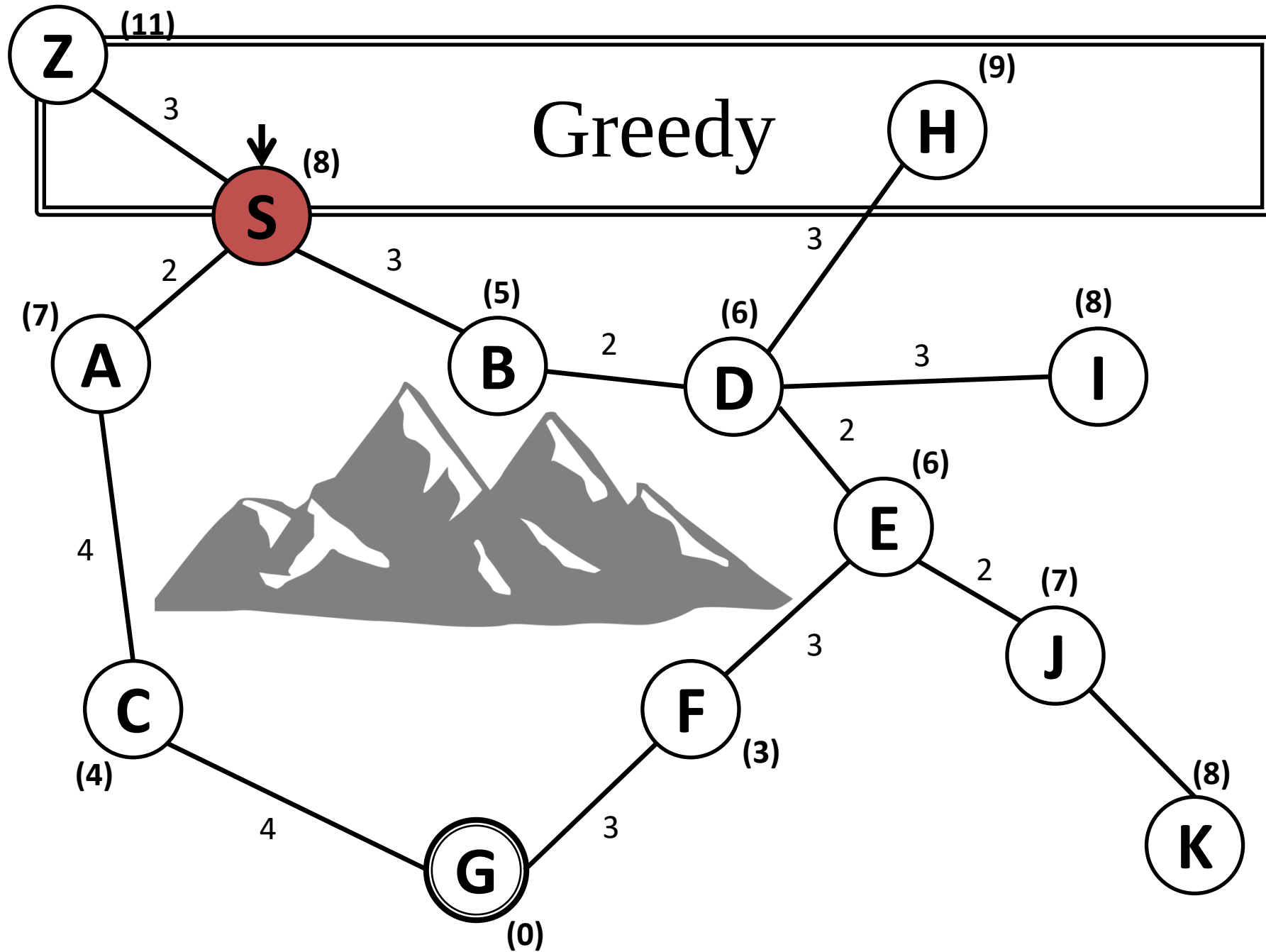
- Evaluation function $h(n)$ (*heuristic*)
= estimate of cost from n to the closest *goal*
- E.g., $h_{SLD}(n)$ = *straight-line distance* from n to *goal* (*Bucharest*)
- **Greedy** search expands the node that **appears** to be closest to *goal*

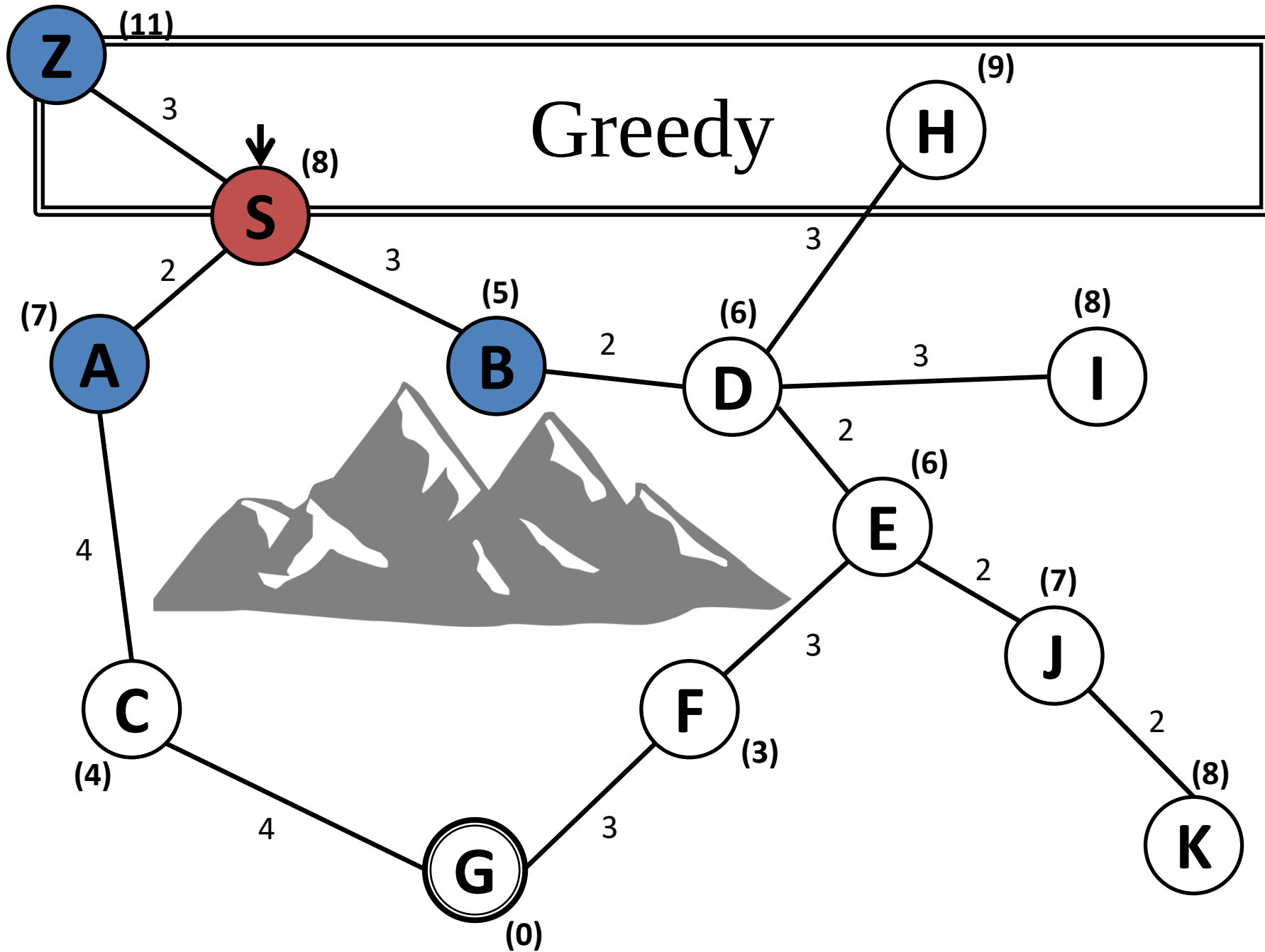
Greedy

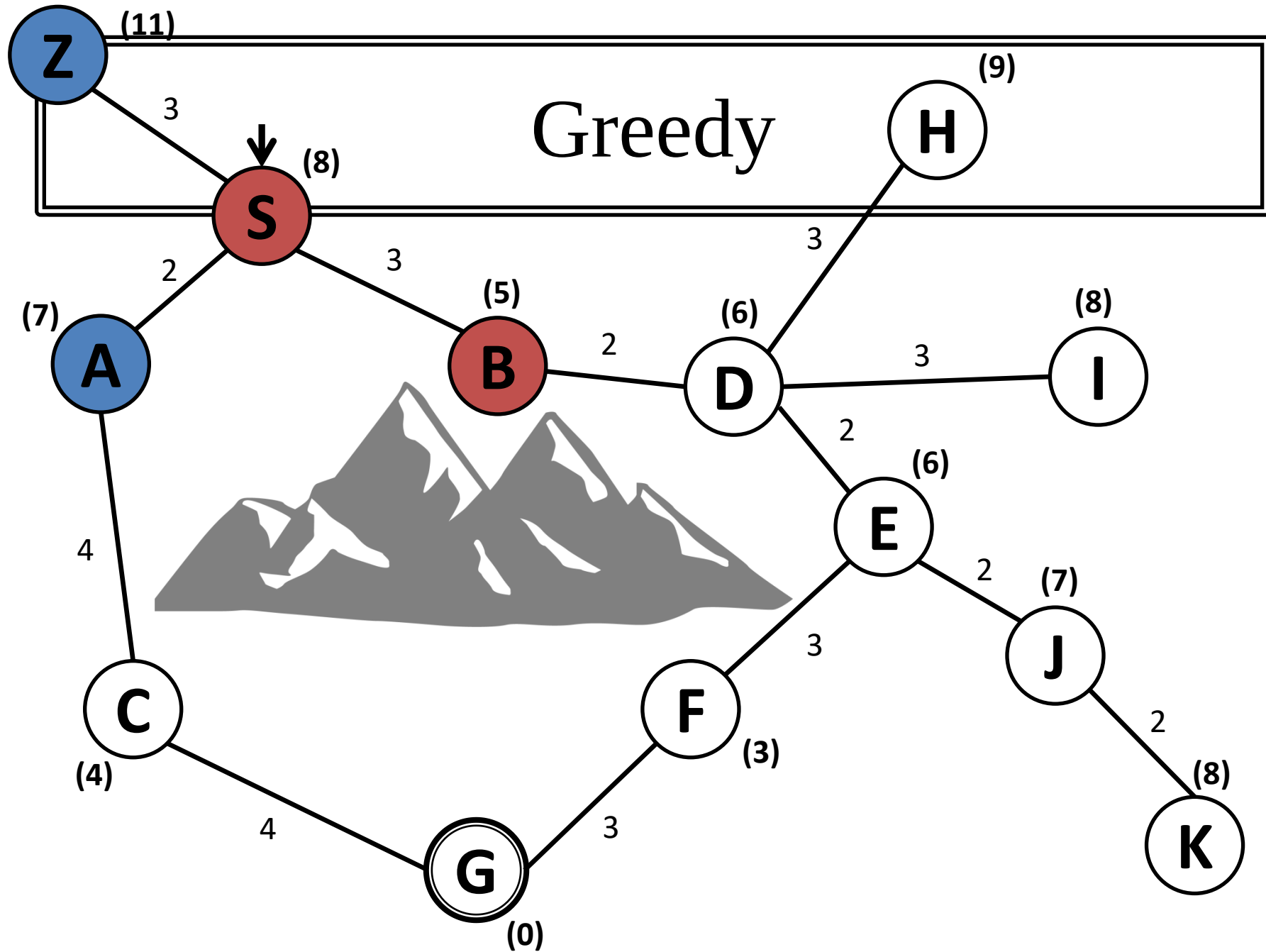


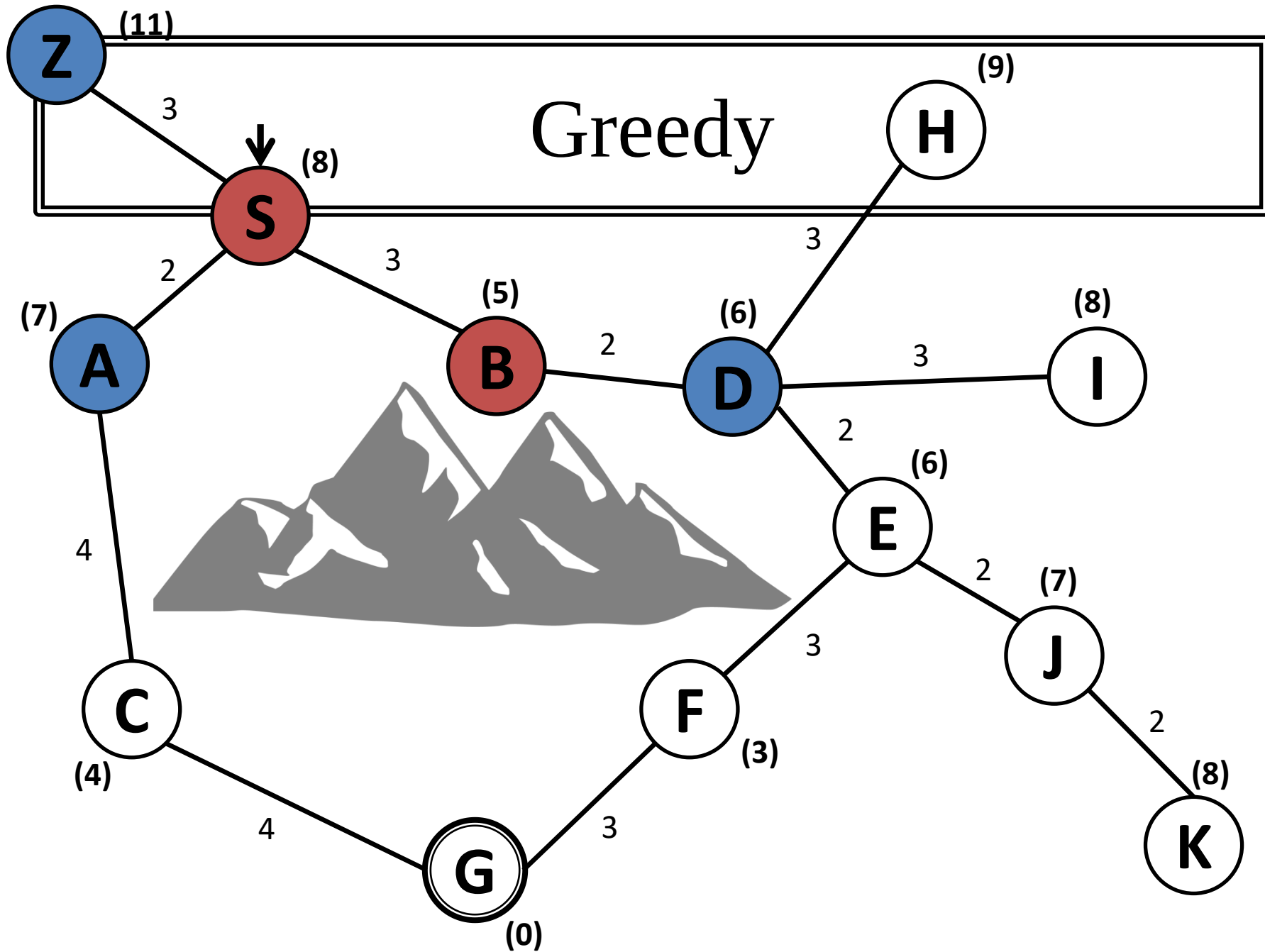


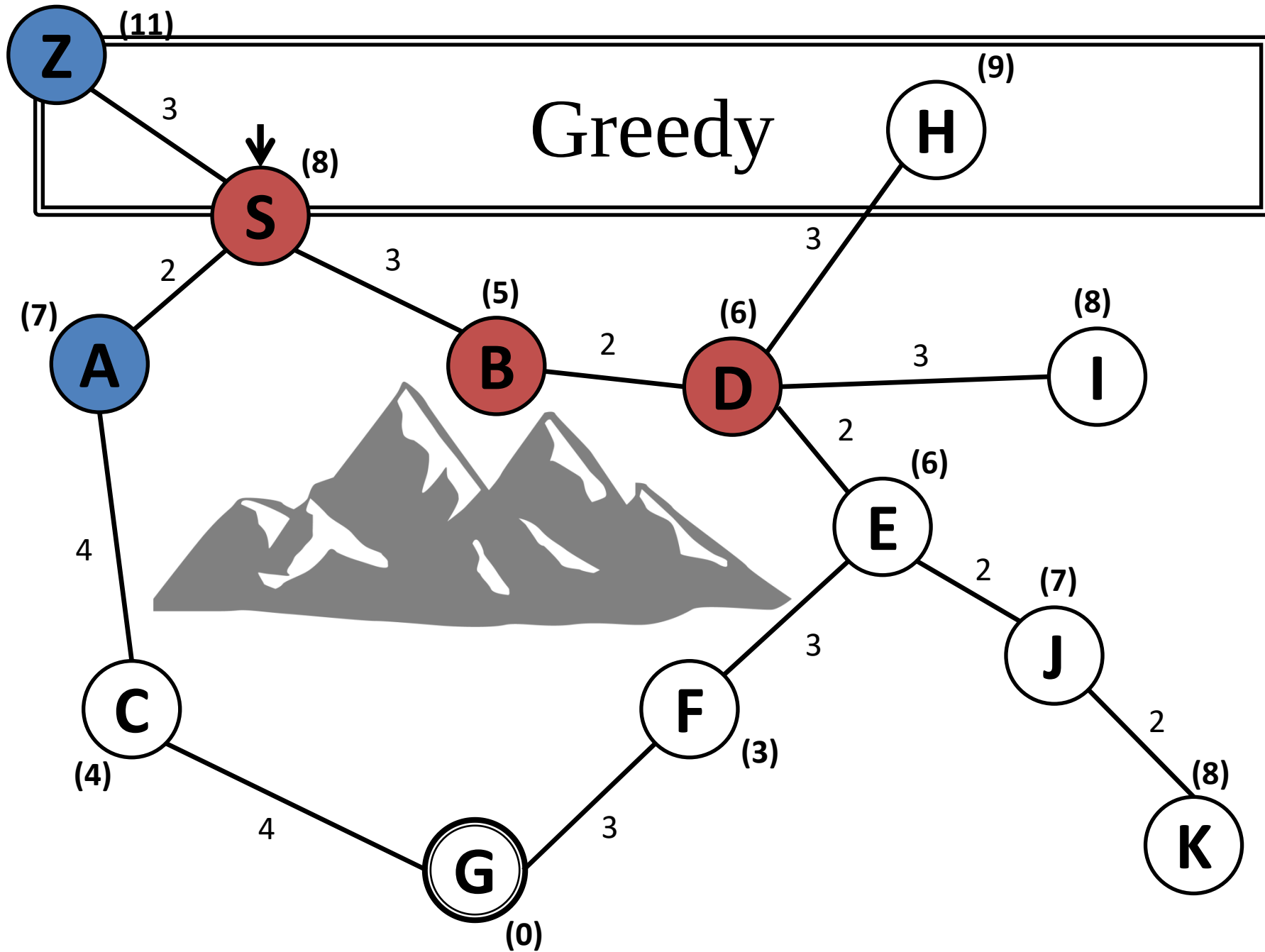


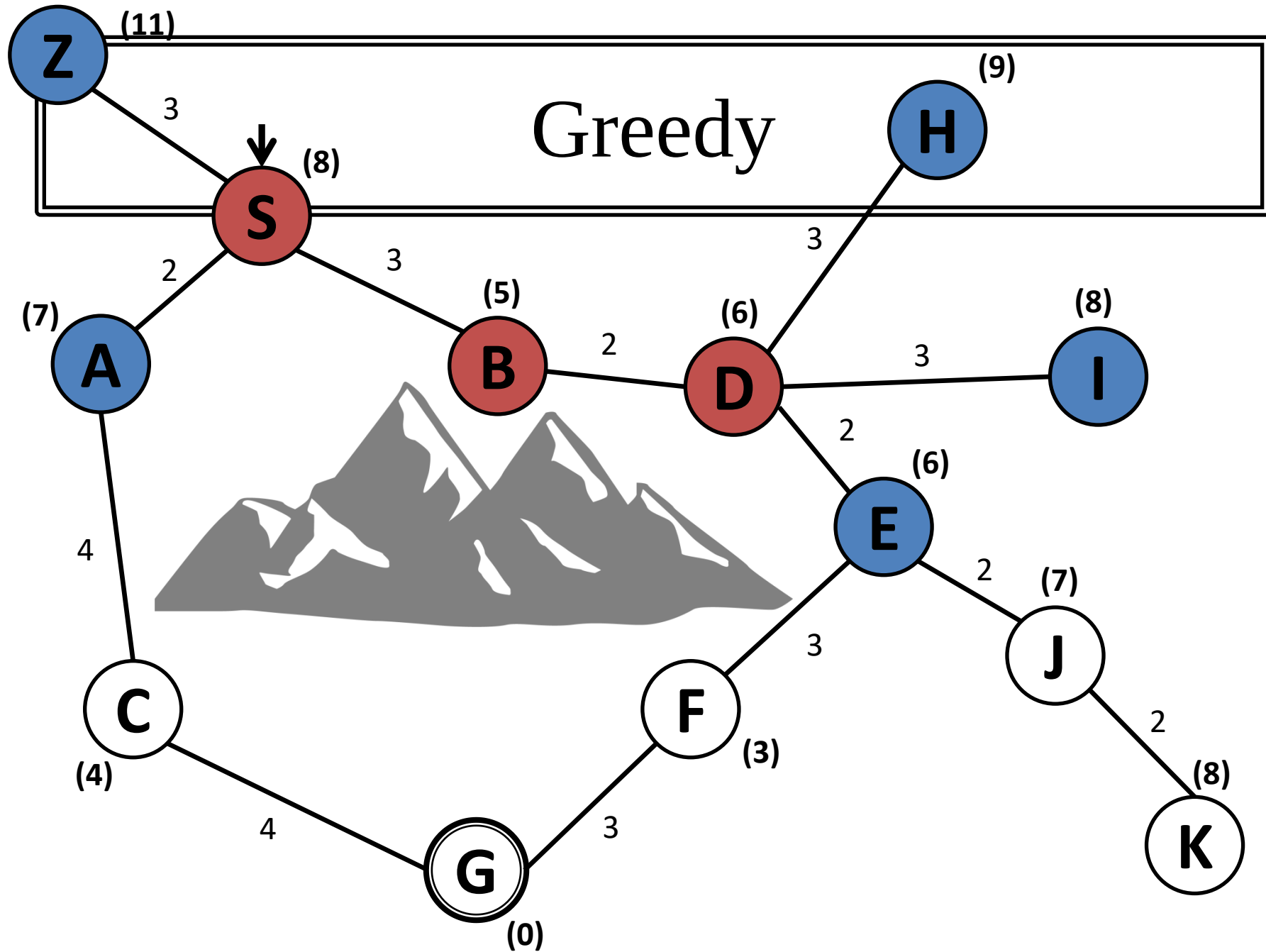


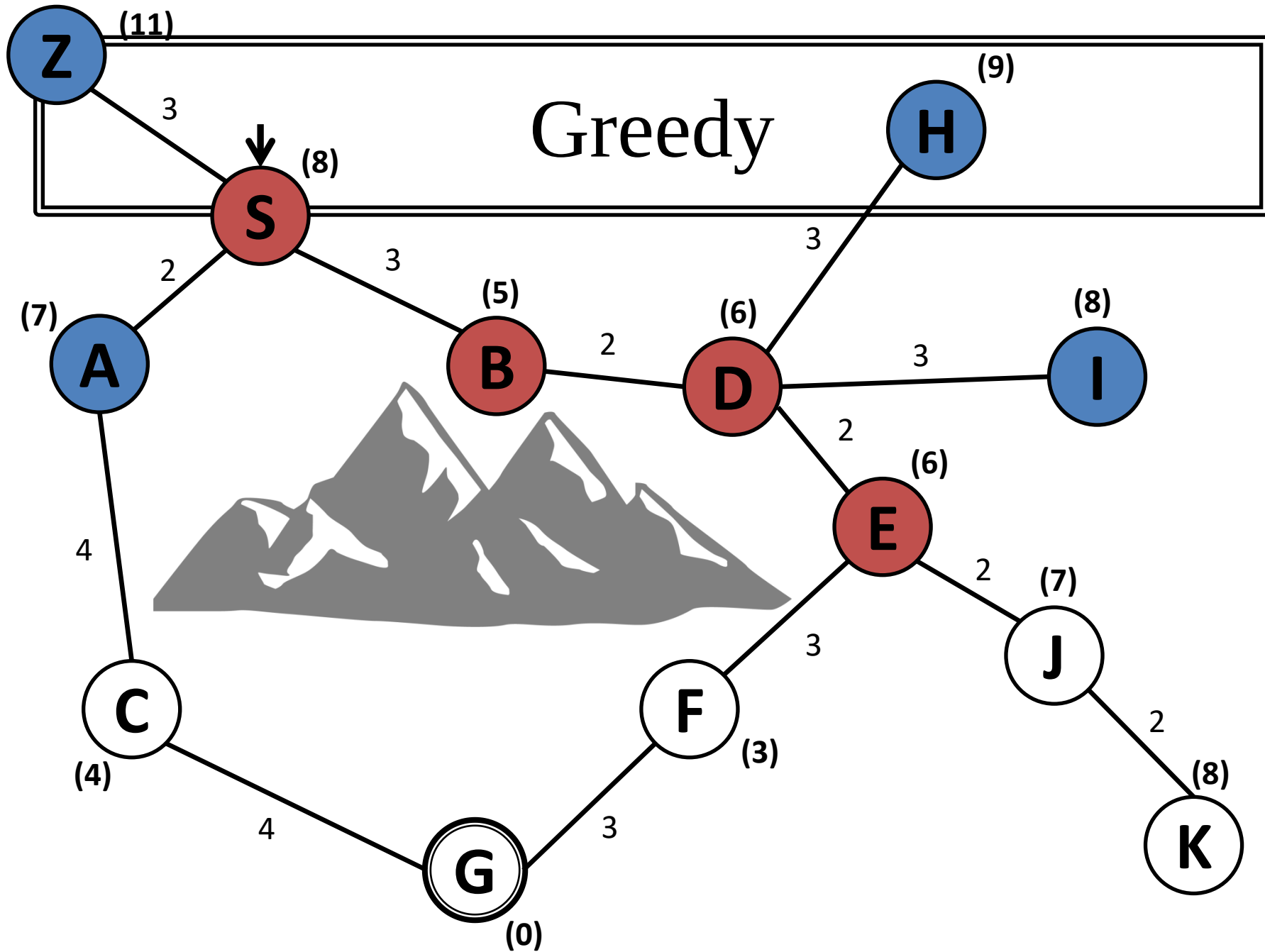


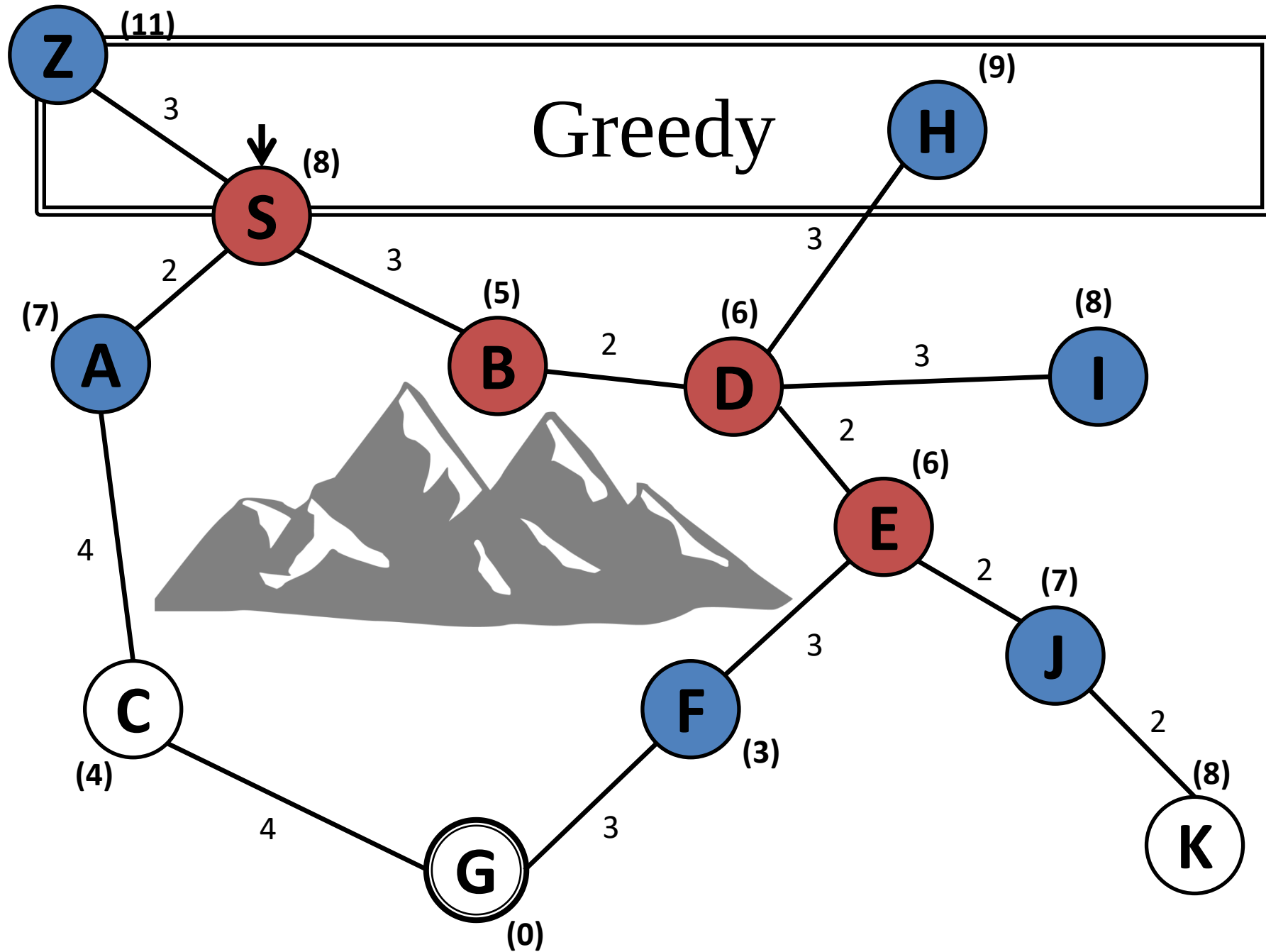


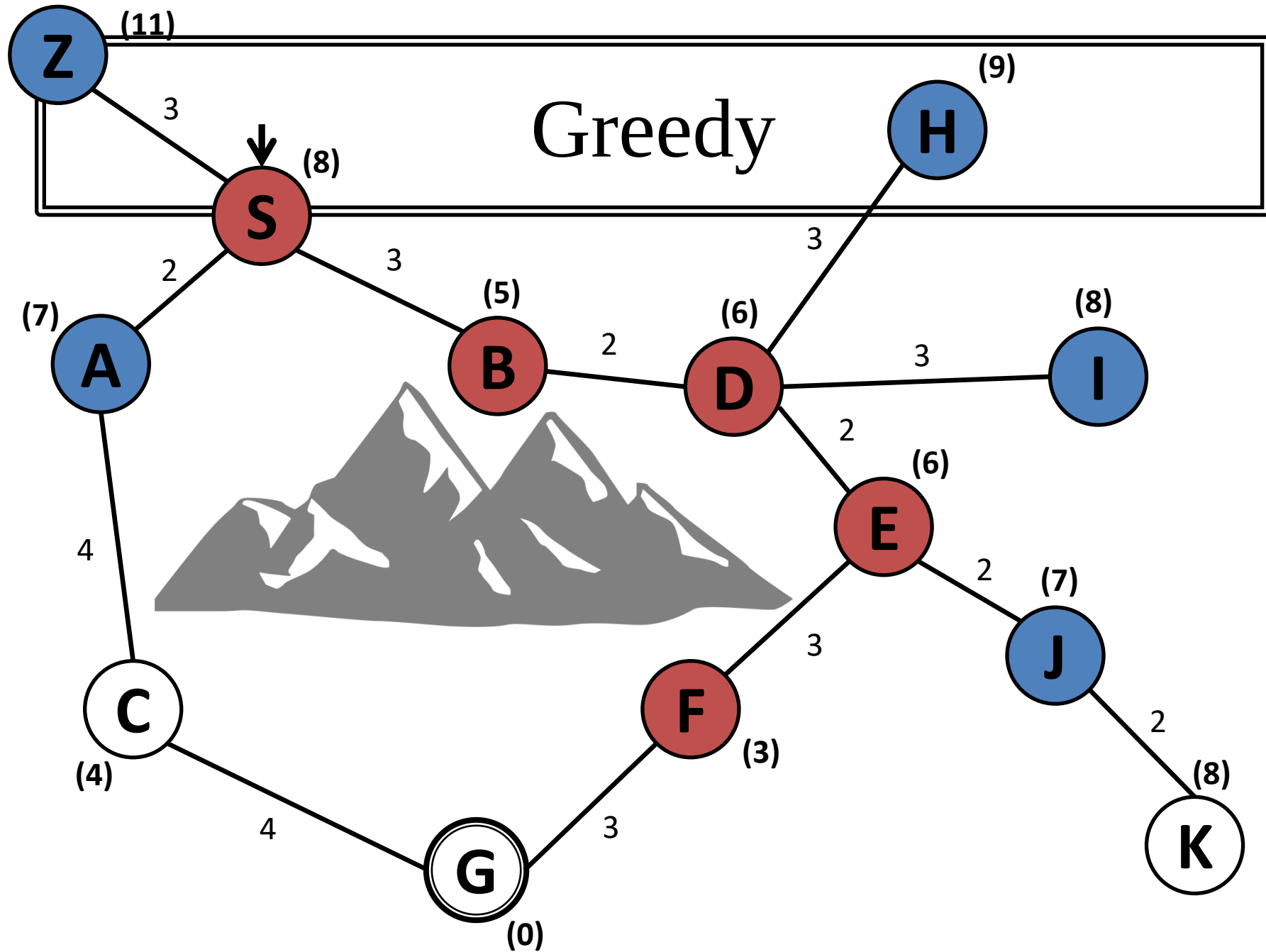


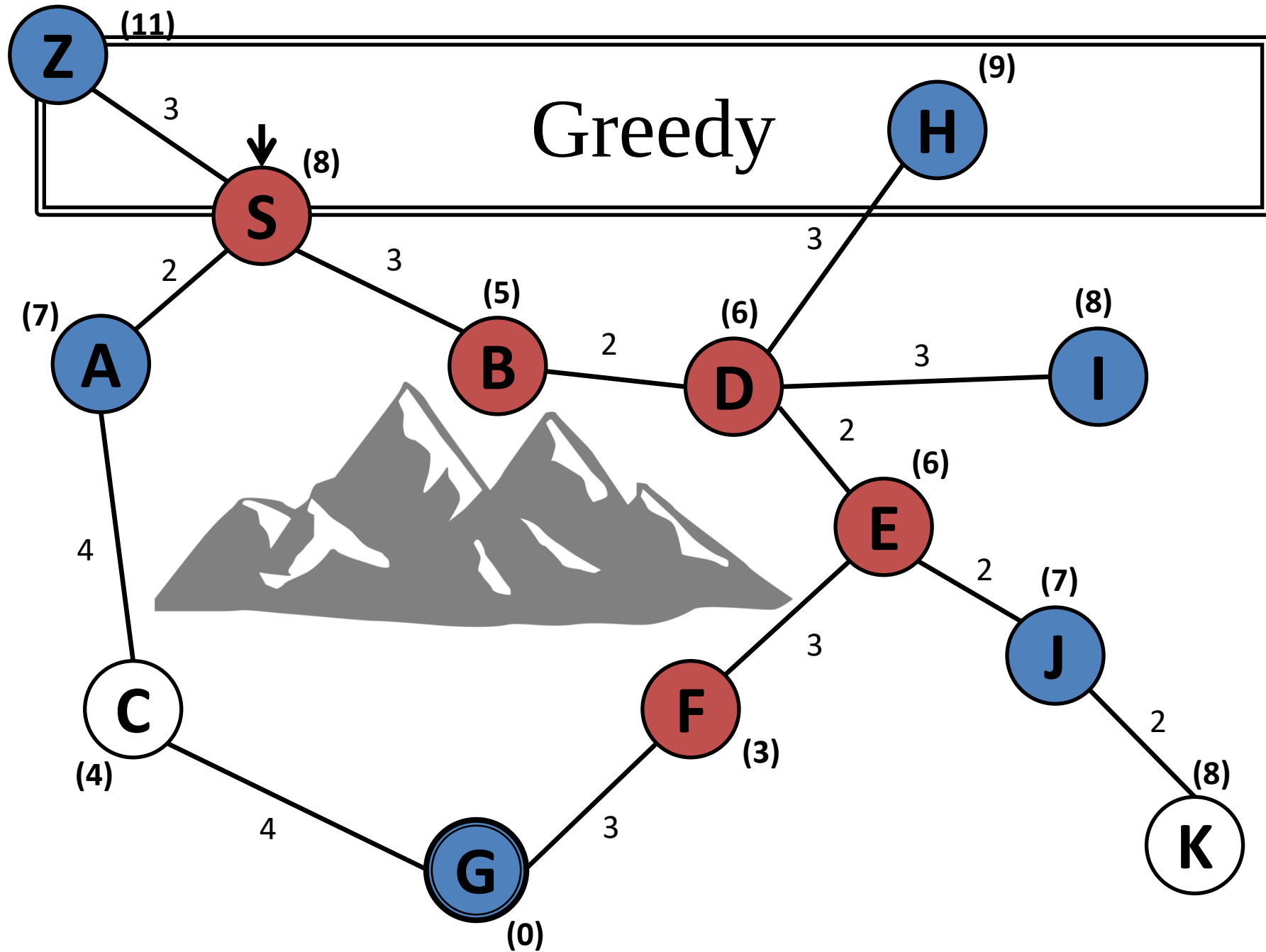


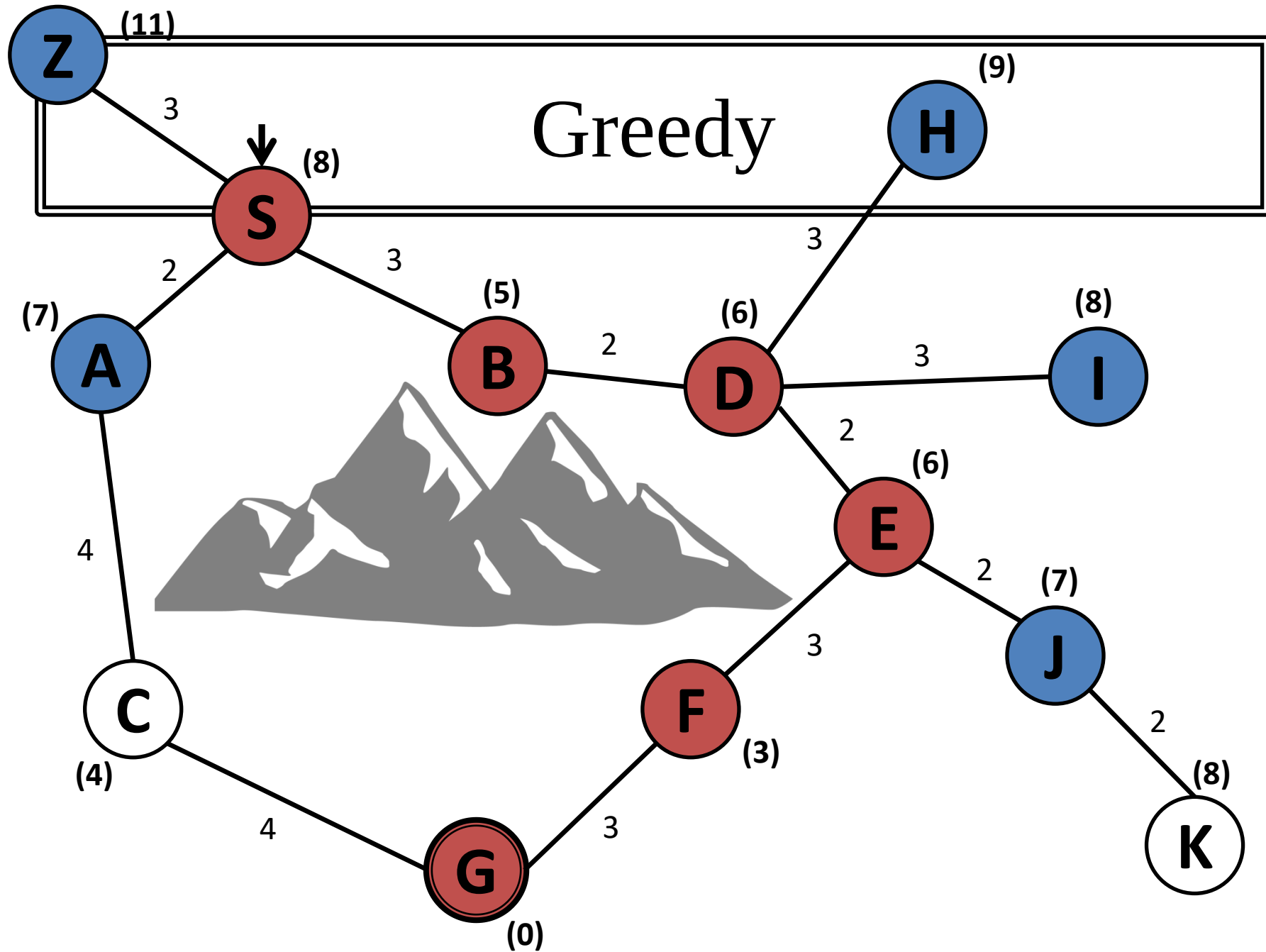


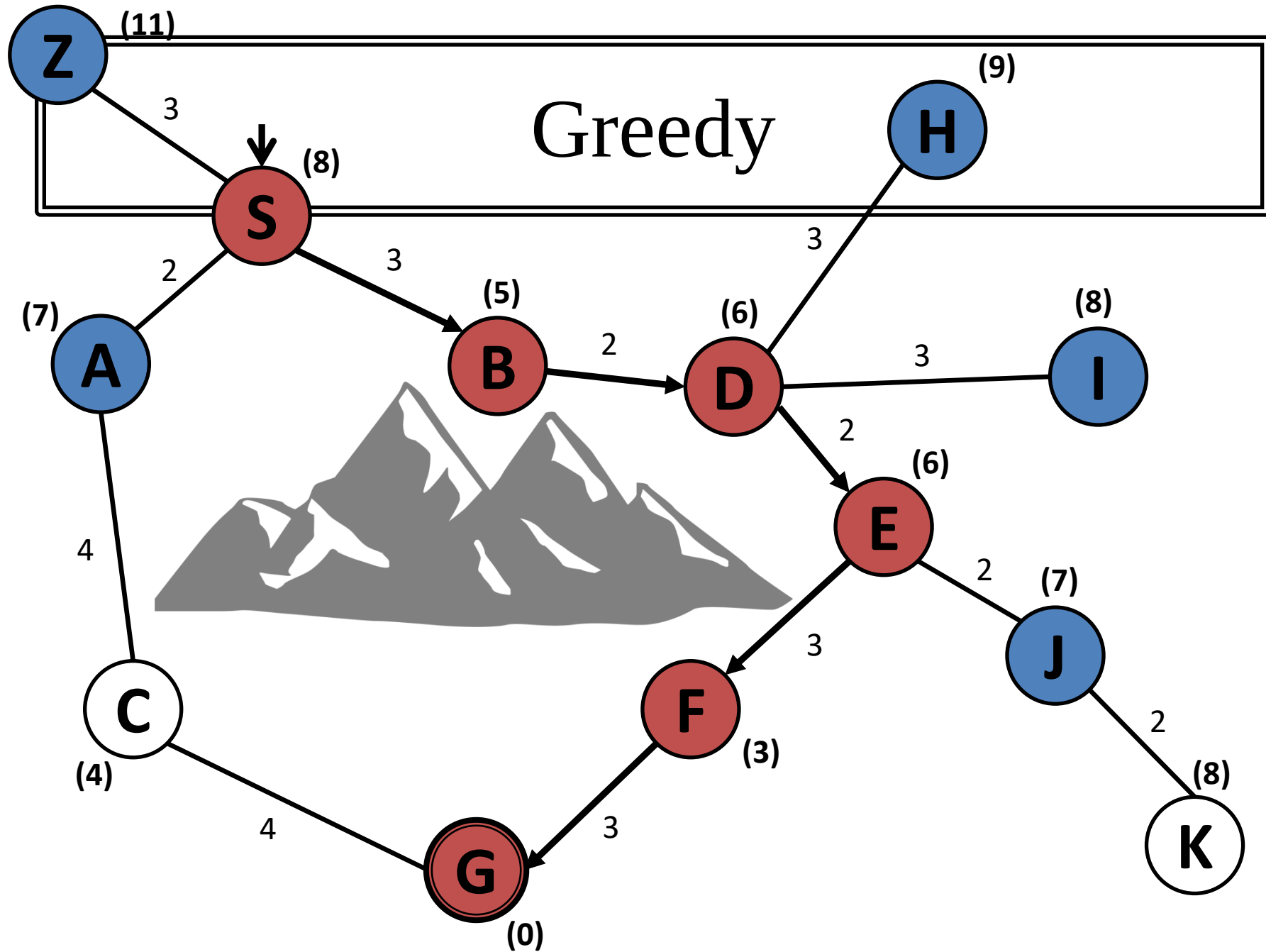








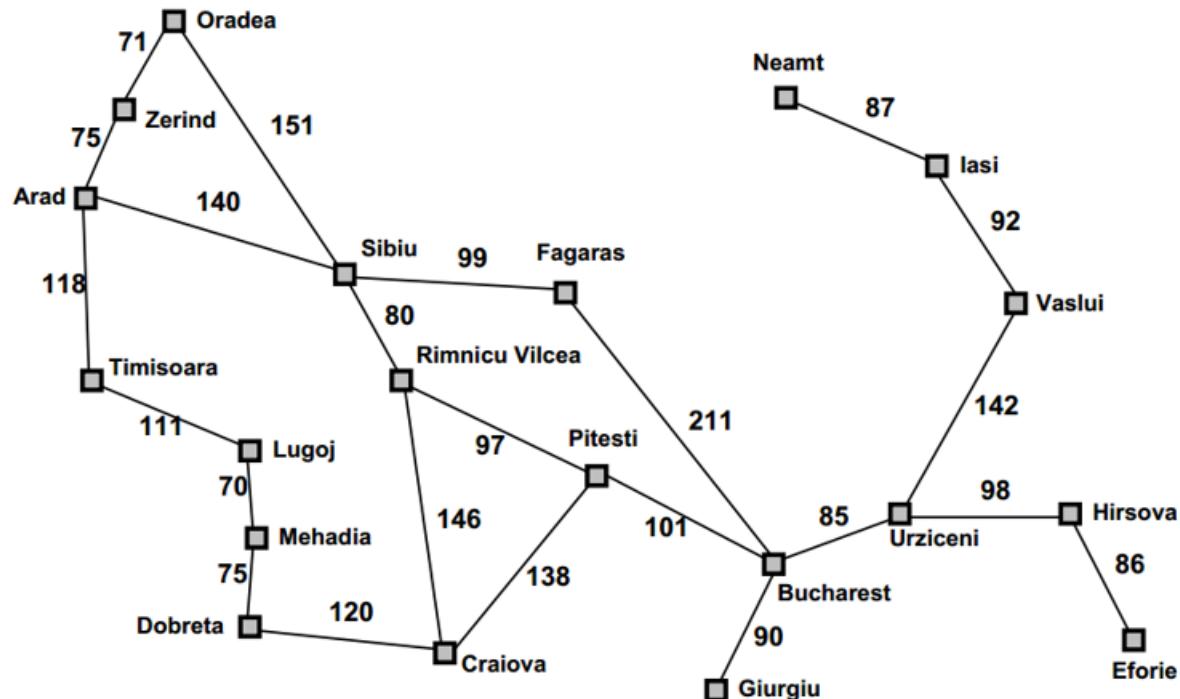




Greedy

- **Complete?**

- **No**, can get stuck in loops, e.g., with Oradea as goal,
Iasi → *Neamt* → *Iasi* → *Neamt* →
Complete in finite space with repeated-state checking

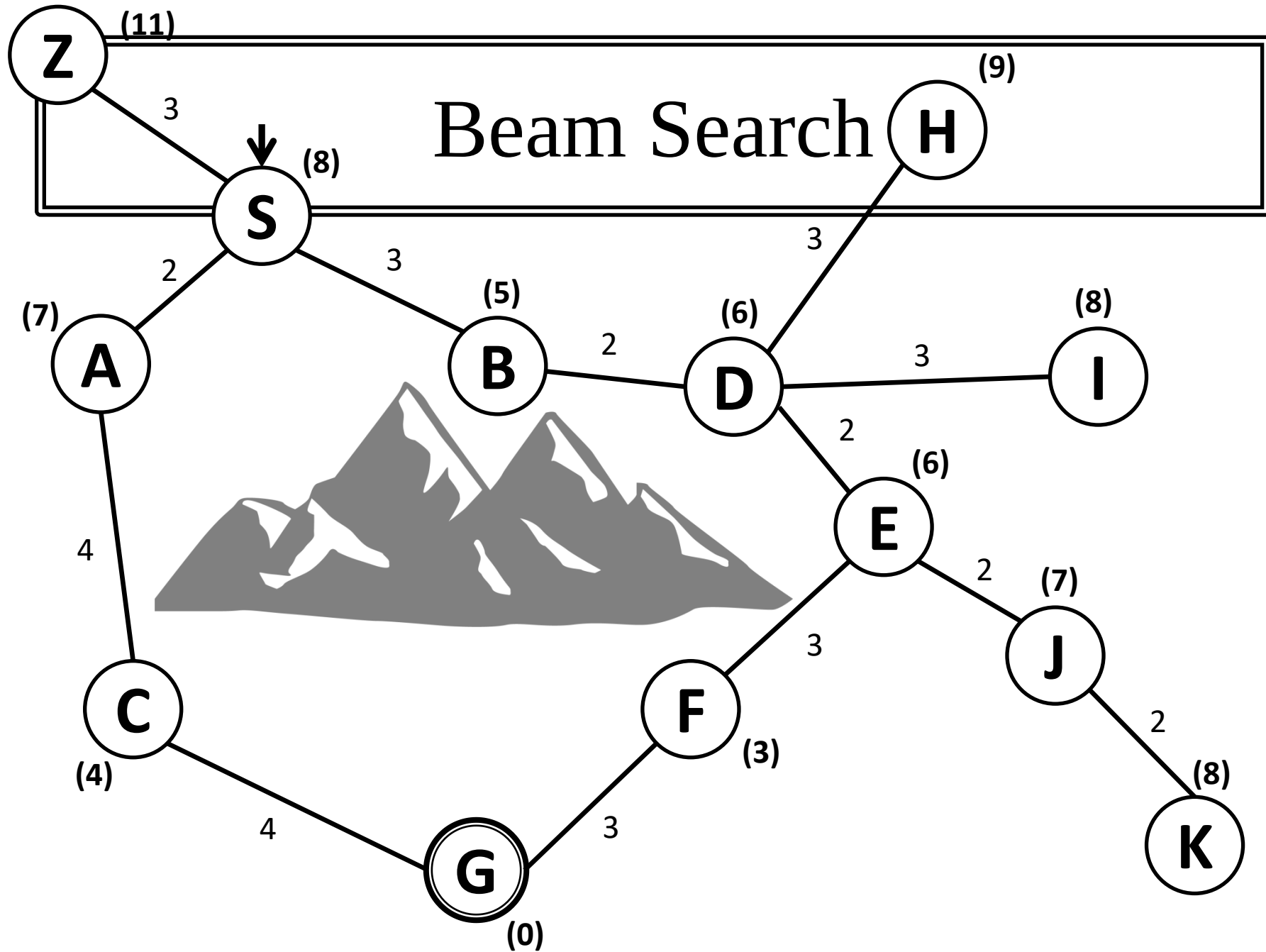


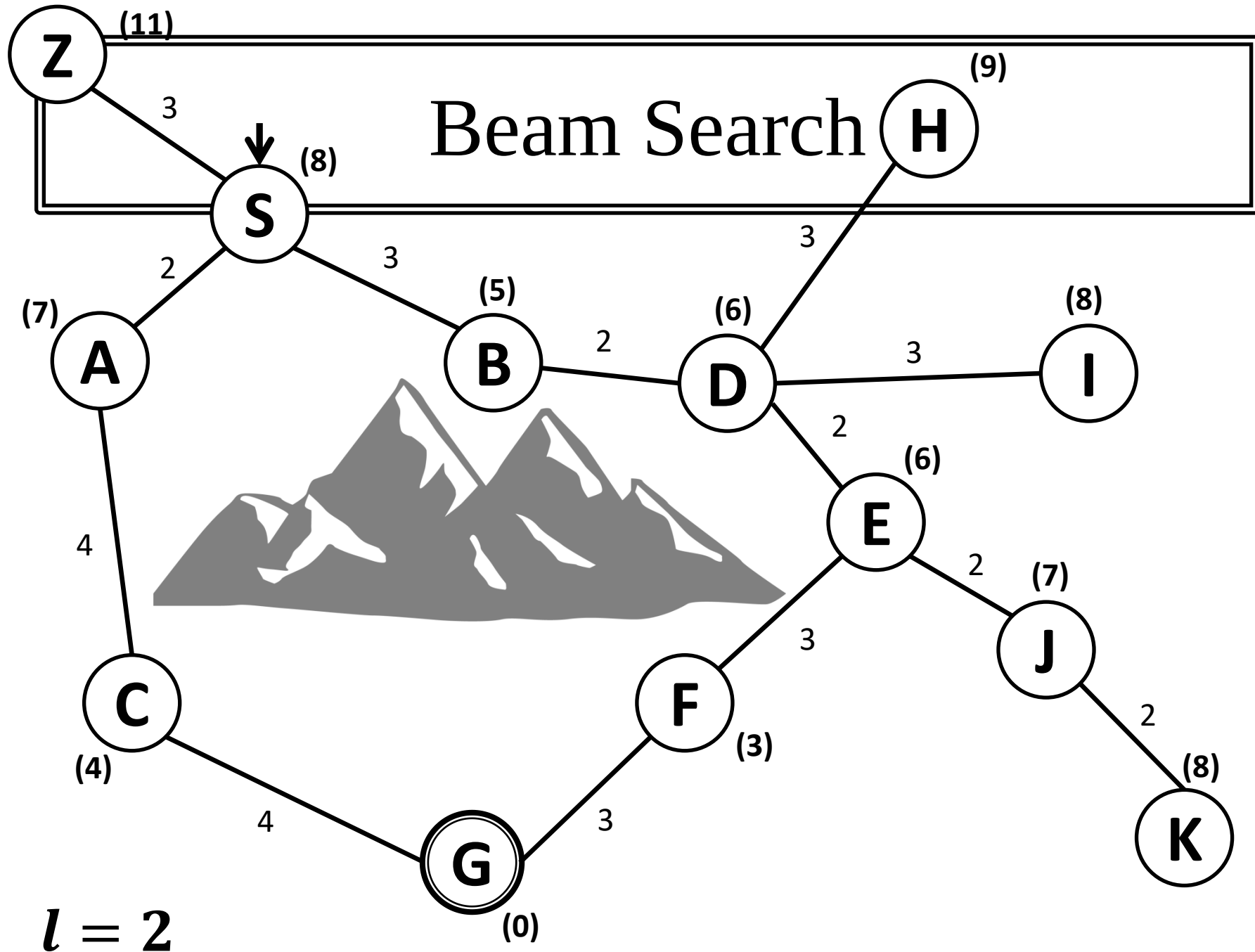
Greedy

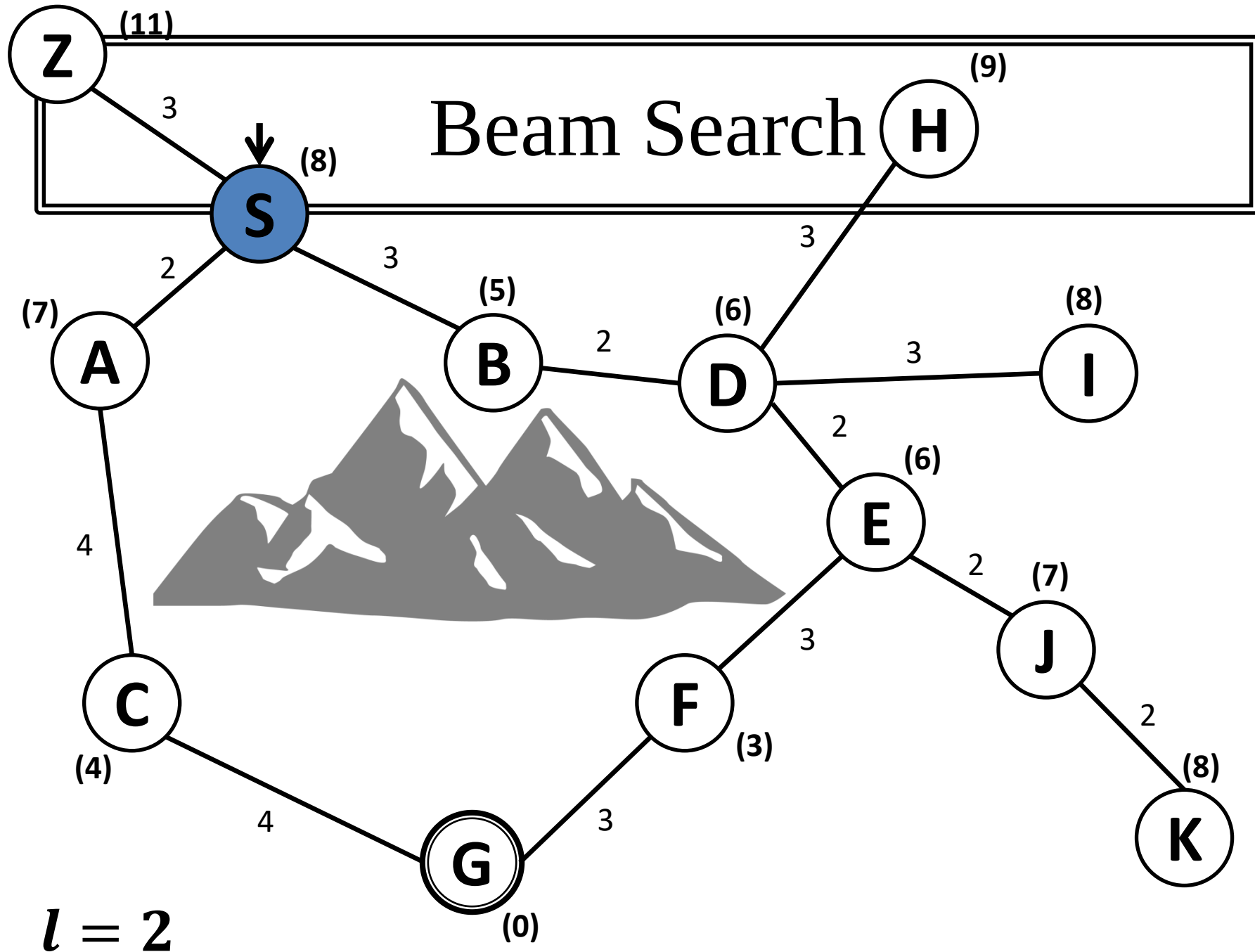
- **Complete?**
 - **No**, can get stuck in loops, e.g., with Oradea as goal,
Iasi → *Neamt* → *Iasi* → *Neamt* →
Complete in finite space with repeated-state checking
- **Optimal?**
 - **No**
- **Time?**
 - $O(b^m)$, but a good heuristic can give dramatic improvement
- **Space?**
 - $O(b^m)$ - keeps all nodes in memory

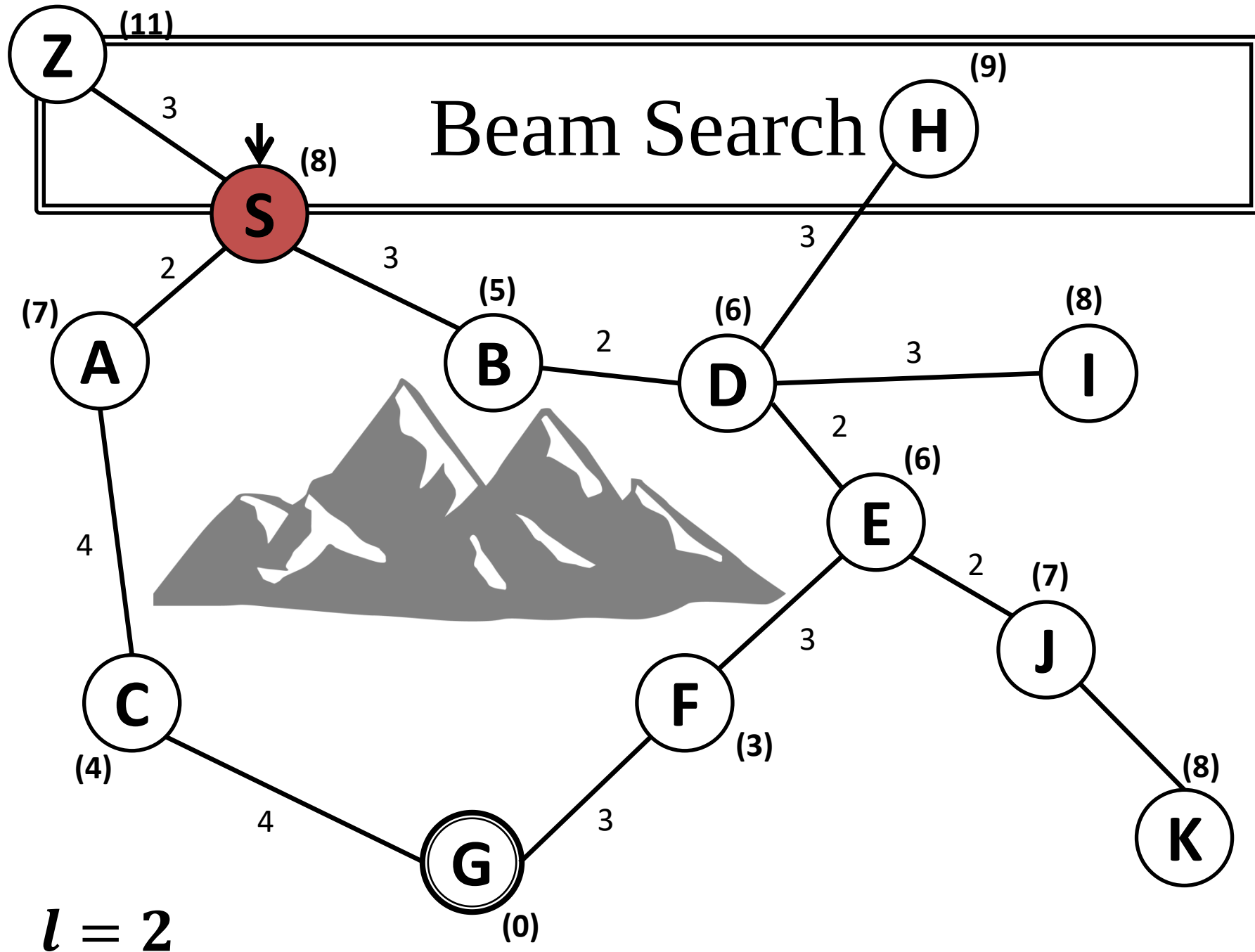
Beam Search

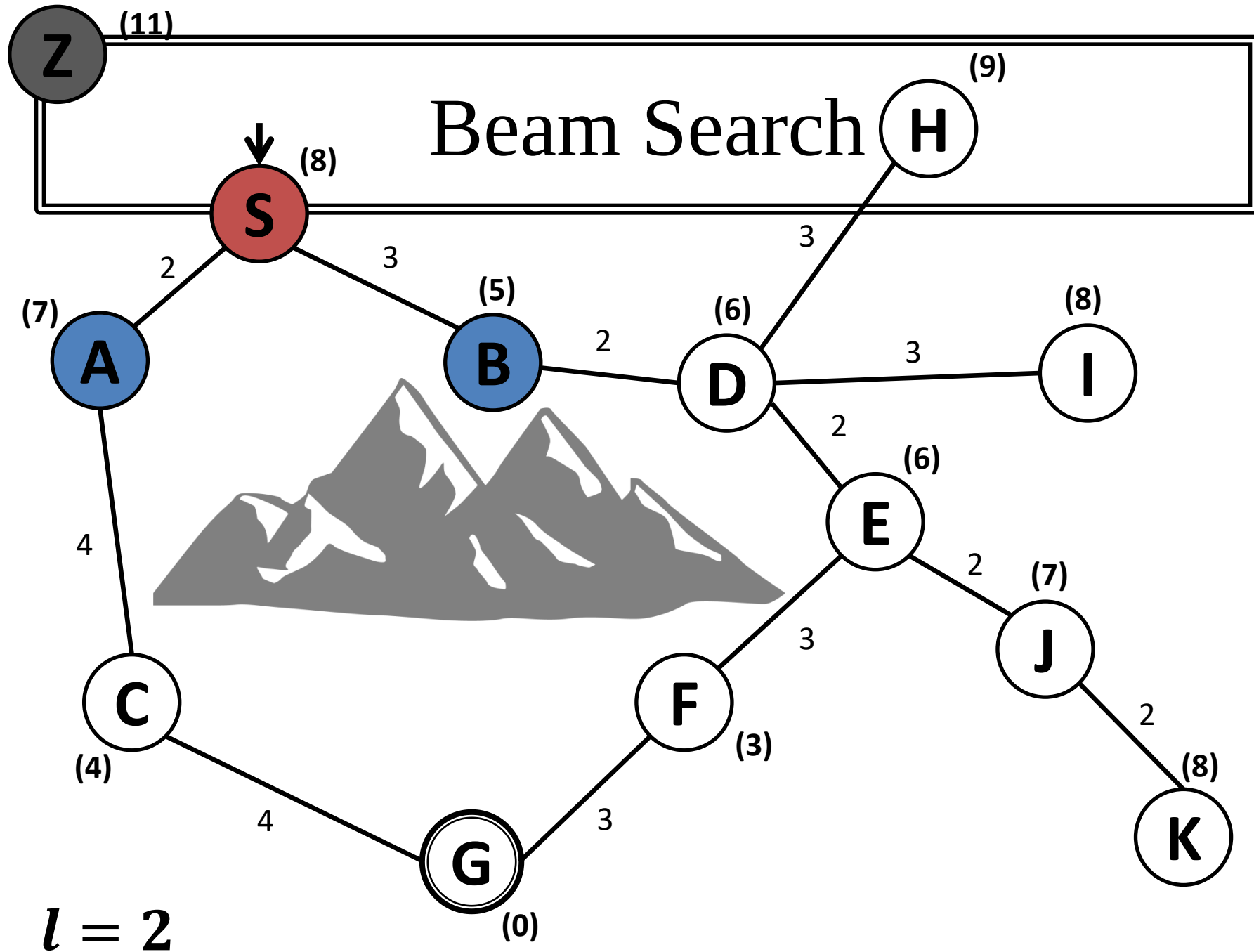
- = *greedy best-first search* with queue limit l ,
i.e., keeps l nodes in the queue

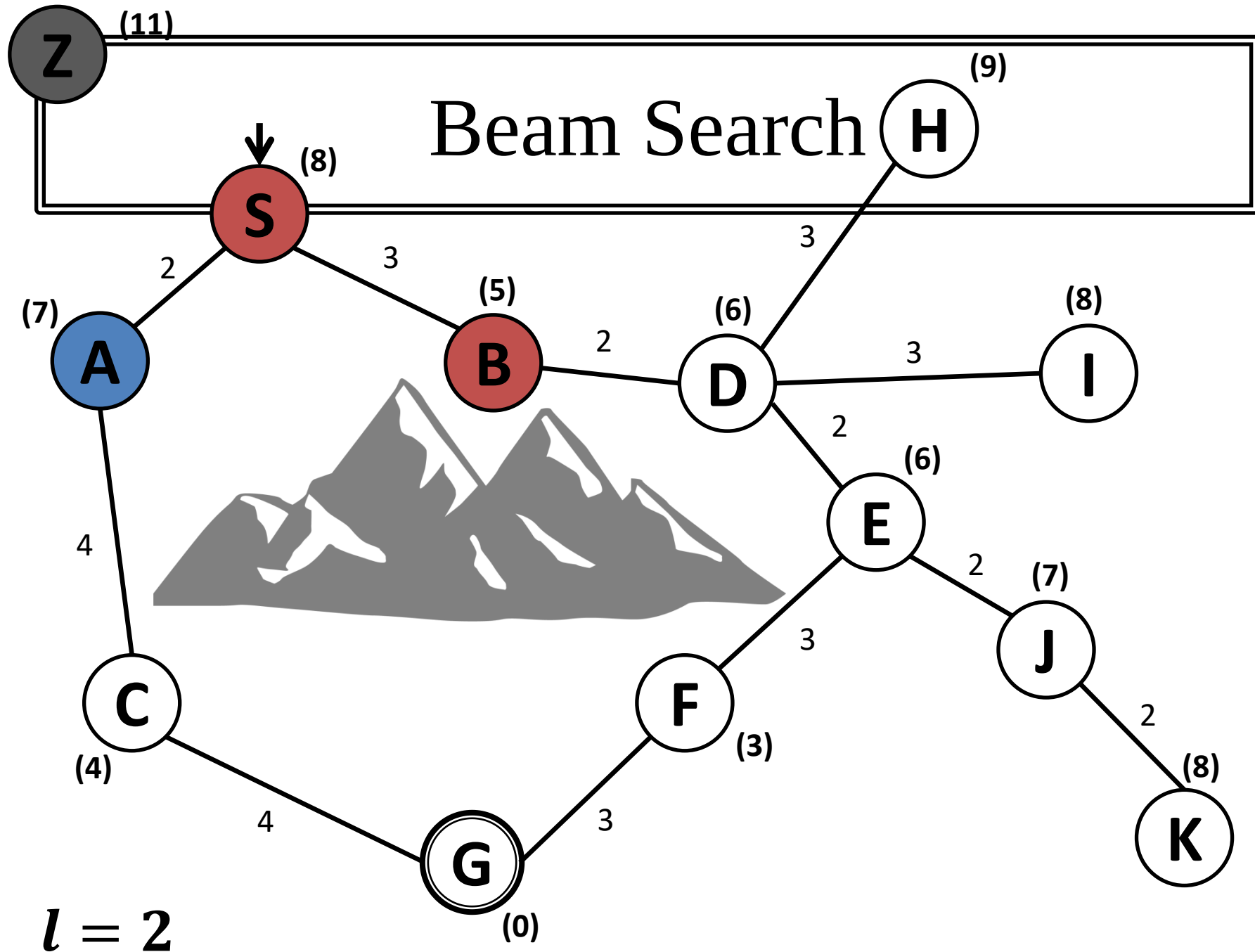


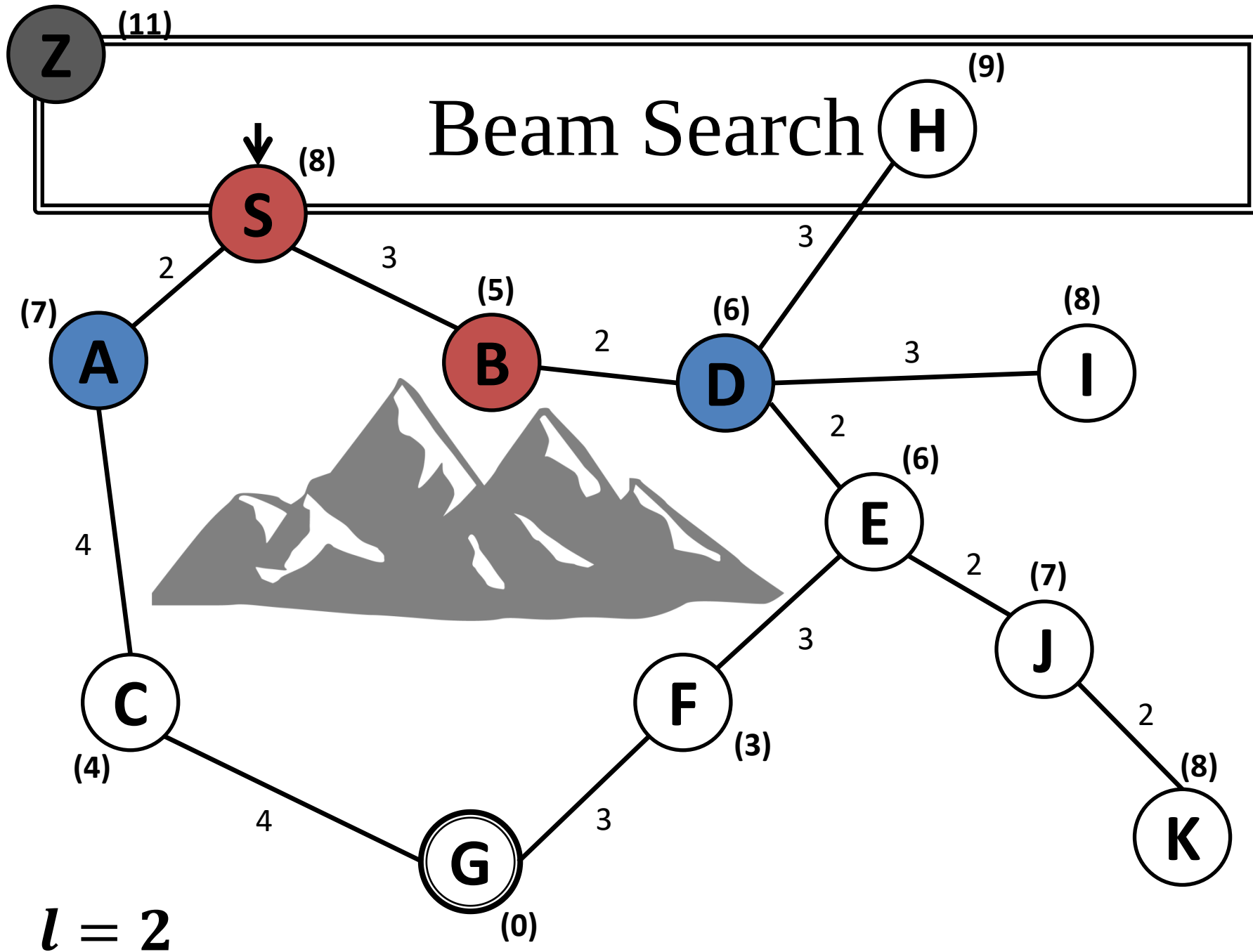


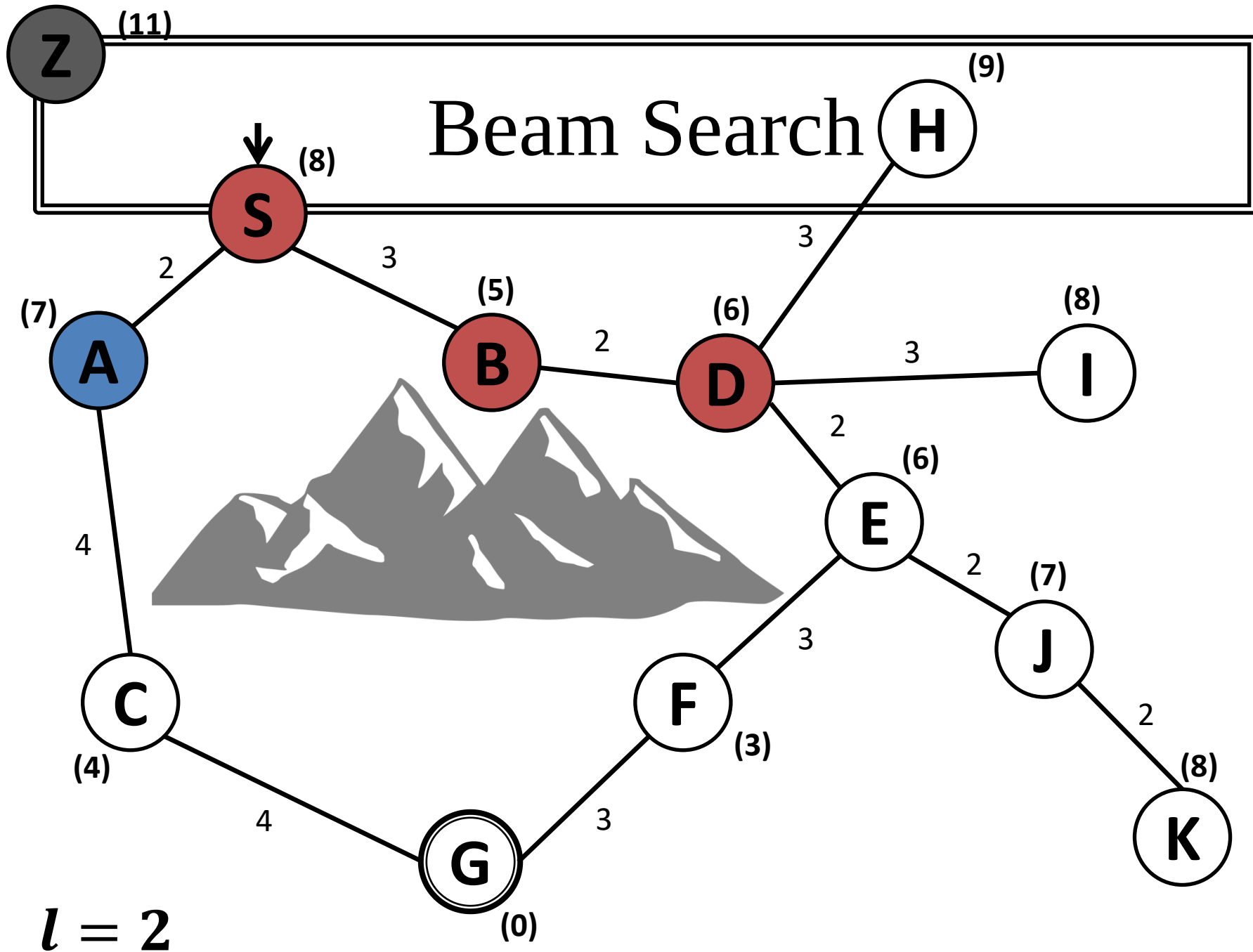


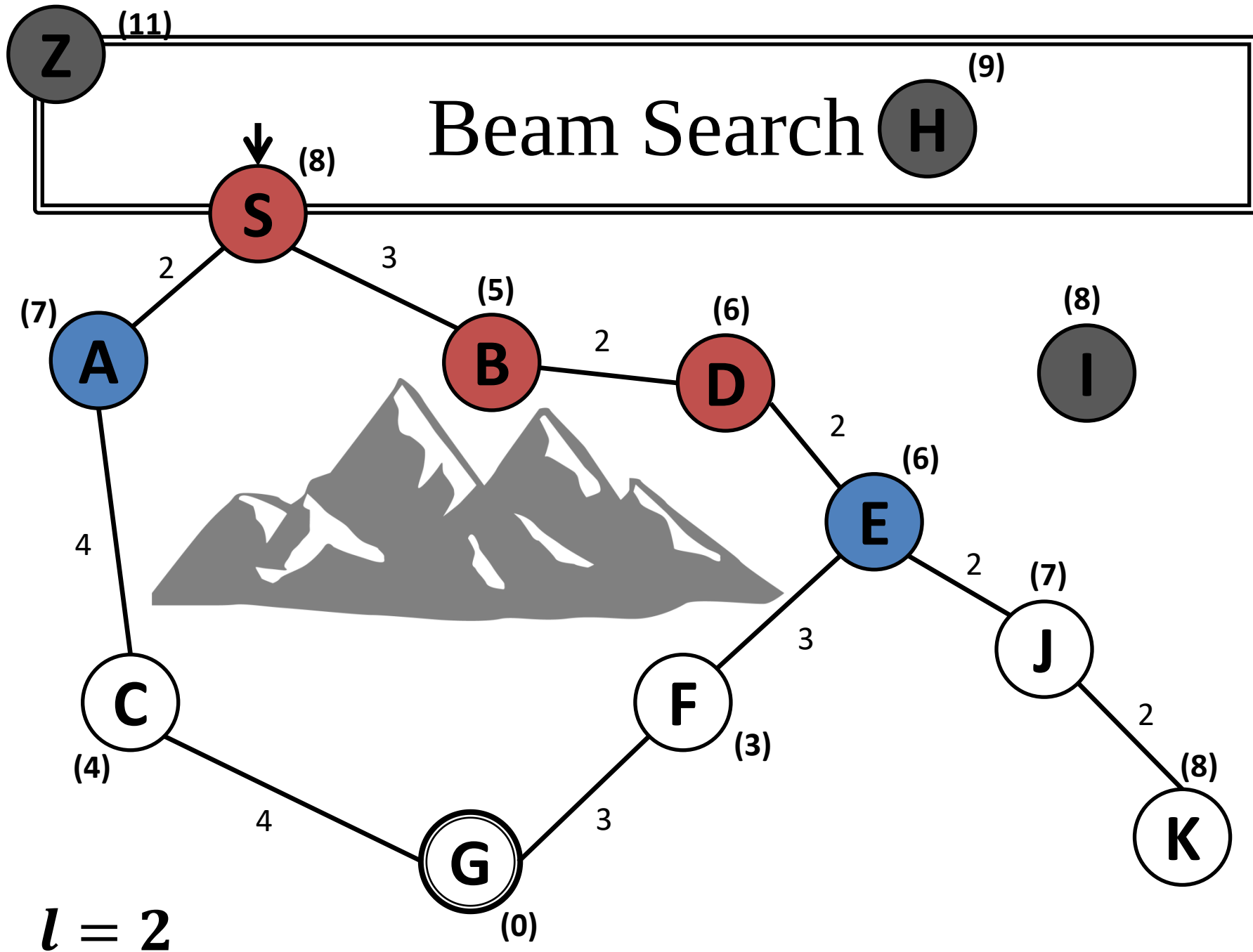


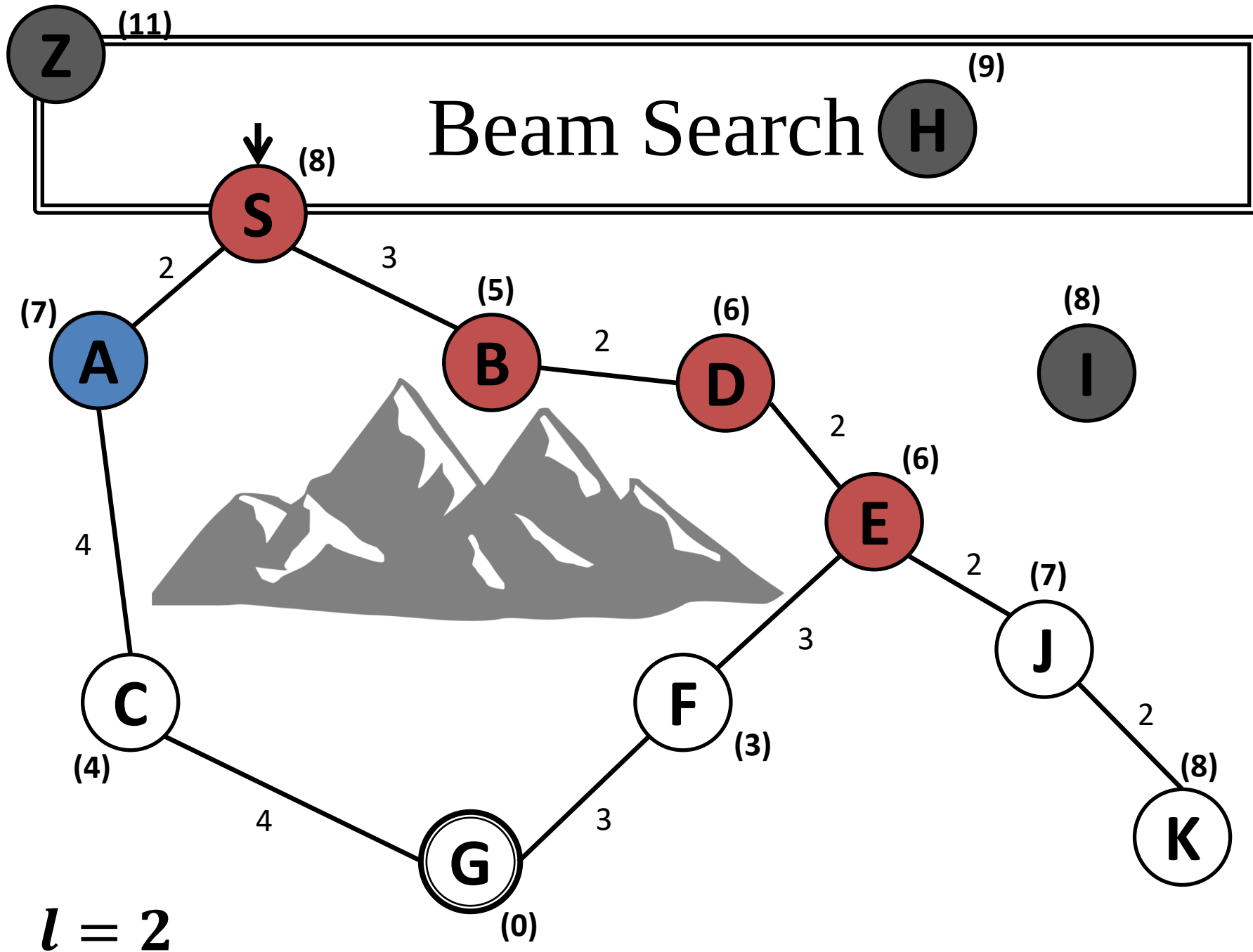


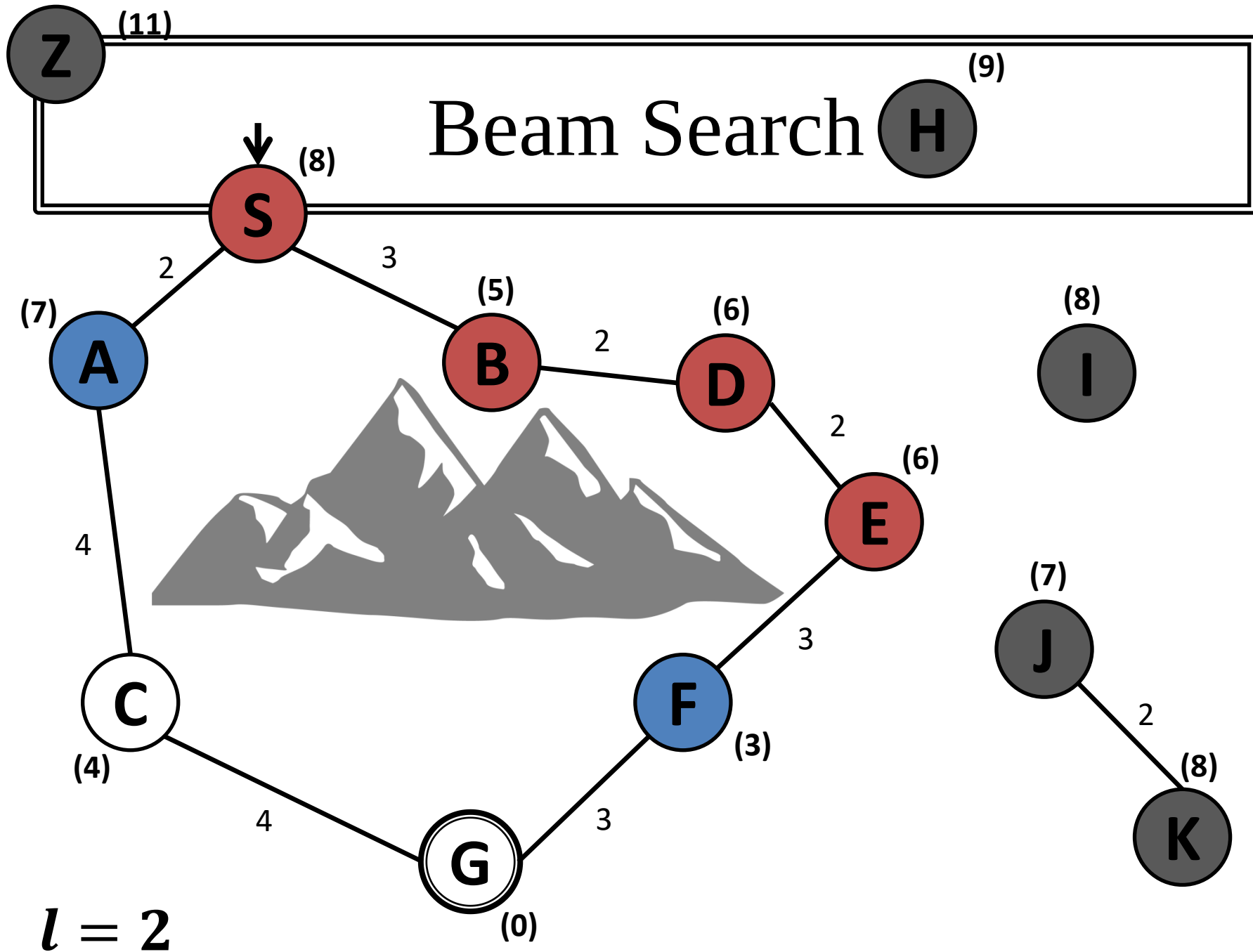


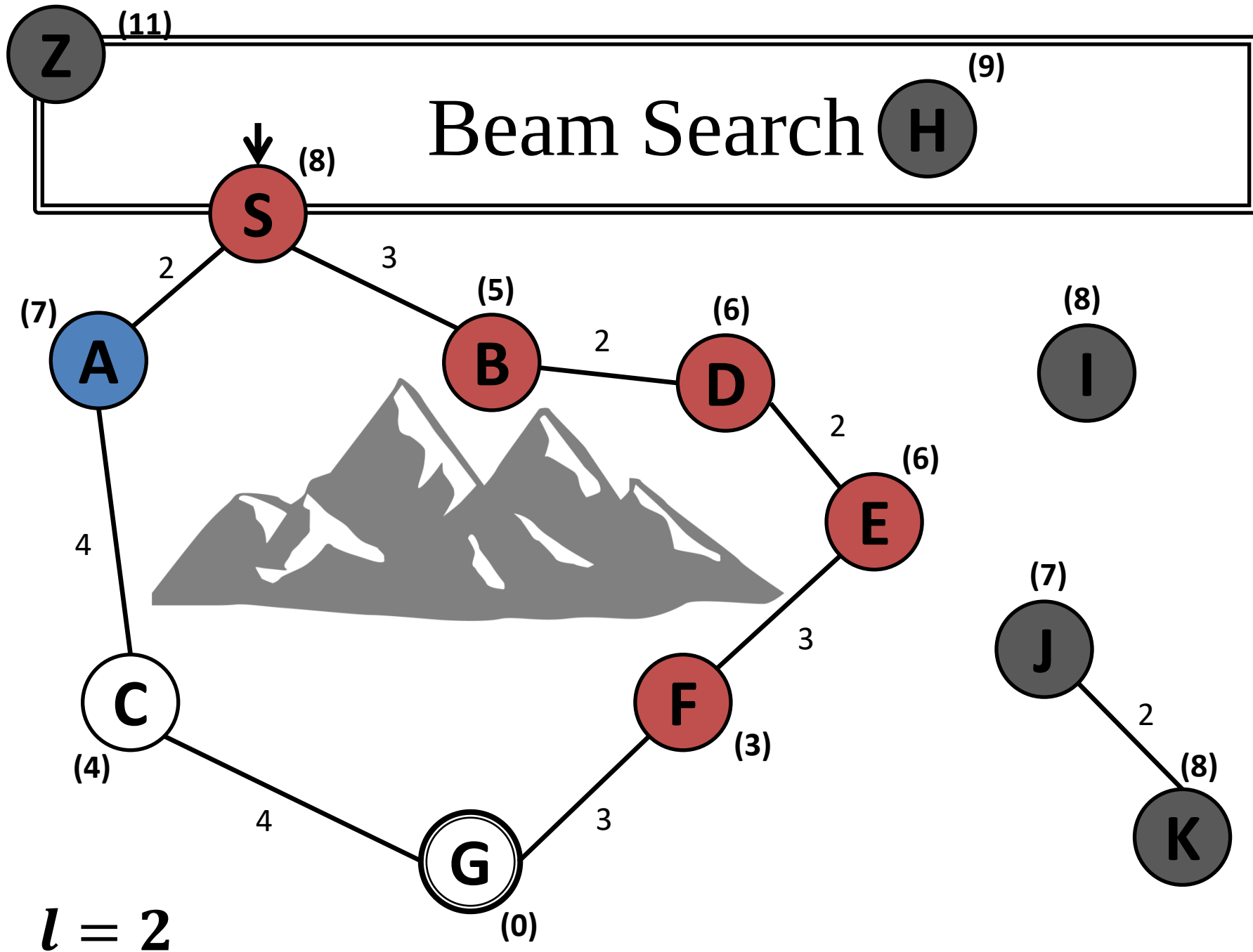


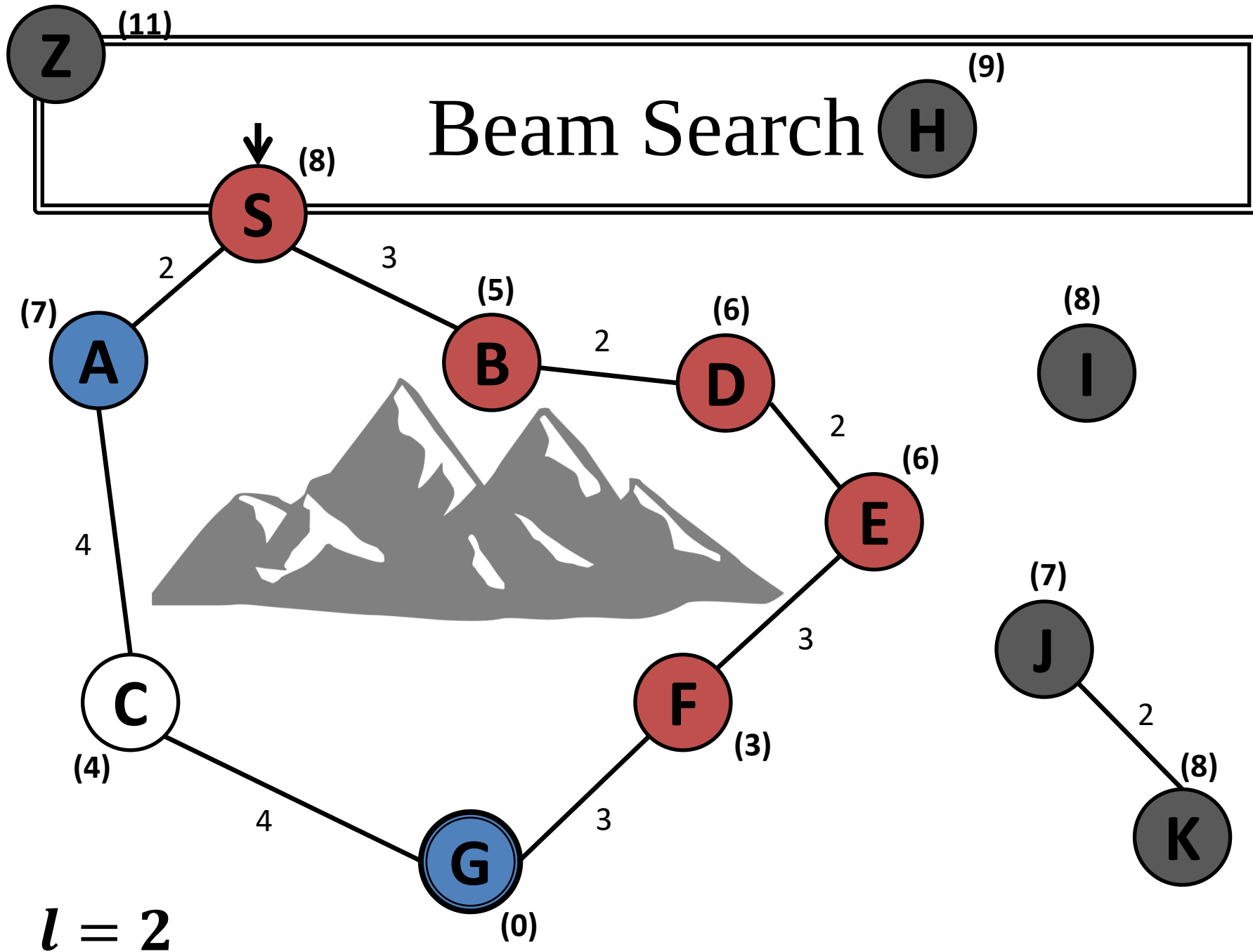


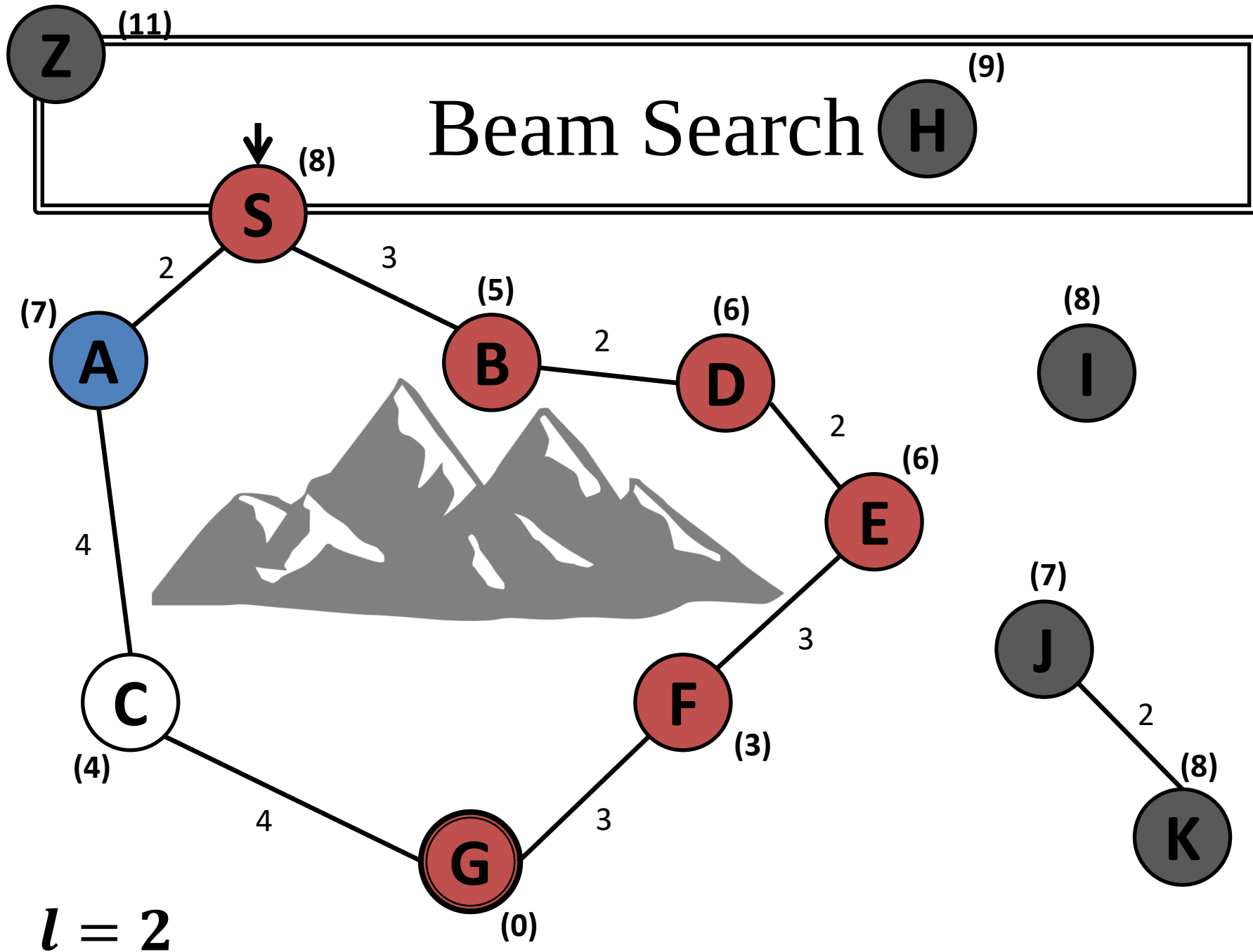


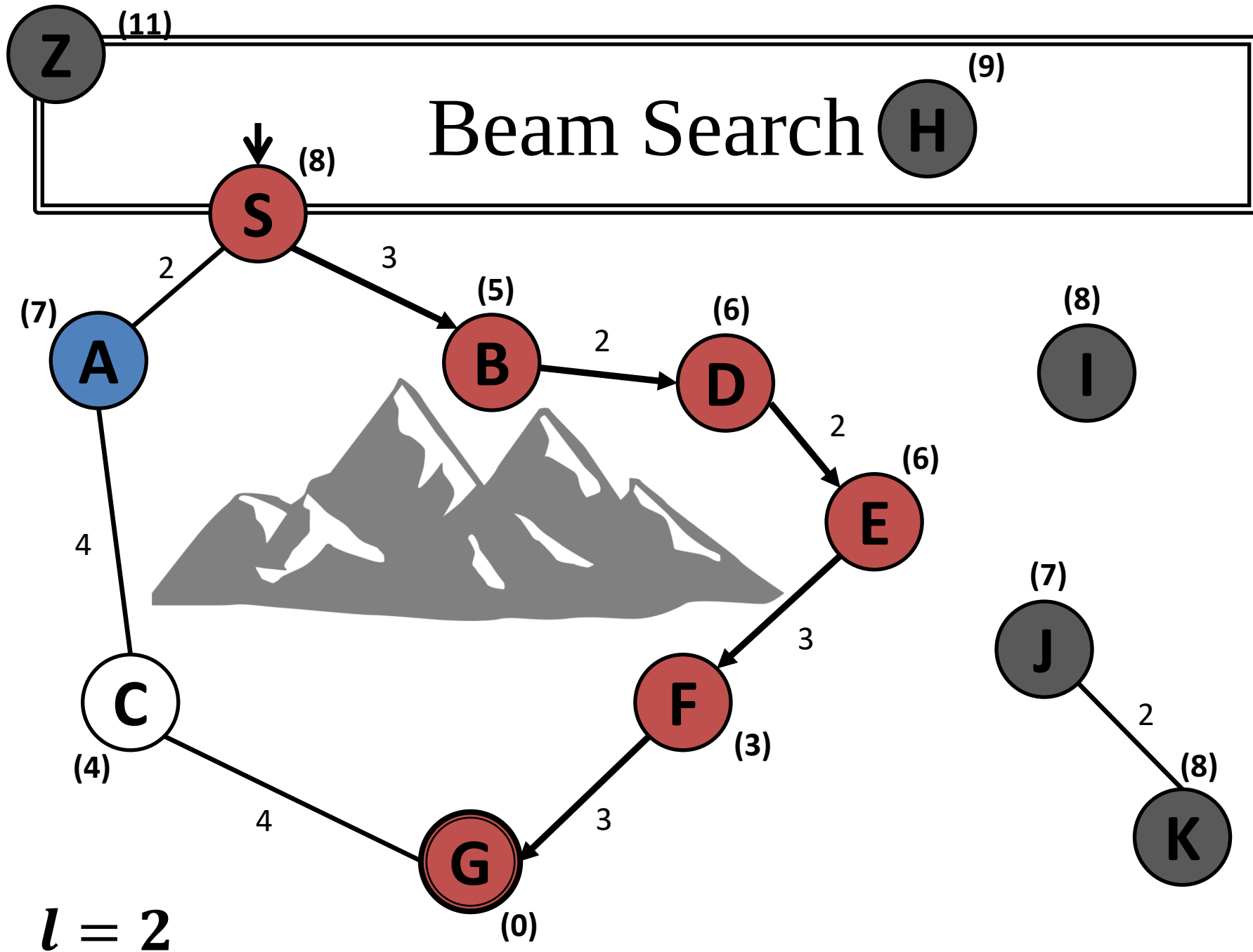












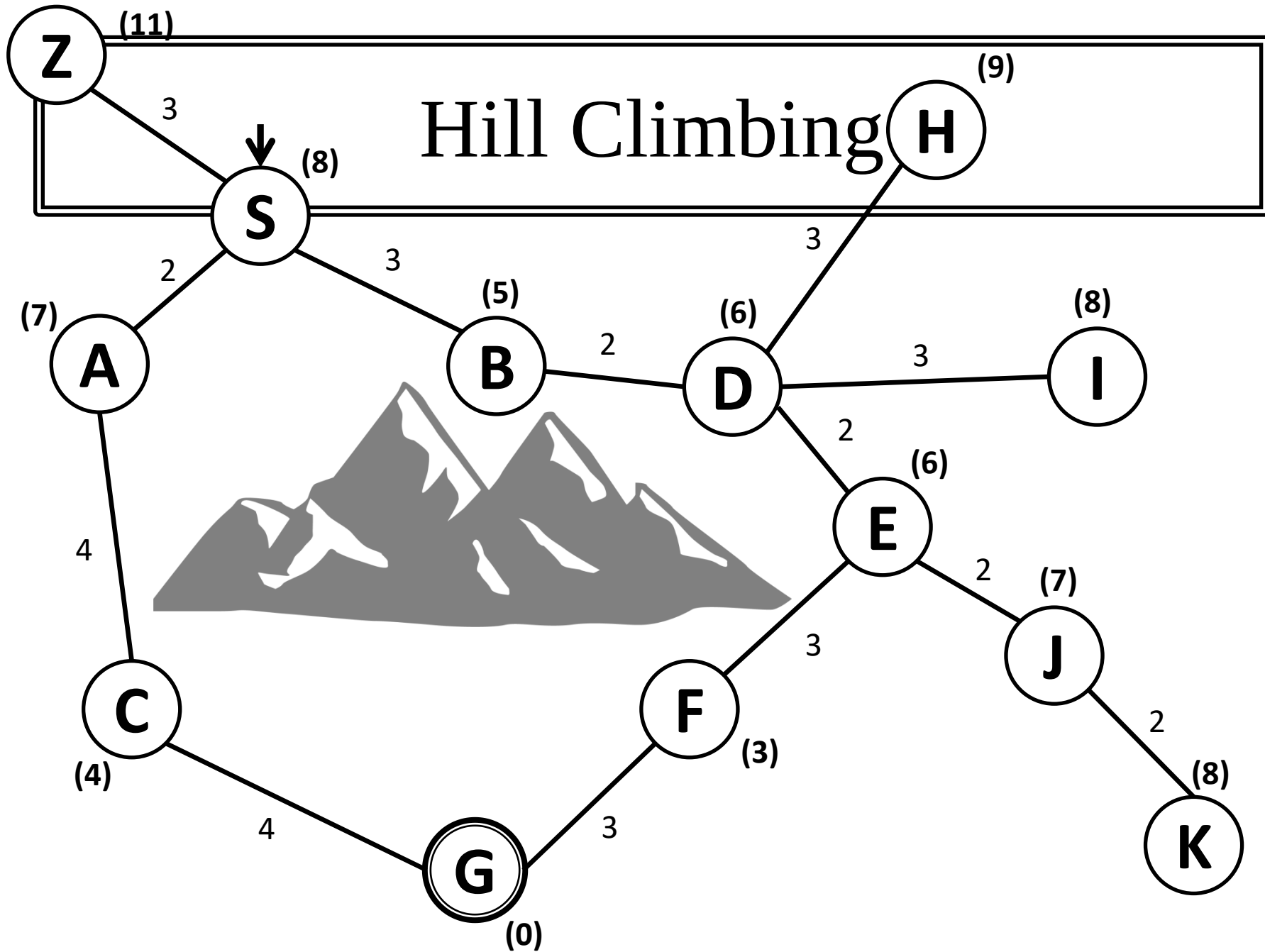
Beam Search

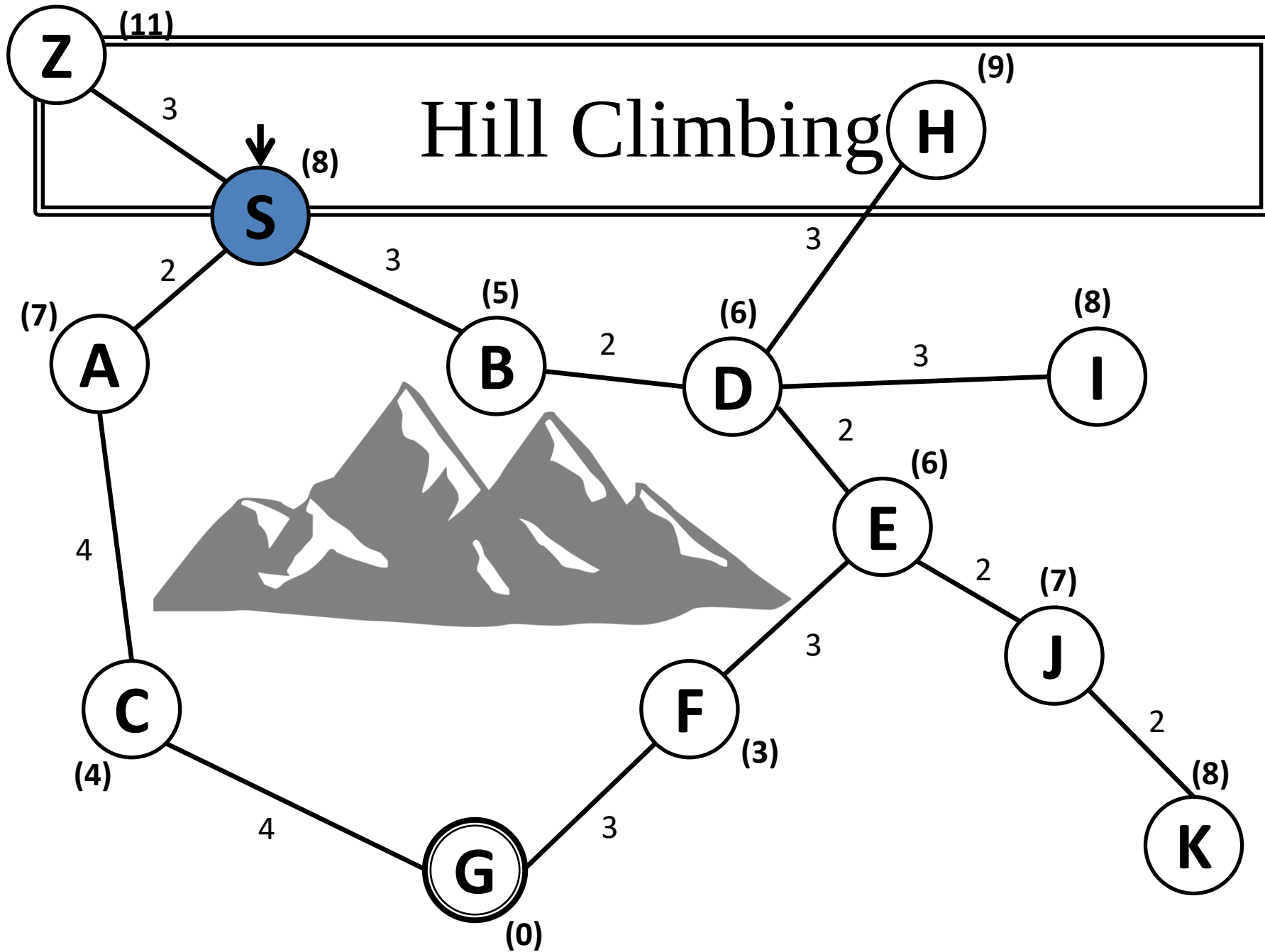
- **Complete?**
 - **No**, local search
- **Optimal?**
 - **No**
- **Time?**
 - **$O(blm)$** – linear time
- **Space?**
 - **$O(bl)$** – linear space

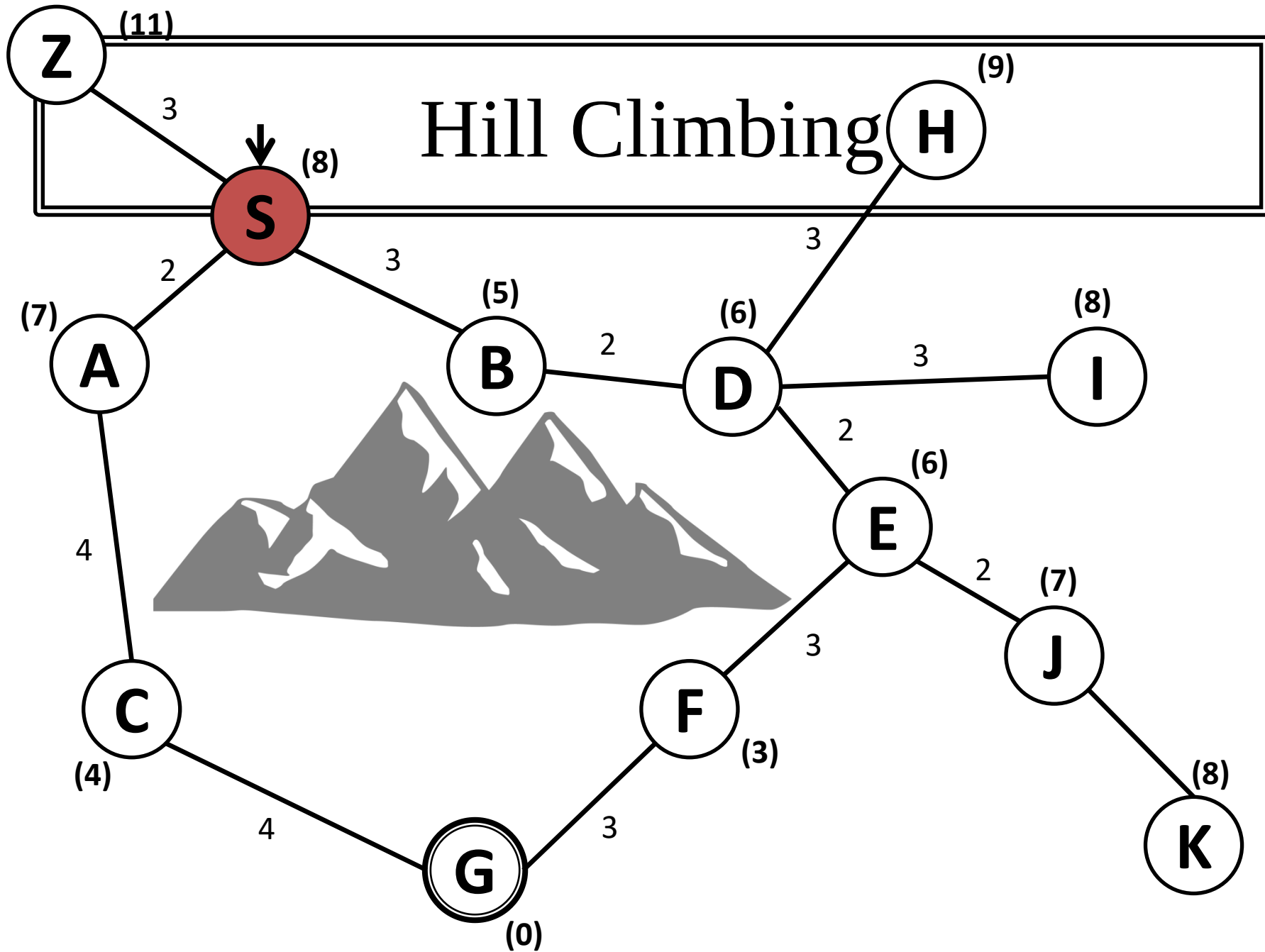
* In local search optimization, this approach is known as **Local Beam Search**.

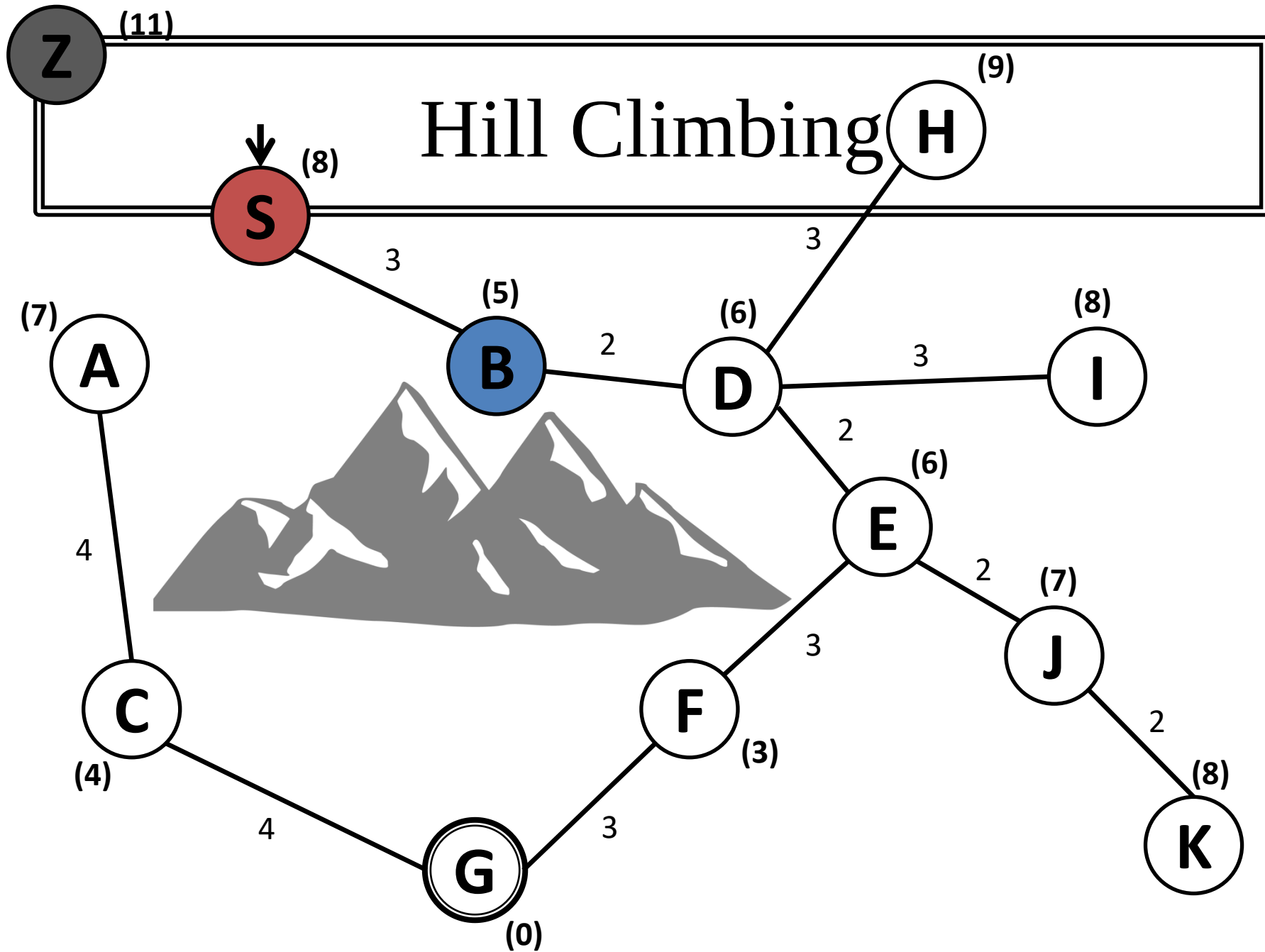
Hill Climbing

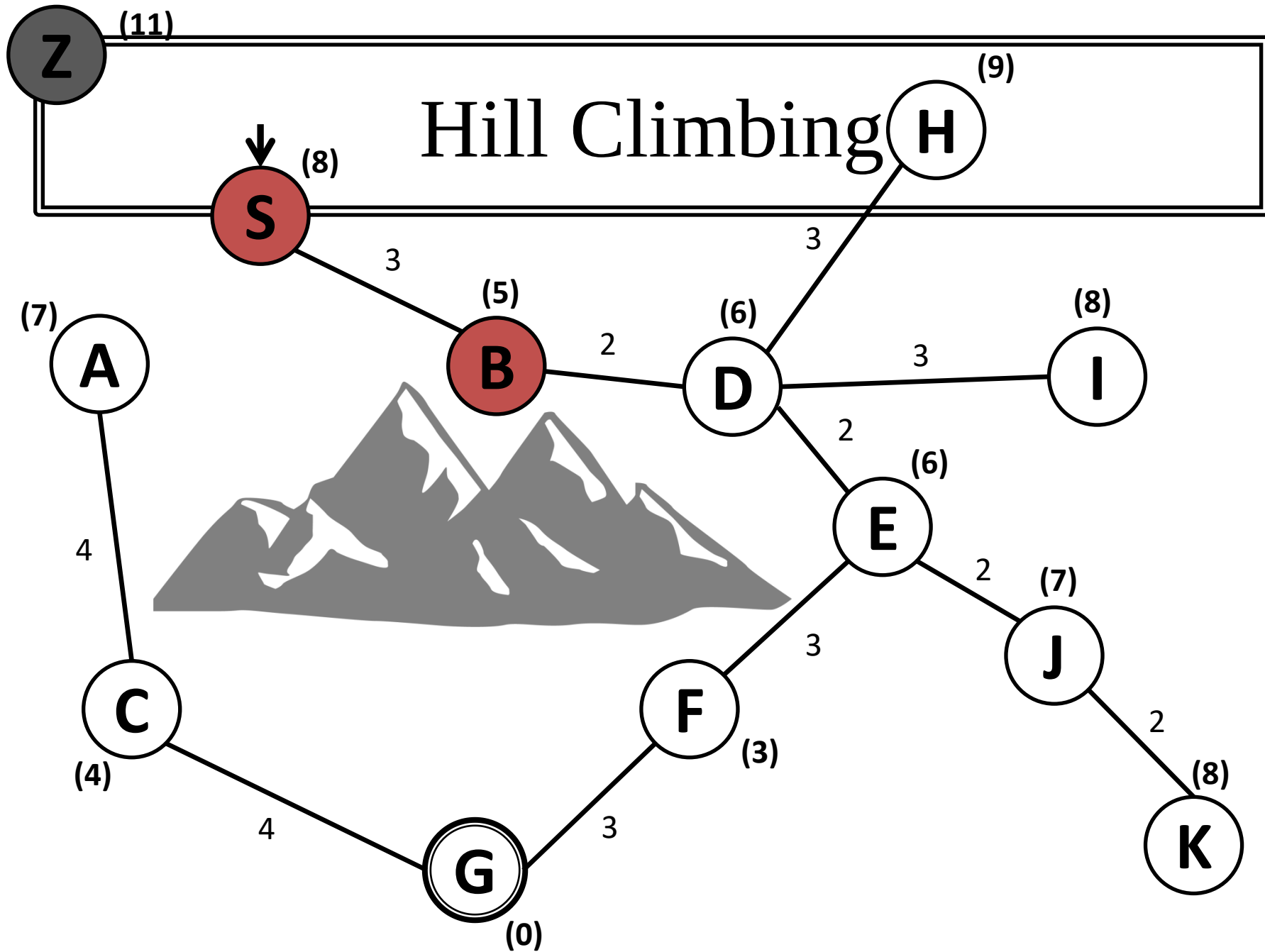
- = *greedy best-first search* with queue limit $l = 1$,
i.e., keeps only the best node

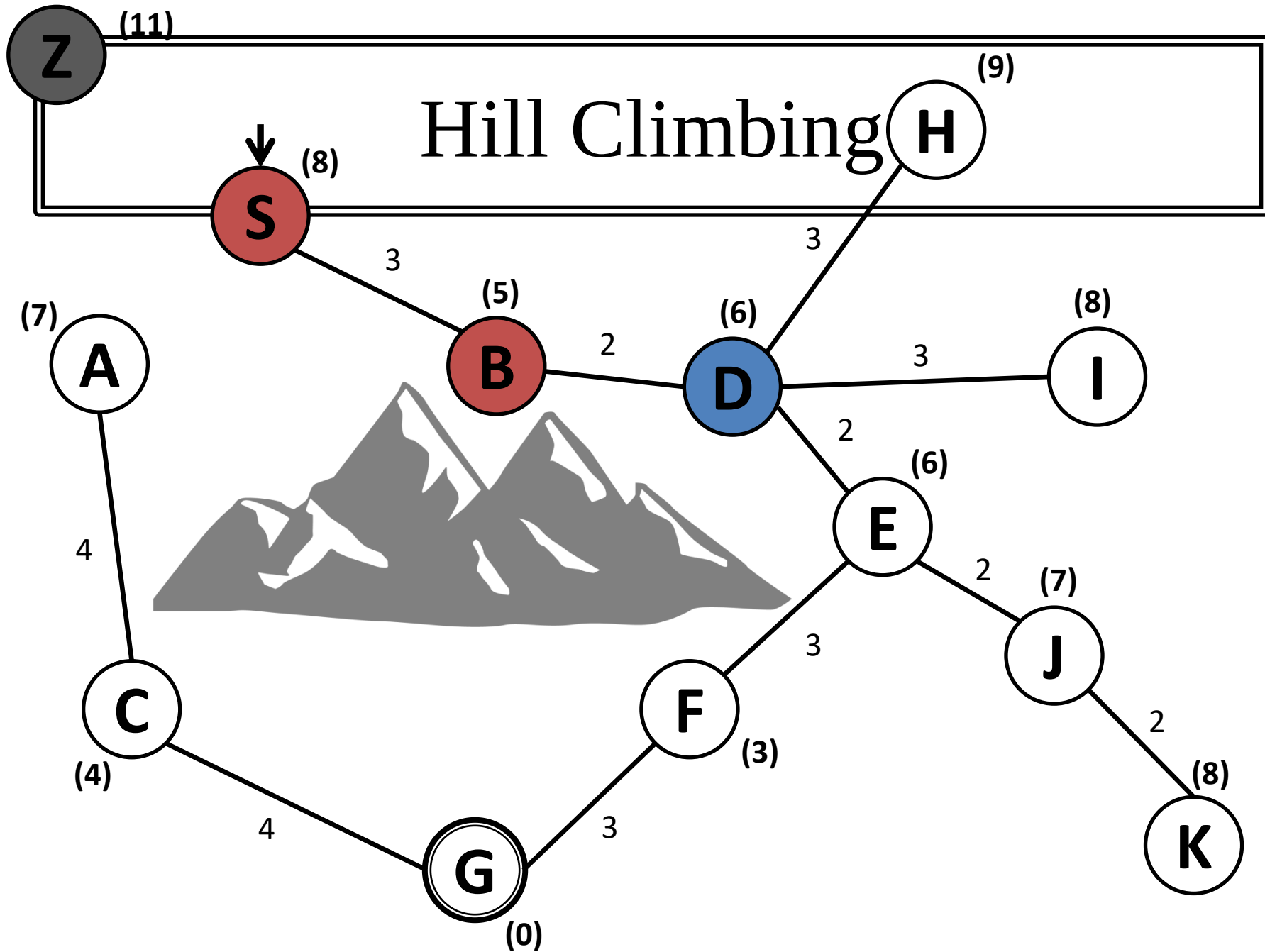


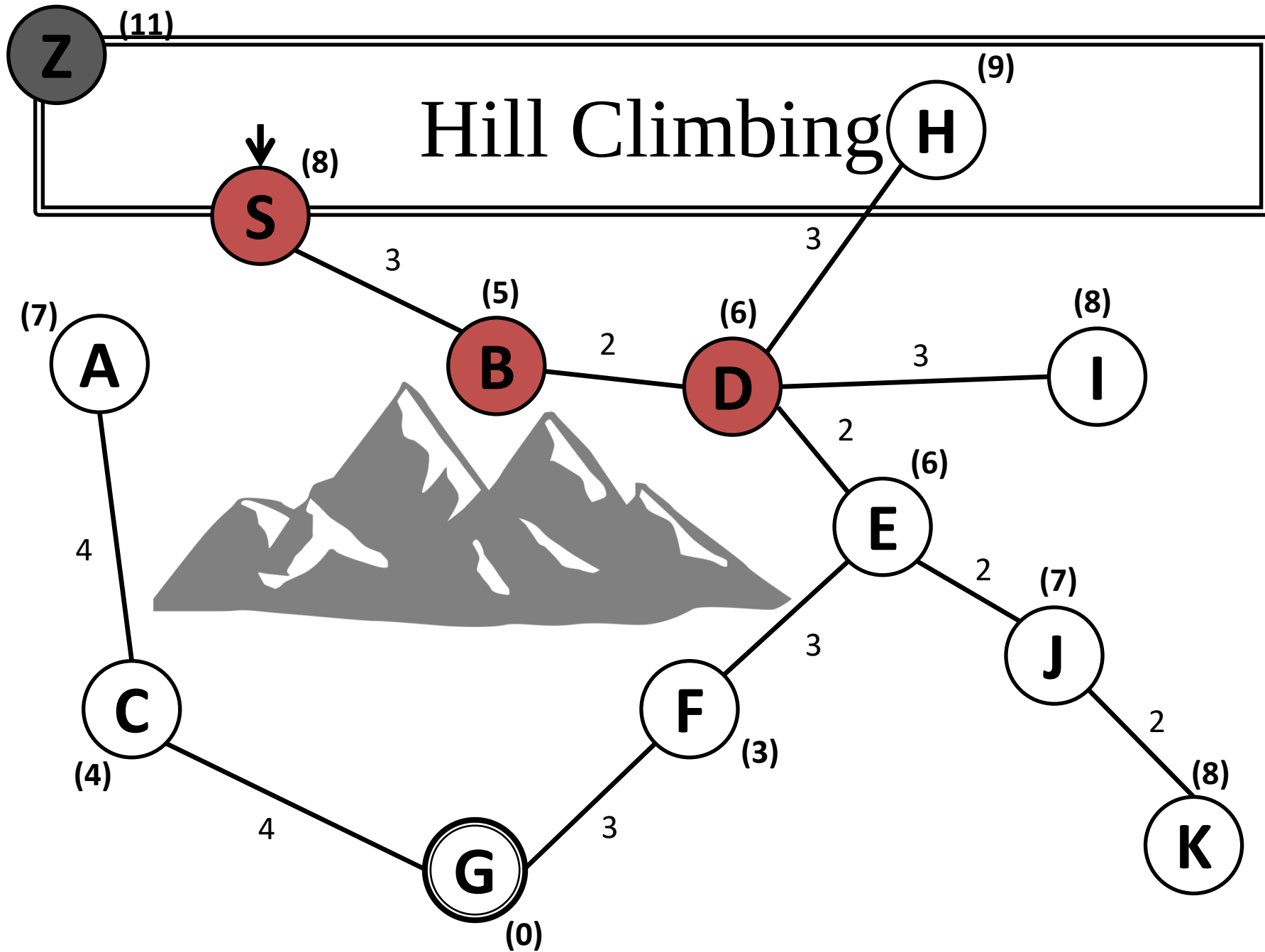


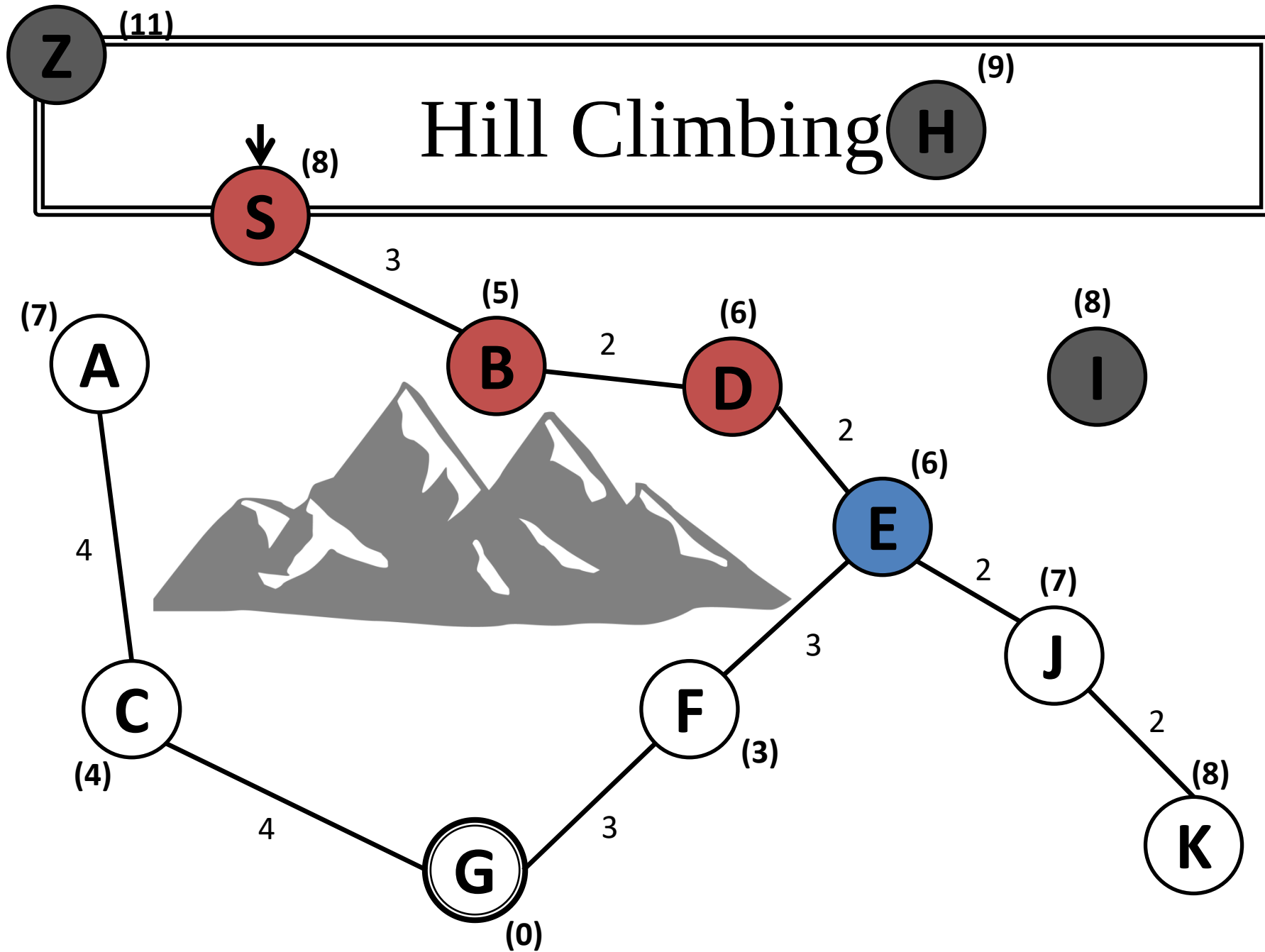


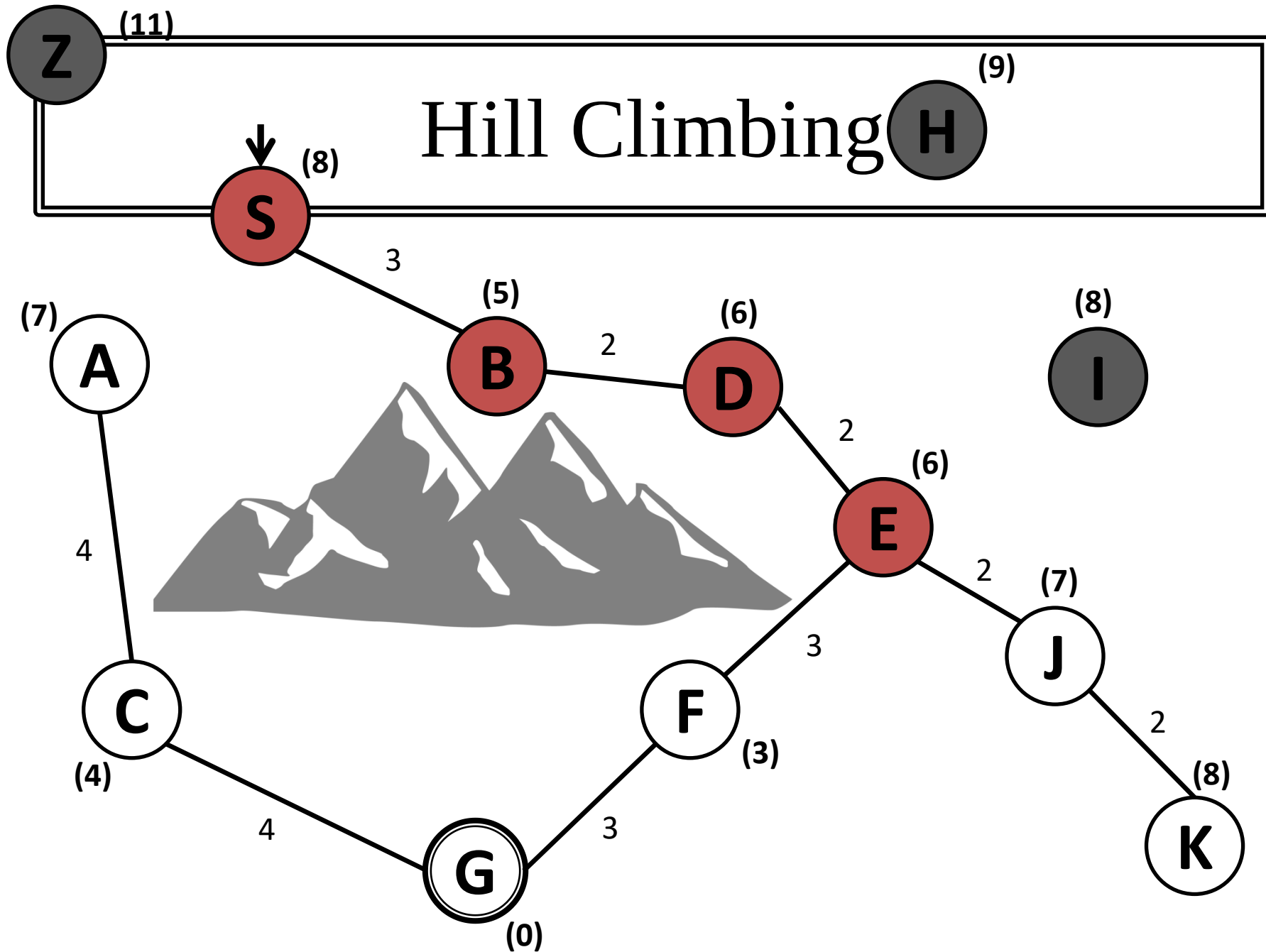


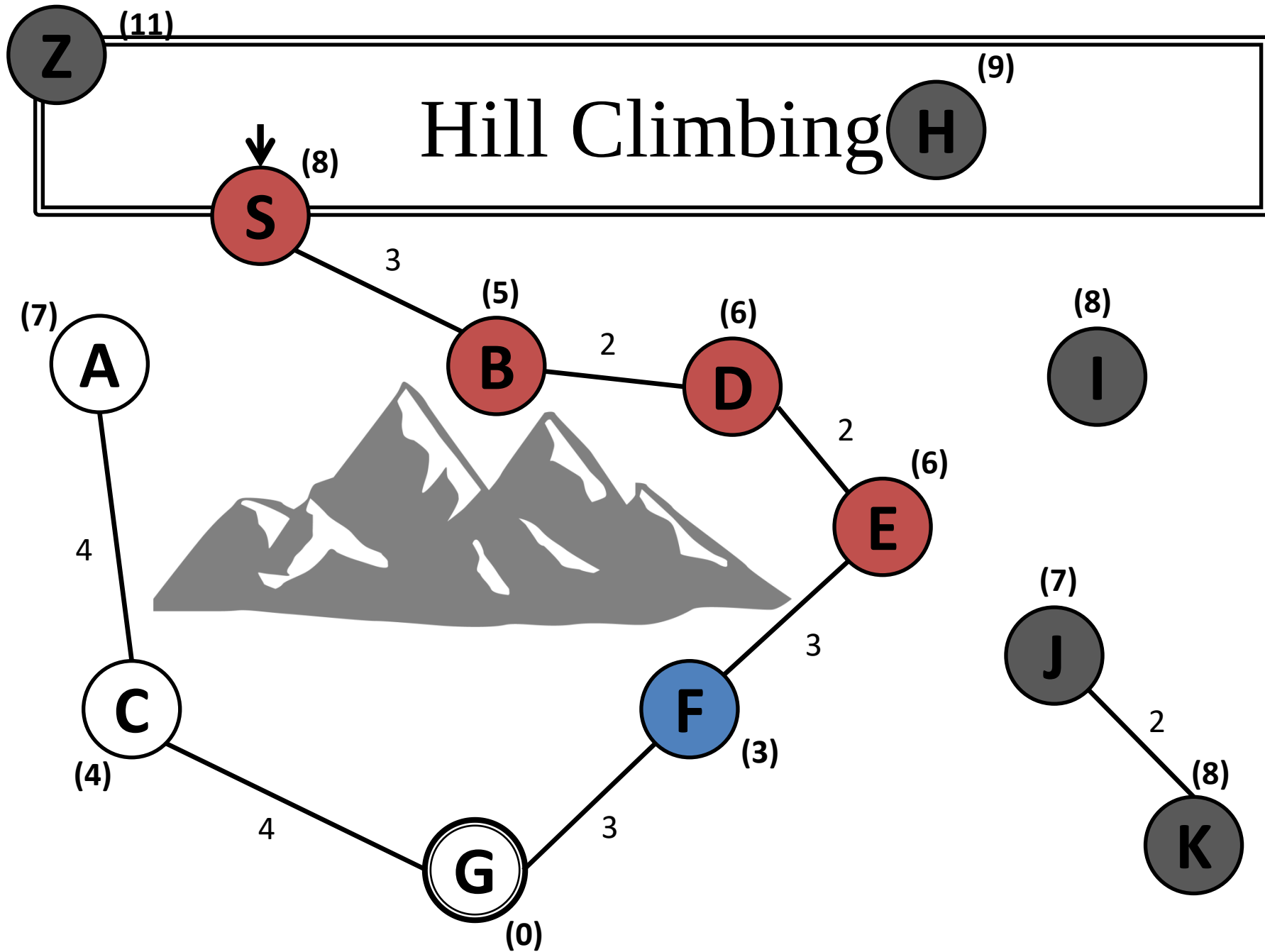


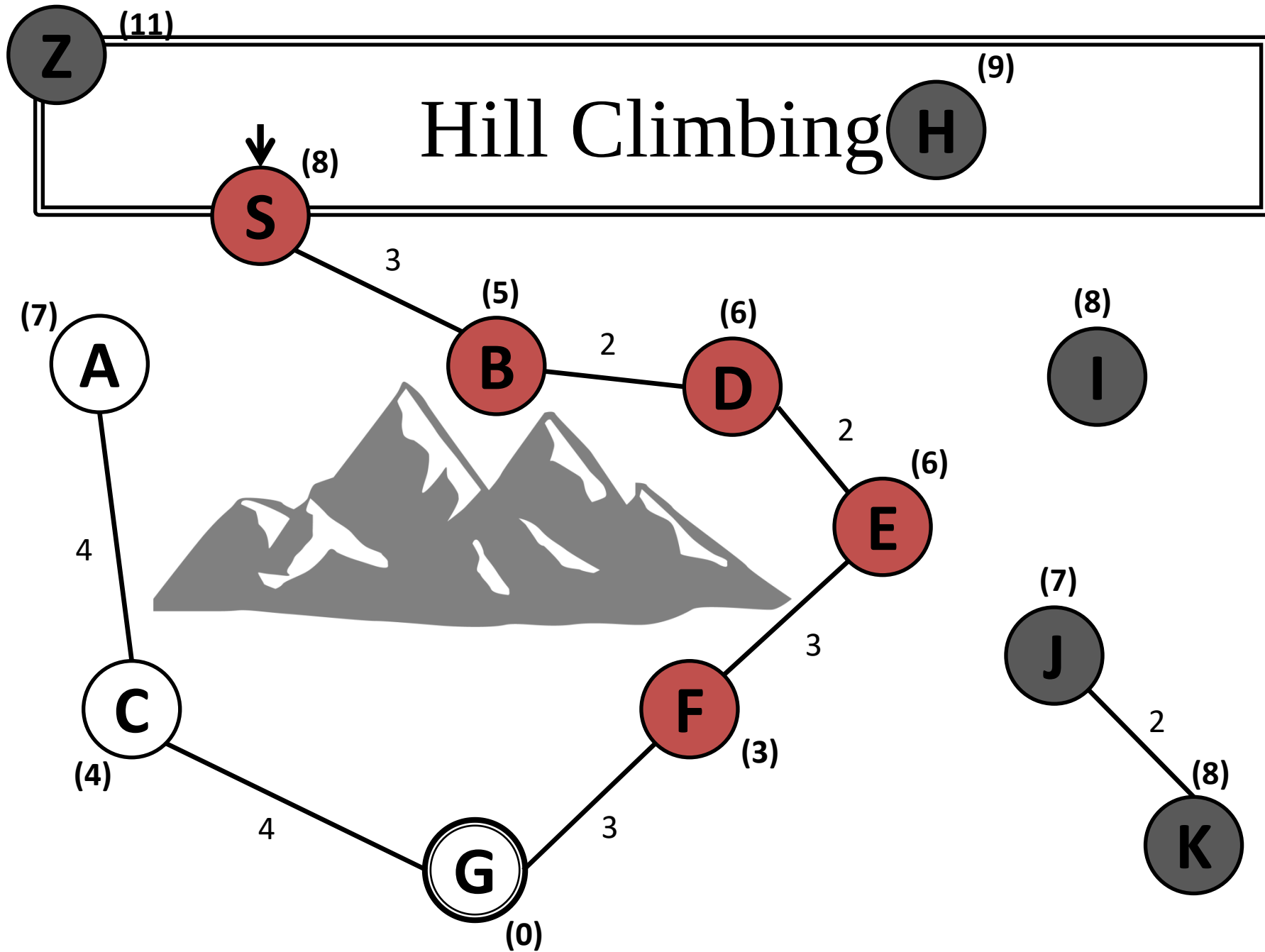


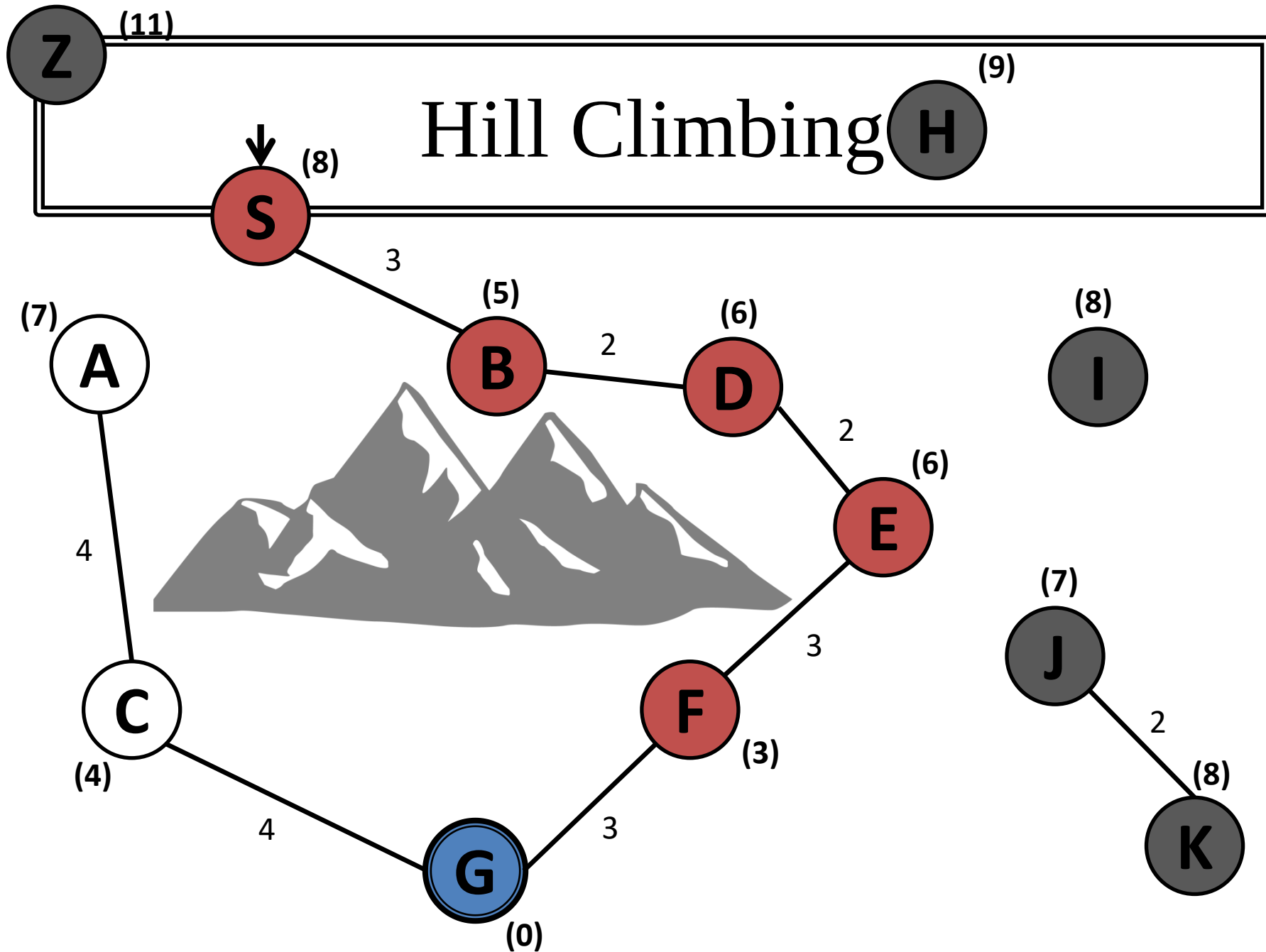


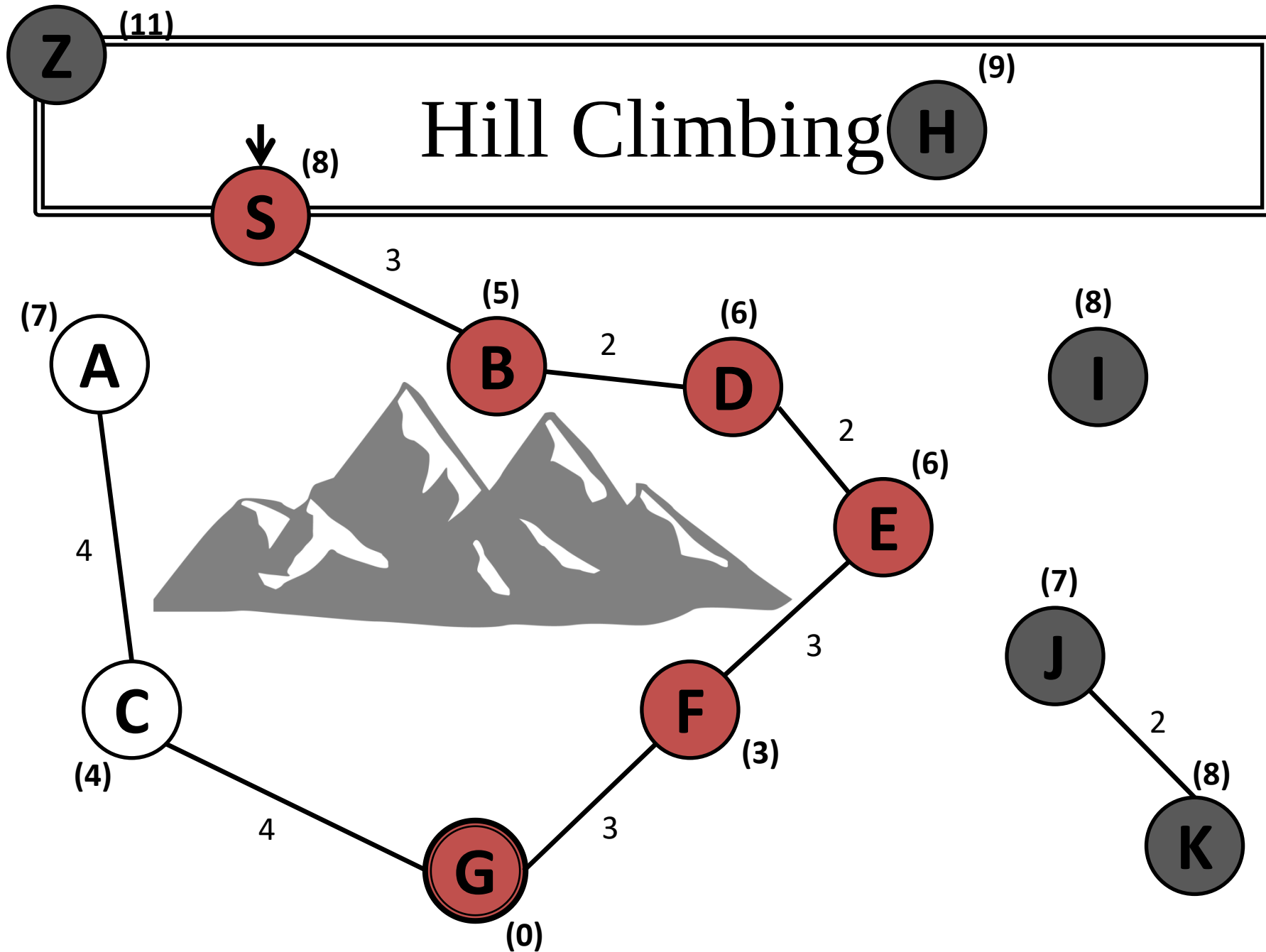


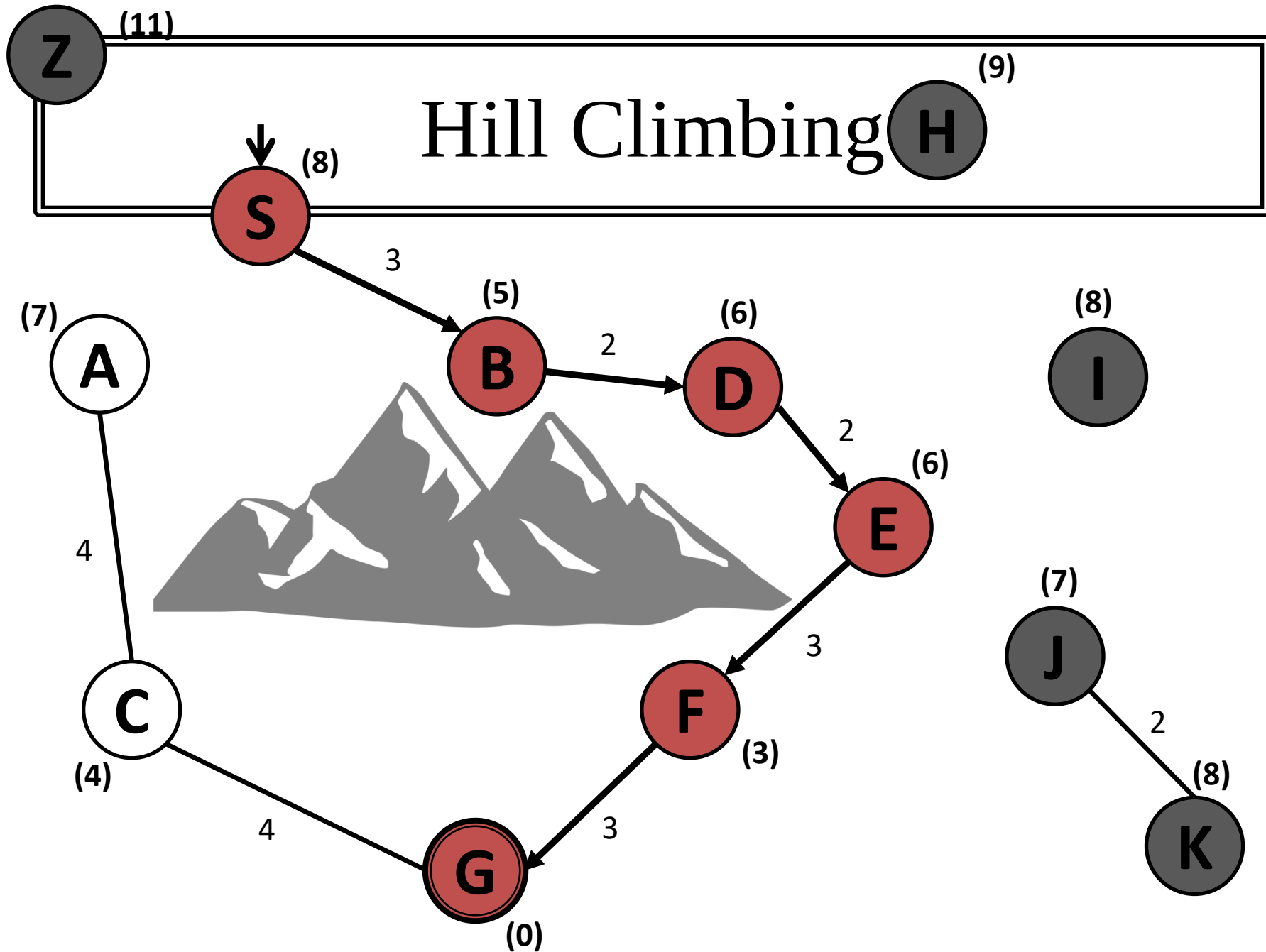












Hill Climbing

- **Complete?**
 - **No**, local search
- **Optimal?**
 - **No**
- **Time?**
 - $O(bm)$ – linear time
- **Space?**
 - $O(b)$ – constant

Beam Search and Hill Climbing

- Key use case: optimization problems
 - Beam Search and Hill Climbing are **better suited for optimization** rather than pathfinding.
 - In optimization, the focus is on **iteratively improving a solution**, rather than searching for a direct path between A and B.
- Completeness and optimality in optimization:
 - Both algorithms are **complete** because they generally converge on a solution, though it may not always be the optimal one.
 - Both algorithms are **not optimal**, due to limited exploration and risks of getting stuck in local optima.

A*

- **Idea:** avoid expanding paths that are already expensive

- Evaluation function $f(n) = g(n) + h(n)$

$g(n)$ = cost so far to reach n

$h(n)$ = estimated cost to goal from n

$f(n)$ = estimated total cost of path through n to goal

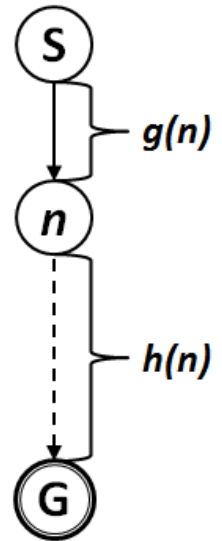
- **A*** search uses an **admissible heuristic**

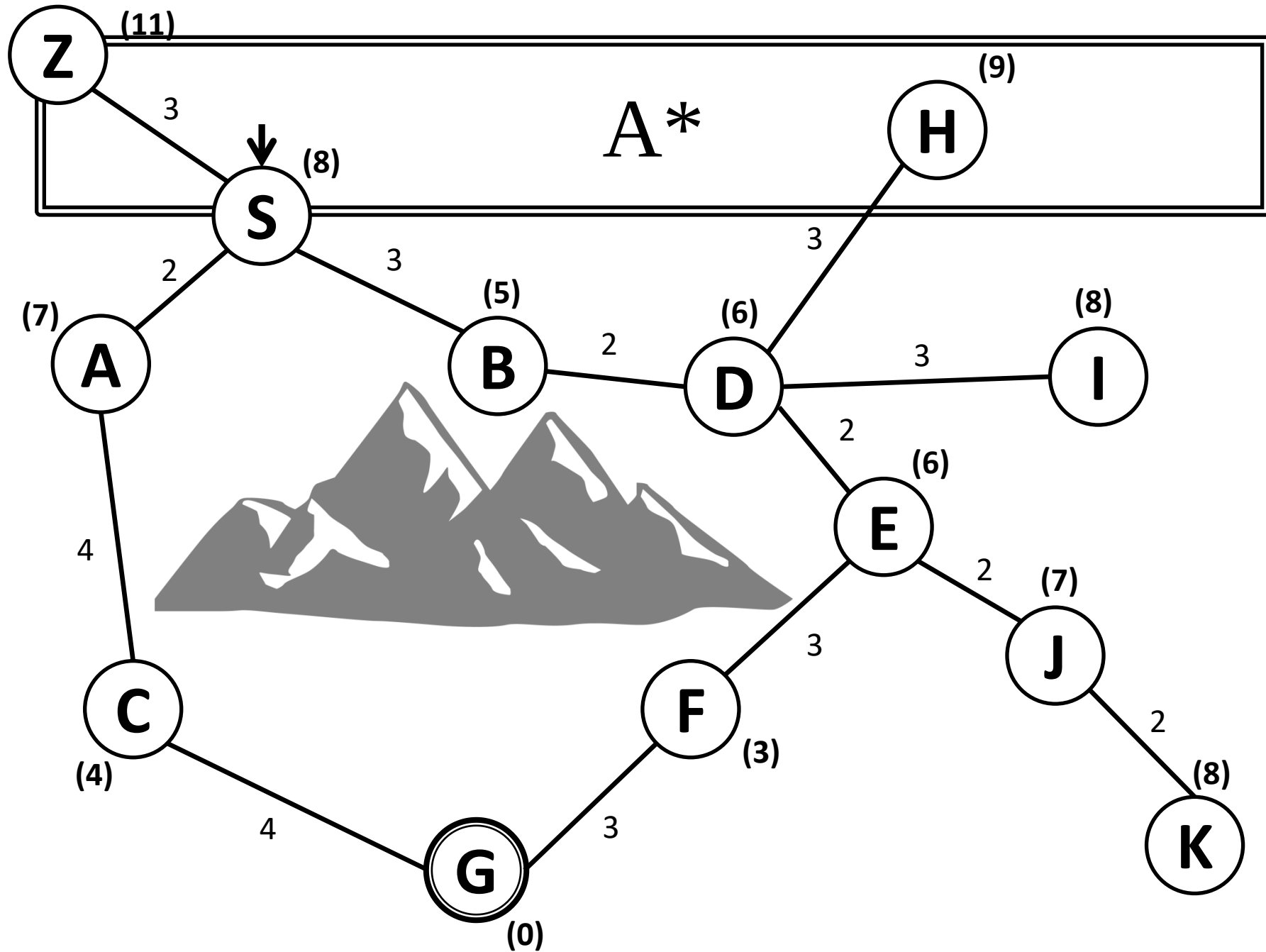
i.e., $h(n) \leq h^*(n)$ where $h^*(n)$ is the **true** cost from n .

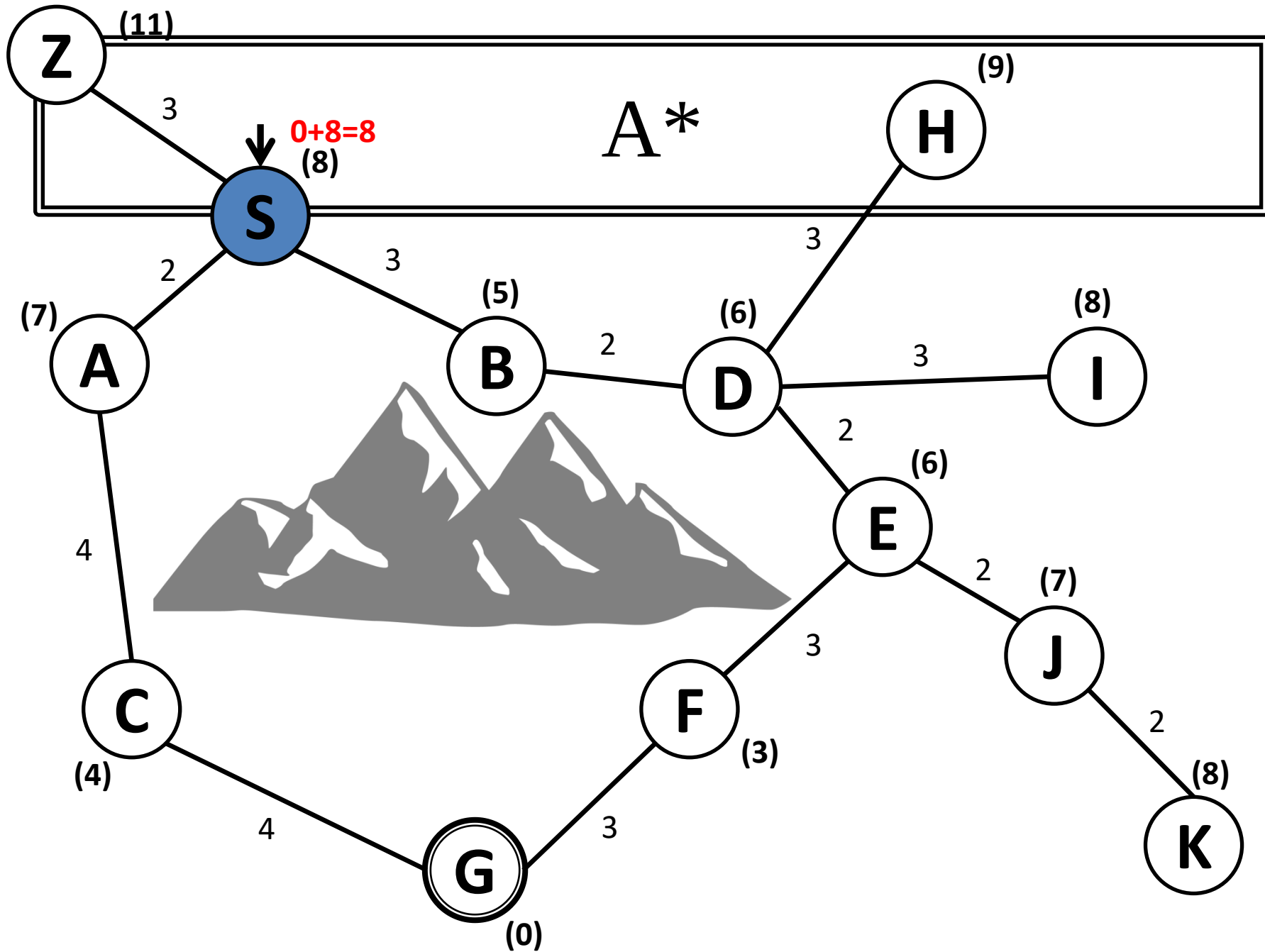
(Also require $h(n) \geq 0$, so $h(G) = 0$ for any goal G .)

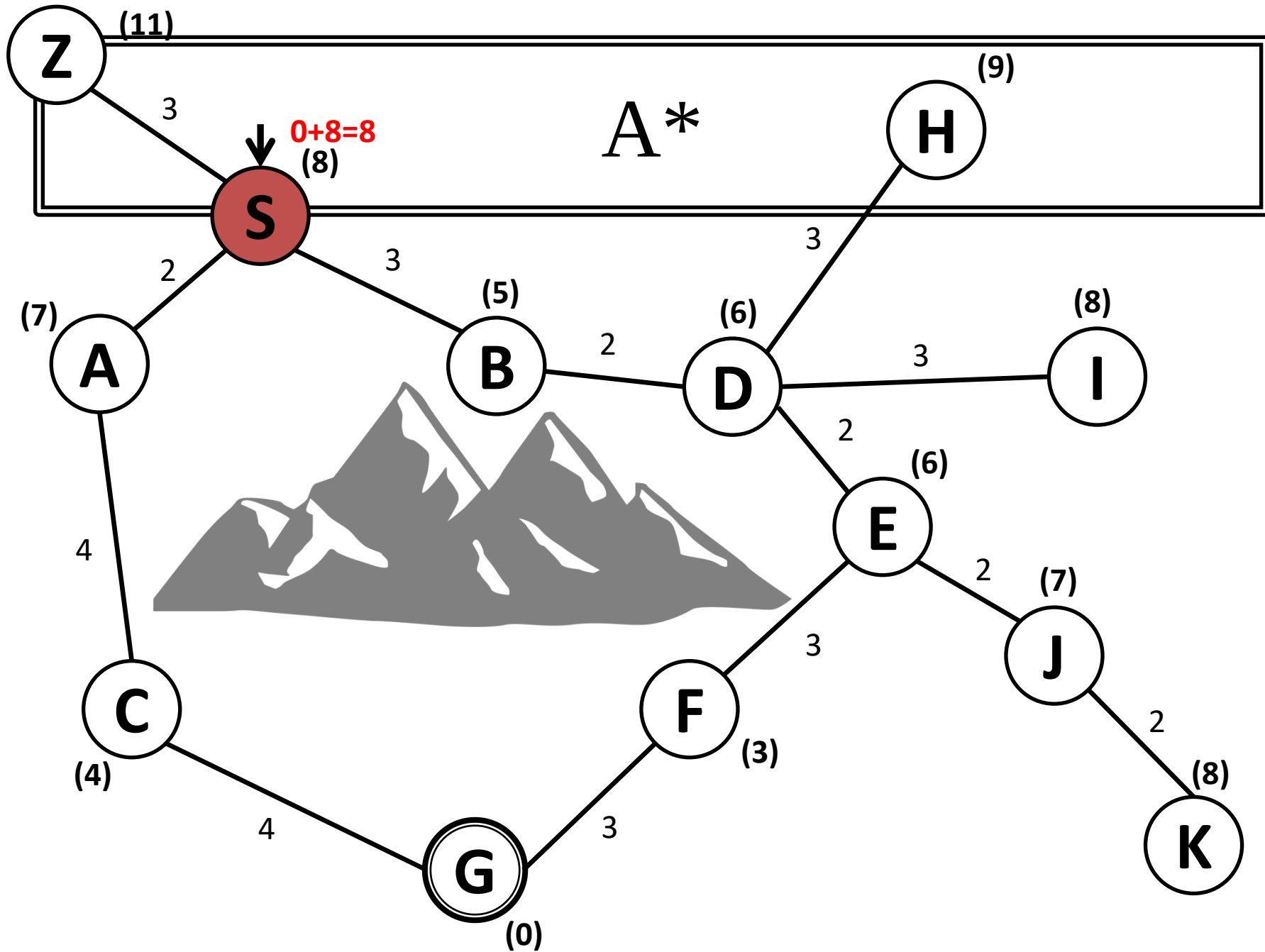
- E.g., $h_{SLD}(n)$ never overestimates the actual road distance

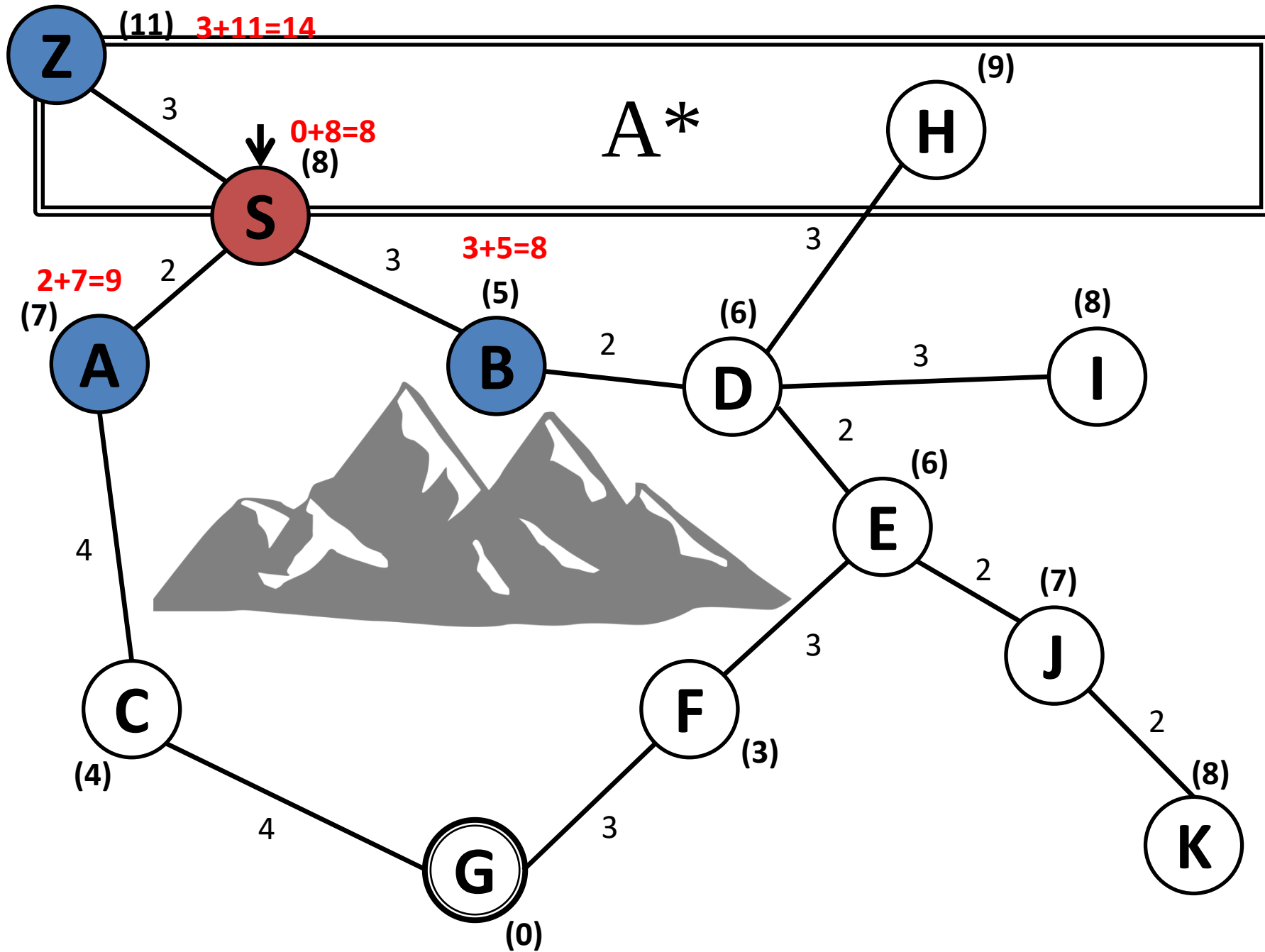
- **Theorem:** **A*** search is optimal

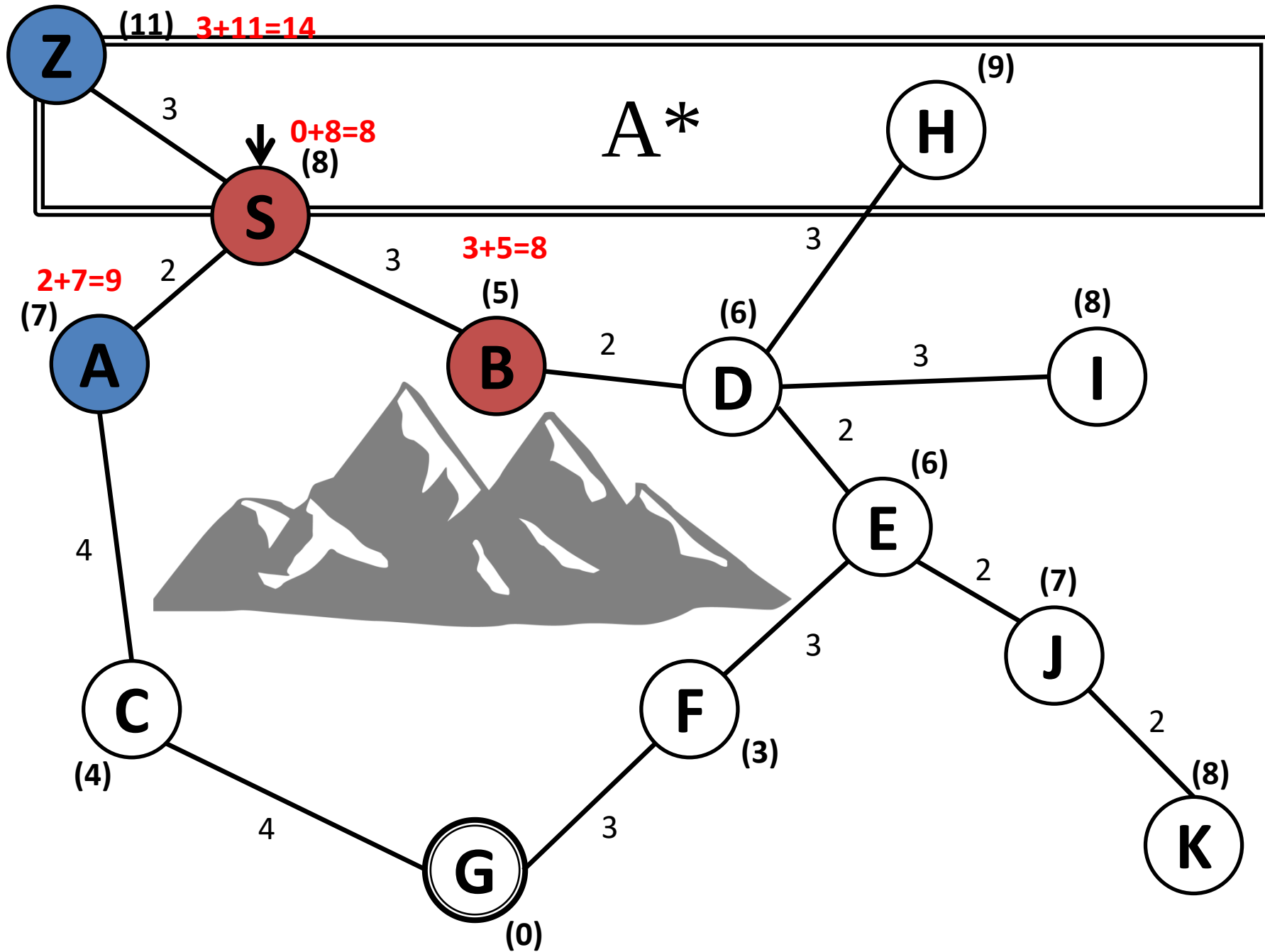


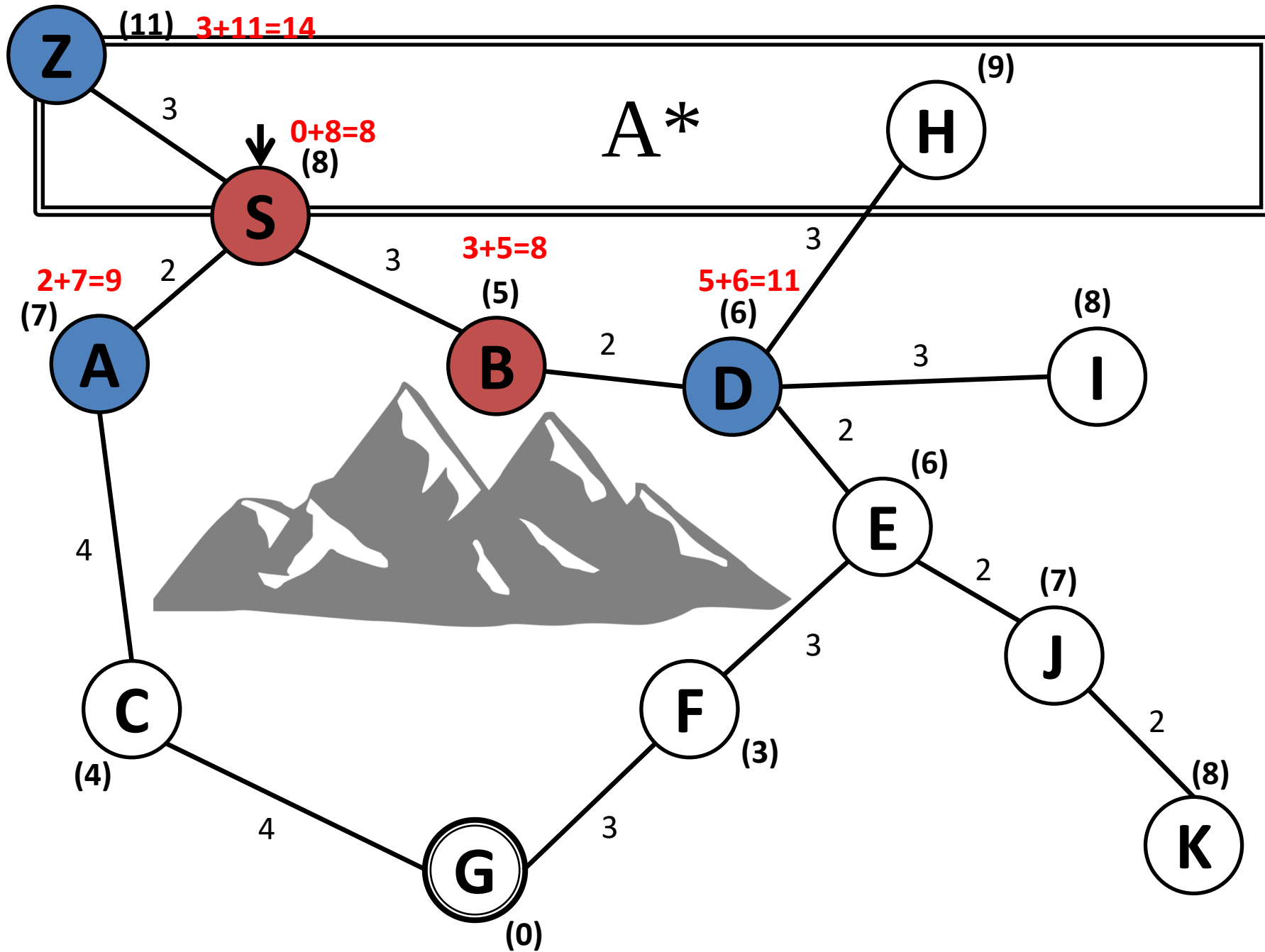


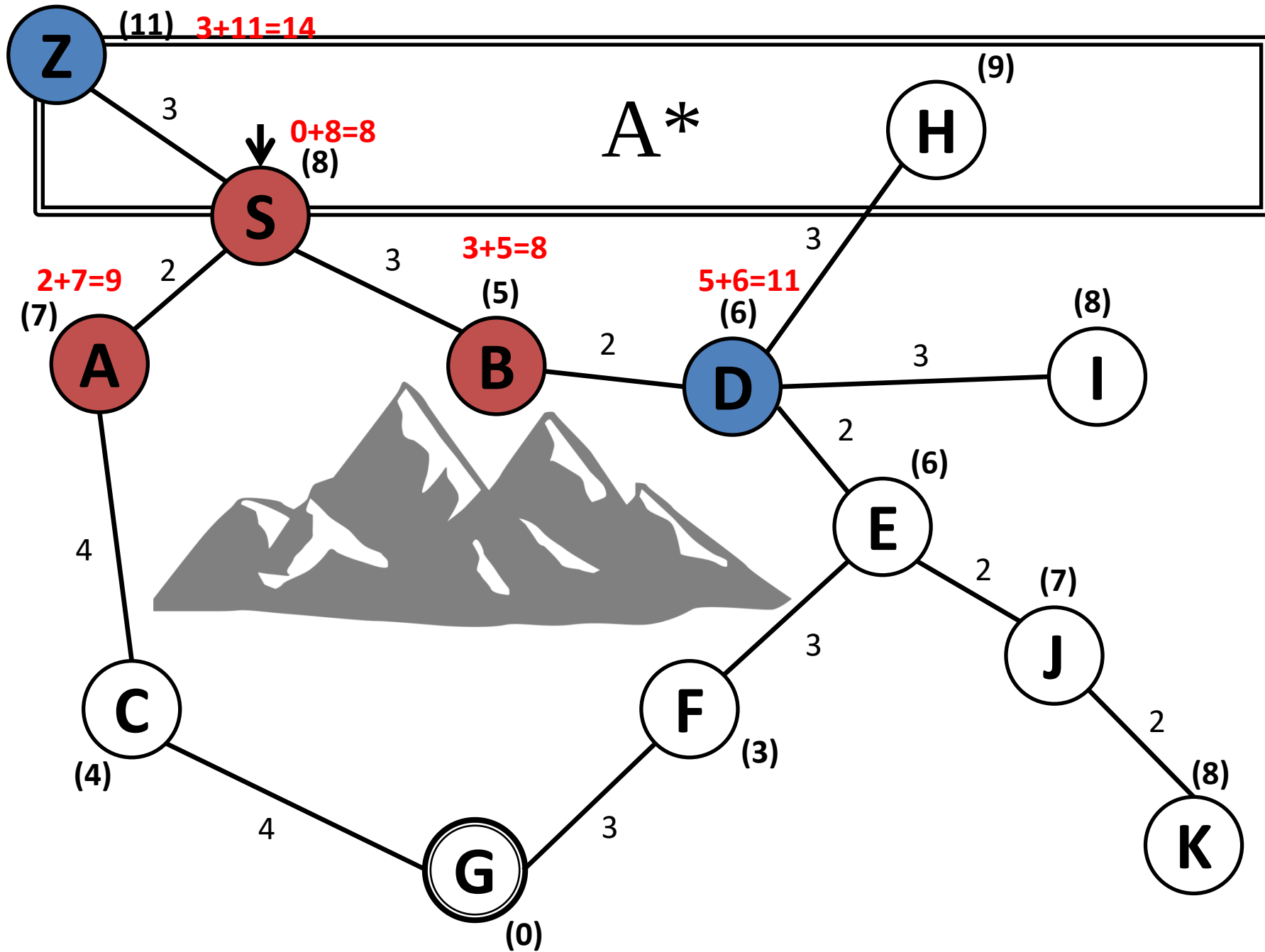


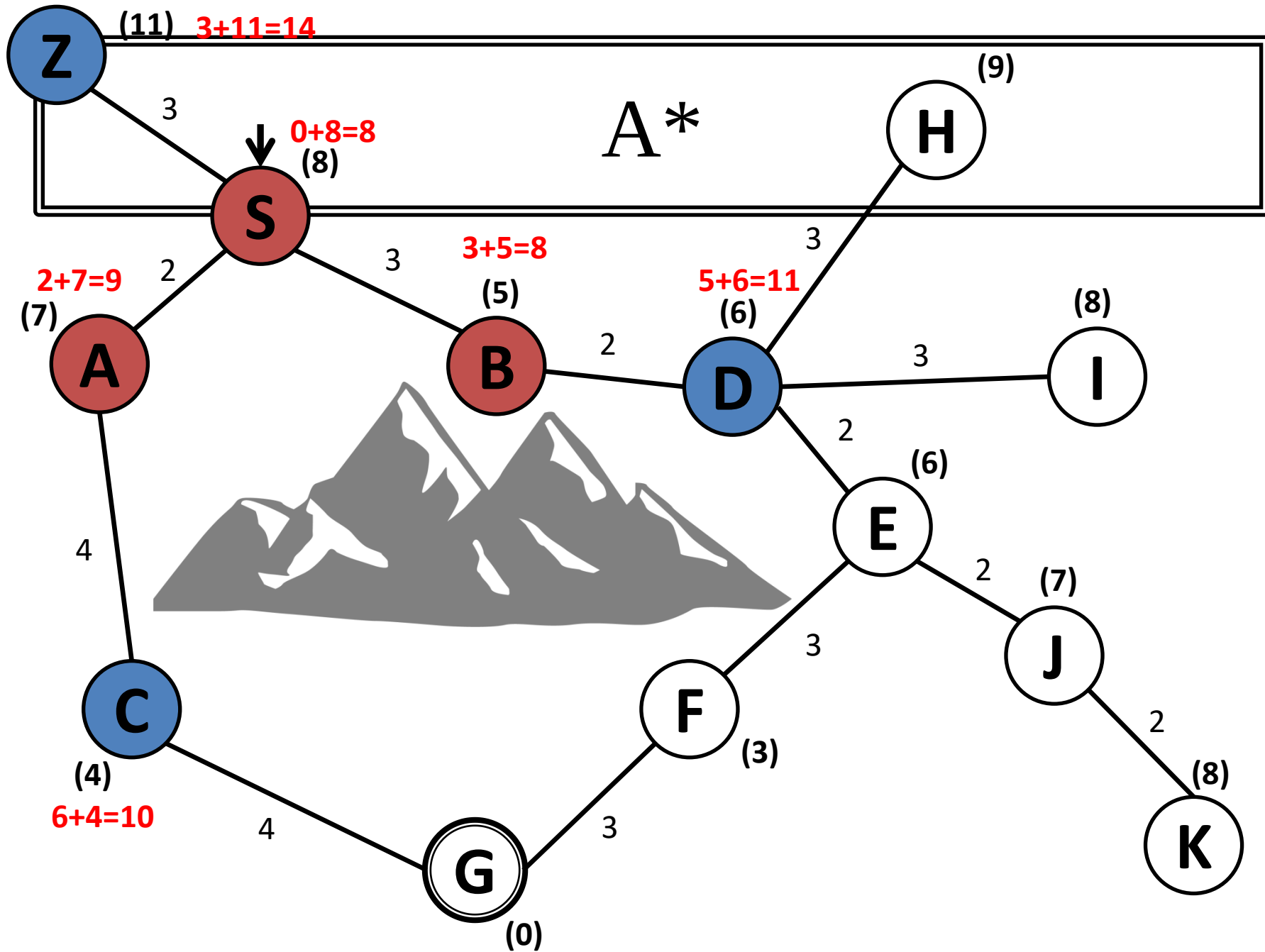


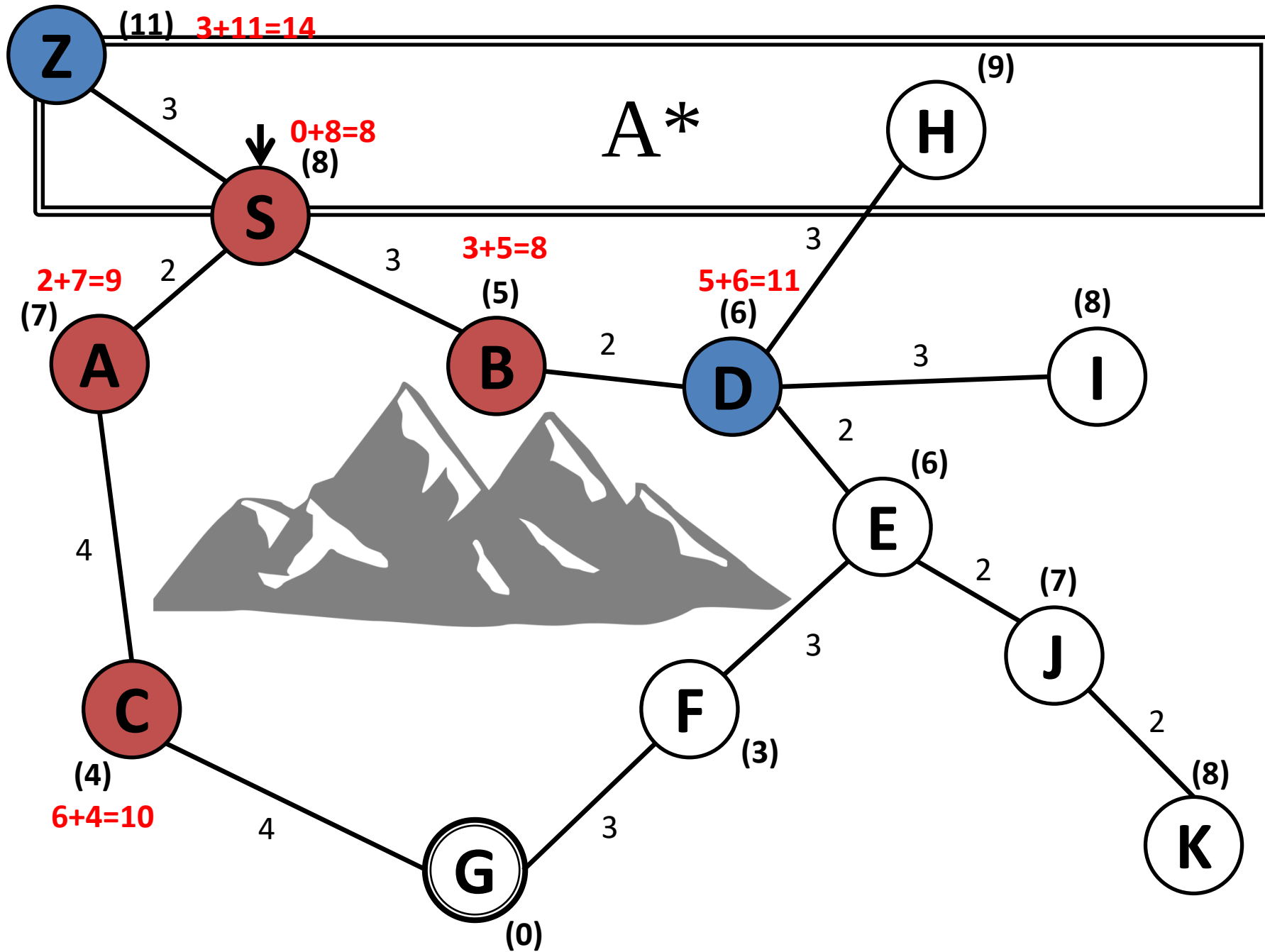


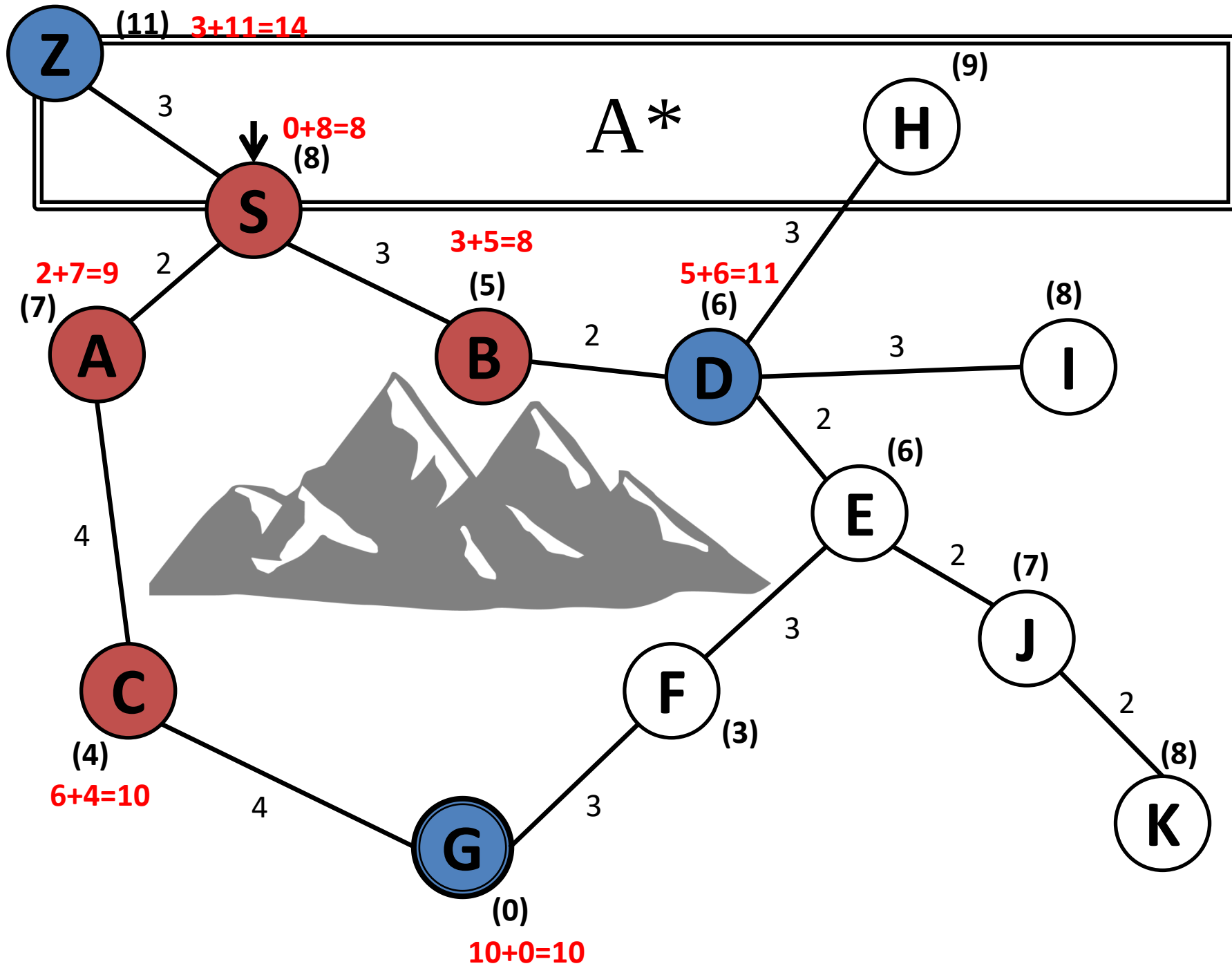


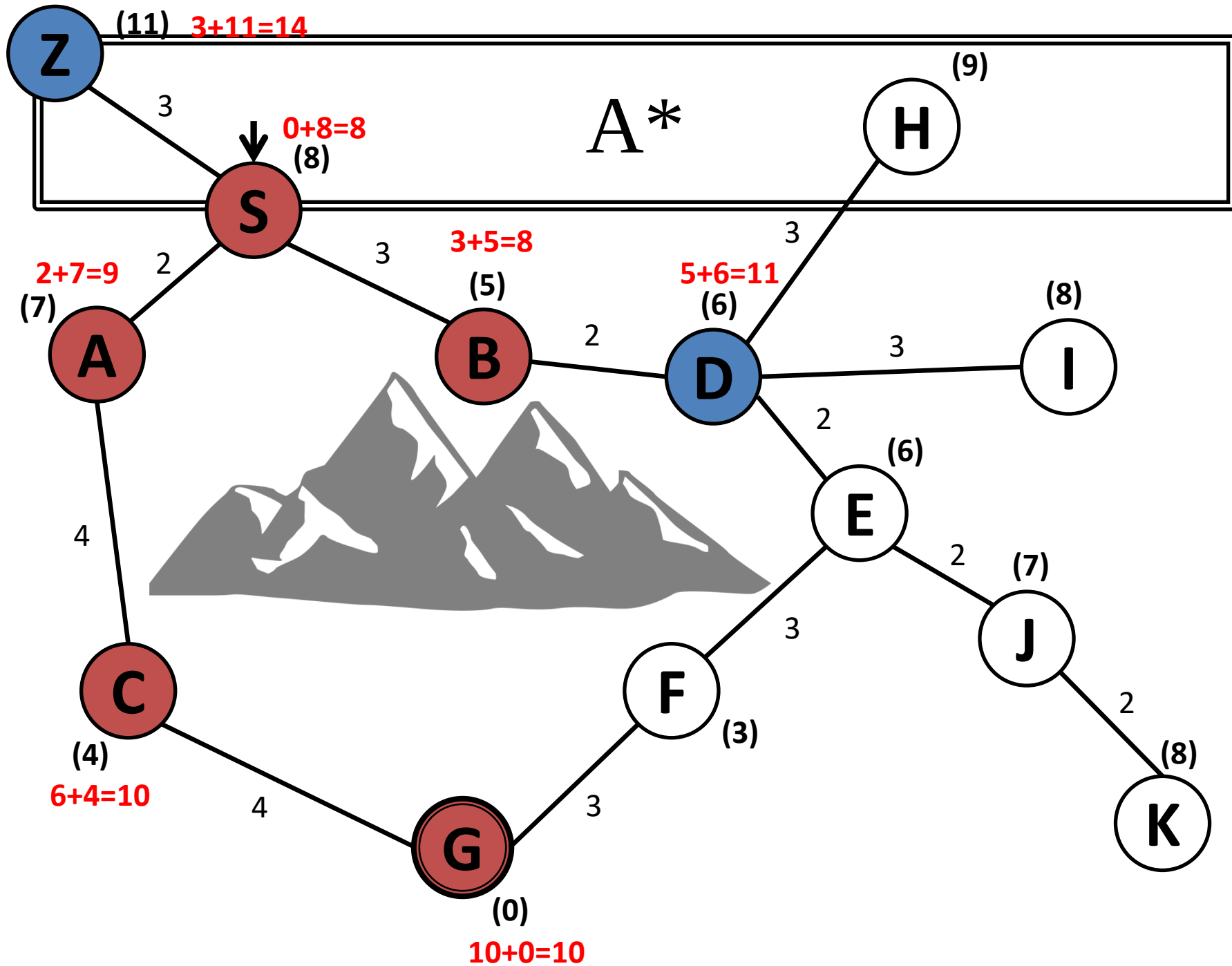


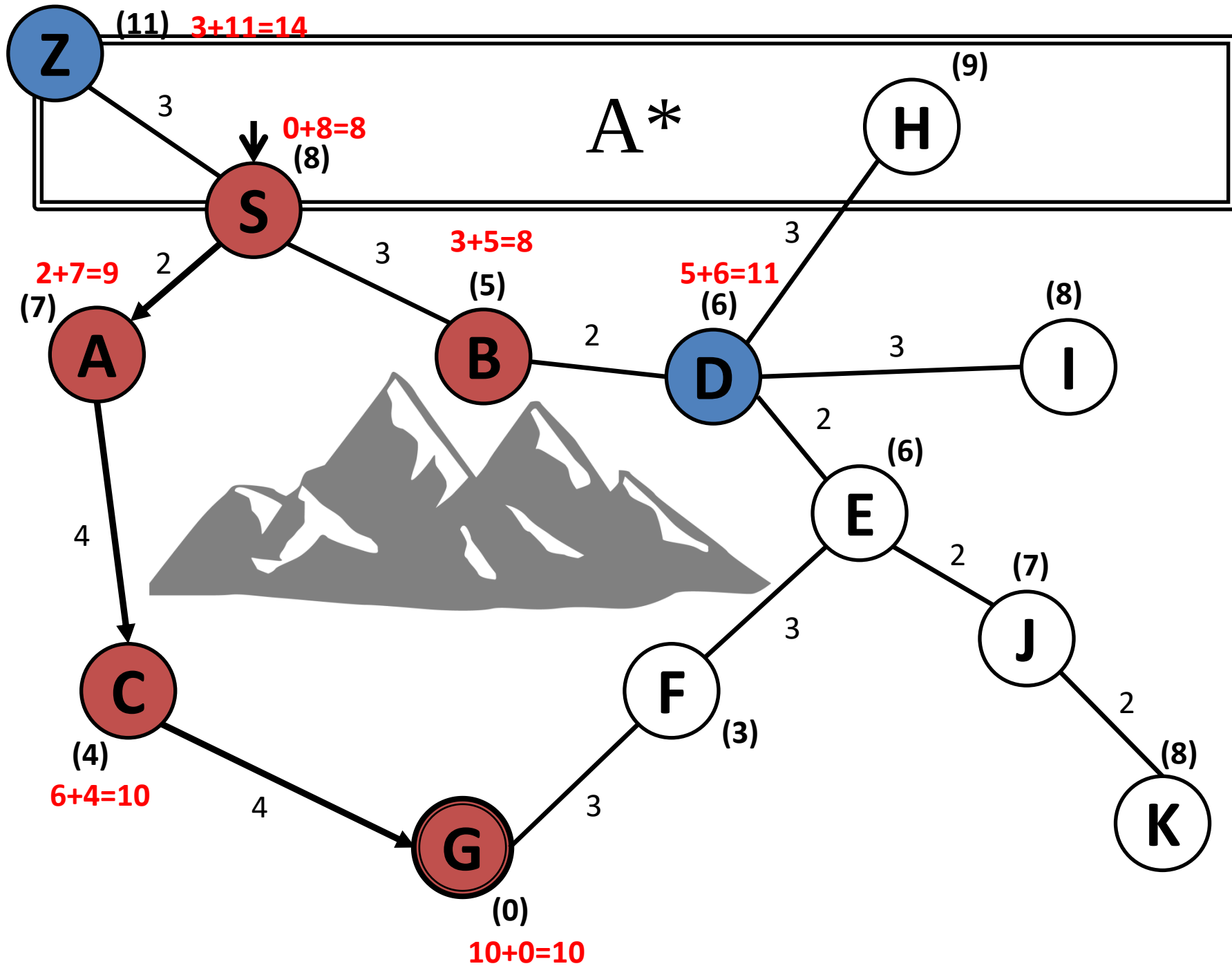












A^*

- **Complete?**
 - *Yes*, unless there are infinitely many nodes with $f \leq f(G)$
- **Optimal?**
 - *Yes*, cannot expand f_{i+1} until f_i is finished
 - A^* expands all nodes with $f(n) < C^*$*
 - A^* expands some nodes with $f(n) = C^*$*
 - A^* expands no nodes with $f(n) > C^*$*
- **Time?**
 - $O(b^d)$: Exponential in [relative error in $h \times$ length of soln.]
- **Space?**
 - $O(b^d)$: Keeps all nodes in memory

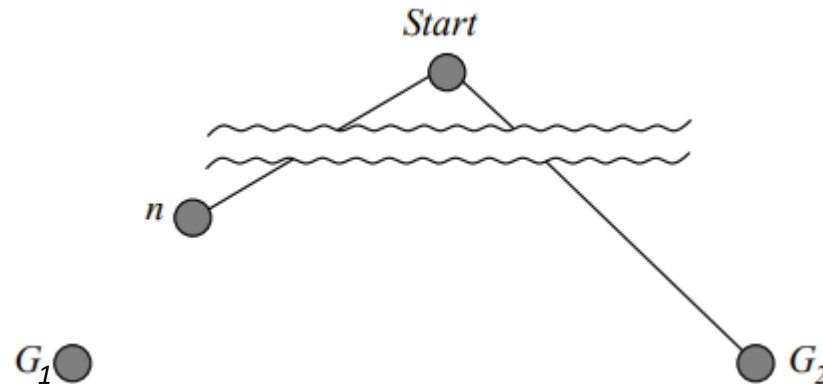
Theorem: A* Search is Optimal

- **Theorem:** If $h(n)$ is admissible, A* using TREE-SEARCH is optimal
- **Theorem:** If $h(n)$ is consistent, A* using GRAPH-SEARCH is optimal
 - **Optimality:** A consistent heuristic guarantees that once a node is expanded, its cost will not be improved upon. Thus, the path found by A* will be the shortest possible.
 - **Efficiency:** Because consistent heuristics prevent the re-expansion of nodes, they help reduce unnecessary evaluations, leading to a more efficient search process.

** If $h(n)$ is **consistent**, then it is also **admissible**; however, the converse is not necessarily true.*

Optimality of A^* (standard proof)

Suppose some suboptimal goal G_2 has been generated and is in the queue.
Let n be an unexpanded node on a shortest path to an optimal goal G_1 .



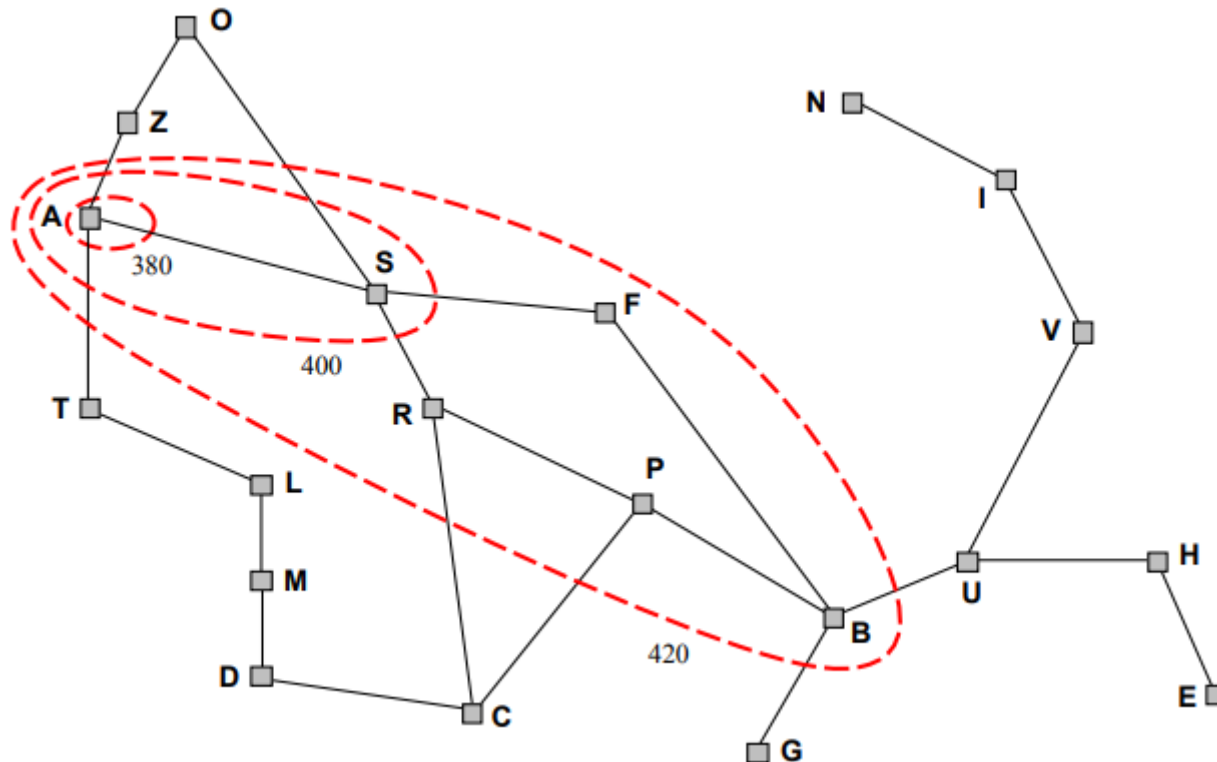
$$\begin{aligned} f(G_2) &= g(G_2) \\ &> g(G_1) \\ &\geq f(n) \end{aligned}$$

since $h(G_2) = 0$
since G_2 is suboptimal
since h is admissible

Since $f(G_2) > f(n)$, A^ will never select G_2 for expansion*

Optimality of A^* (more useful)

- **Lemma:** A^* expands nodes in order of increasing f value*
Gradually adds “ f -contours” of nodes (cf. breadth-first adds layers)
Contour i has all nodes with $f = f_i$, where $f_i < f_{i+1}$



Proof of Lemma: Consistency

- A heuristic is **consistent** if

$$h(n) \leq c(n, a, n') + h(n')$$

- If h is consistent, we have

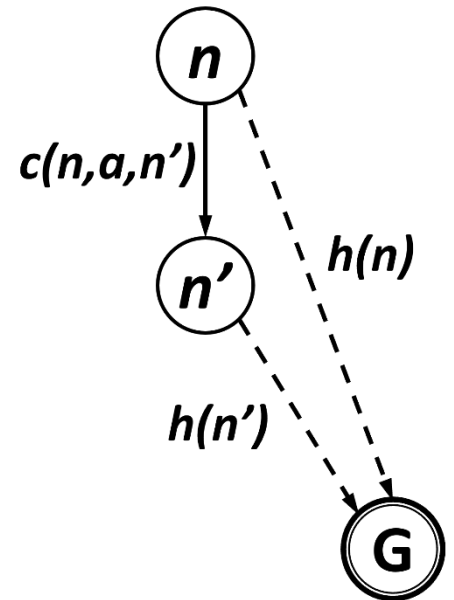
$$f(n') = g(n') + h(n')$$

$$= g(n) + c(n, a, n') + h(n')$$

$$\geq g(n) + h(n)$$

$$= f(n)$$

I.e., $f(n)$ is nondecreasing along any path.



Admissible and Inconsistent Heuristic

Is **A*** with **graph search** always guaranteed to find the **optimal solution** when using an **admissible heuristic**?

The heuristic h is admissible:

$$h(S) = 4 \leq 4 = 1+1+2$$

$$h(A) = 3 \leq 3 = 1+2$$

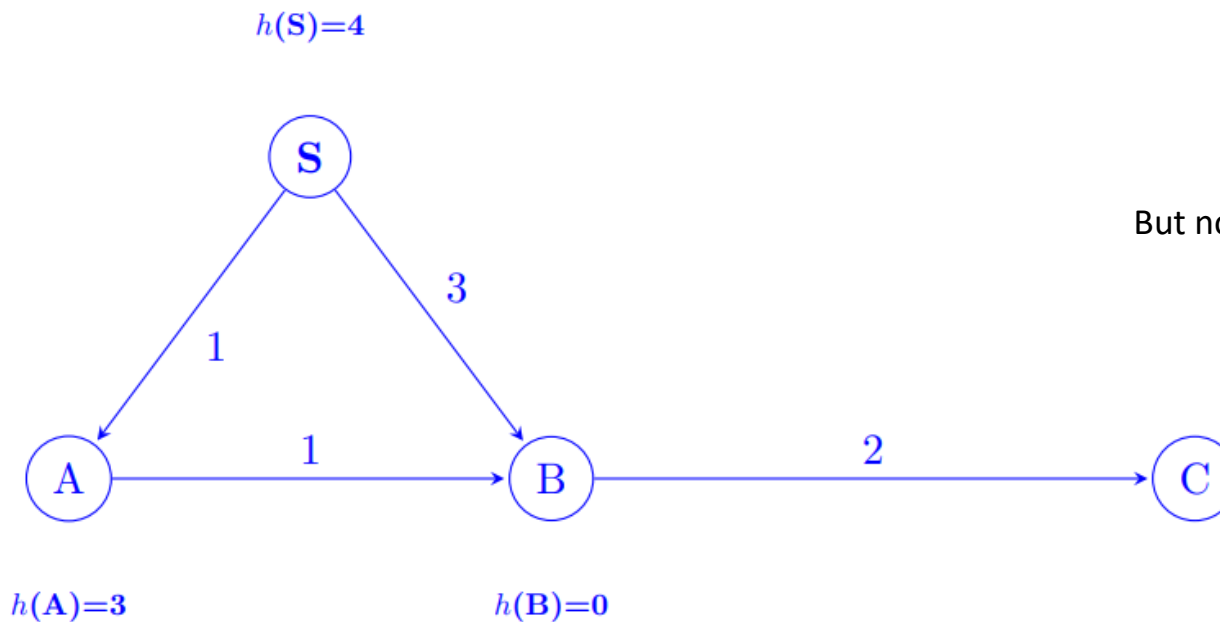
$$h(B) = 0 \leq 2$$

$$h(C) = 0 \leq 0$$

But not consistent (inconsistent):

$$h(A) \leq c(A,B) + h(B)$$

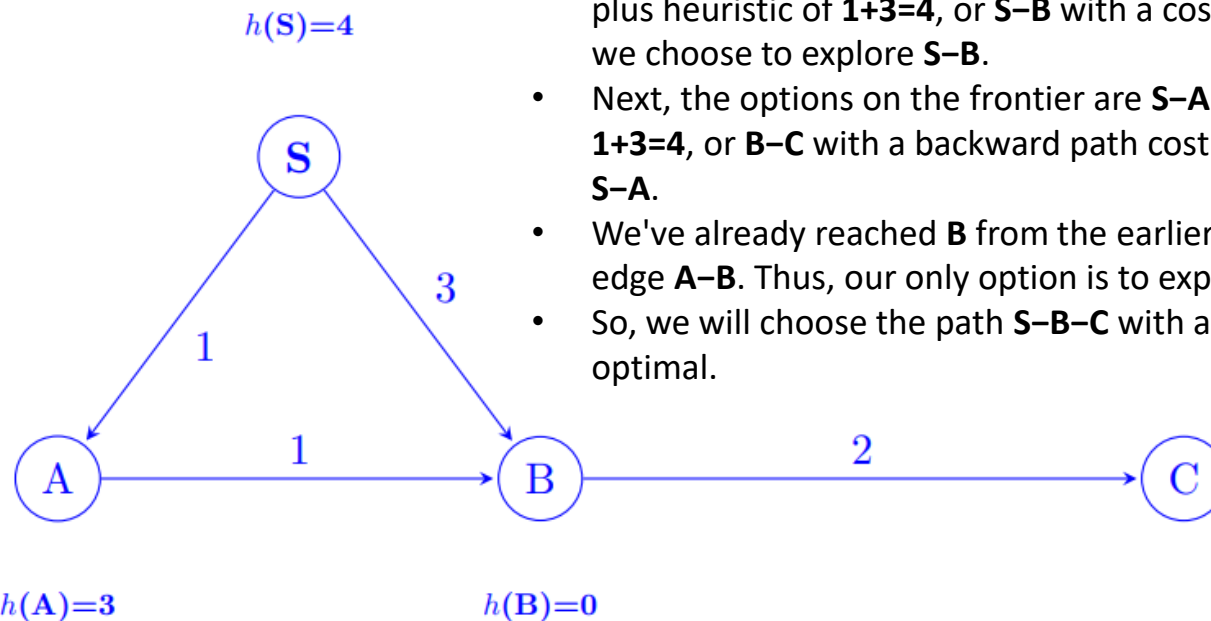
$$\begin{array}{ccccc} \downarrow & & \downarrow & & \downarrow \\ \textcolor{red}{3} & > & \textcolor{red}{1} & + & \textcolor{red}{0} \end{array}$$



Admissible and Inconsistent Heuristic

Is **A*** with **graph search** always guaranteed to find the **optimal solution** when using an **admissible heuristic**?

The optimal path from **S** to **C** is **S-A-B-C** with a total cost of **4**. However, with **A*** **graph search**:



- After starting from **S**, the options on the frontier are **S-A** with a cost plus heuristic of **1+3=4**, or **S-B** with a cost plus heuristic of **3+0=3**. So, we choose to explore **S-B**.
- Next, the options on the frontier are **S-A** with a cost plus heuristic of **1+3=4**, or **B-C** with a backward path cost of **5**. So, we choose to explore **S-A**.
- We've already reached **B** from the earlier path, so we don't consider edge **A-B**. Thus, our only option is to explore **B-C** for a cost of **5**.
- So, we will choose the path **S-B-C** with a total cost of **5**, which is not optimal.

There is a distinction between *tree search* and *graph search*: **A*** with **tree search** is guaranteed to be optimal if the heuristic is admissible, but **A*** with **graph search** is only guaranteed to be optimal if the heuristic is consistent.

Tree Search

- In tree search, a **closed set is not needed** because nodes are not revisited; there's no way to reach the same node via multiple paths.
- The **heuristic must be admissible**, ensuring that **A*** will find the **optimal path**.

** In A*, the **open set** is managed as a **priority queue**, holding nodes yet to be explored, while the **closed set** contains **visited nodes** that have already been evaluated, preventing their re-exploration.*

Graph Search

- In graph search, a **closed set is essential** to avoid revisiting nodes through different paths (e.g., due to loops).
- The heuristic must be both **admissible** and **consistent** to guarantee that **A*** finds the **optimal path efficiently**.

Graph Search & Consistent Heuristic

- Once a node is expanded, there is no chance to reach this node through a shorter path.
- Even without a **closed set**, a consistent heuristic guarantees that **A*** will find the **optimal path**.
- However, reprocessing the same node through longer paths can lead to *inefficiencies*, so it is still beneficial to use a **closed set** for better performance.

Graph Search & Inconsistent Heuristic

- Efficiency Issues
 - The **inconsistent heuristic** does not guarantee that a node's cost is optimal when it is first expanded. This means that **A*** might discover a better path to a node later and have to re-expand it, leading to more node expansions and potentially slowing down the search.
 - In the worst case, **A*** with an inconsistent heuristic could re-expand up to 2^N nodes, where **N** is the number of states.
- Suboptimal Path Risk
 - With an **inconsistent heuristic**, the **closed set** may lead to **suboptimal solutions** by preventing the re-evaluation of nodes, as the first expansion might not reflect the true shortest path.

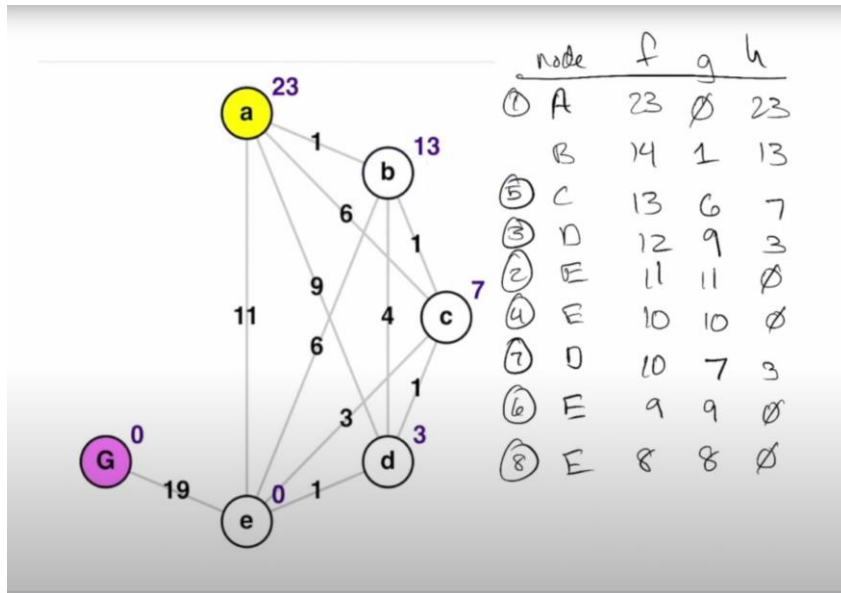
[1] [*Single Agent Search Video 31: Inconsistent Heuristics*](#)

[2] [*Single Agent Search Video 32: Handling Inconsistent Heuristics*](#)

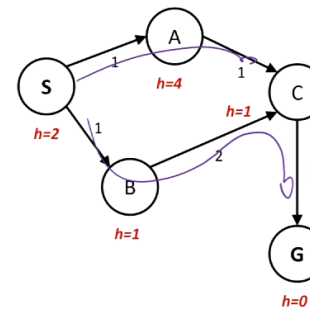
[3] [*COMPSCI 188 - 2018-08-30 - A* Search and Heuristics*](#)

Graph Search & Inconsistent Heuristic

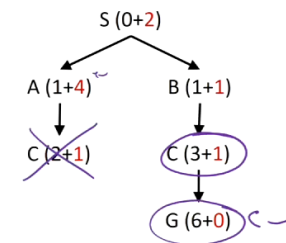
A* Graph Search Gone Wrong?



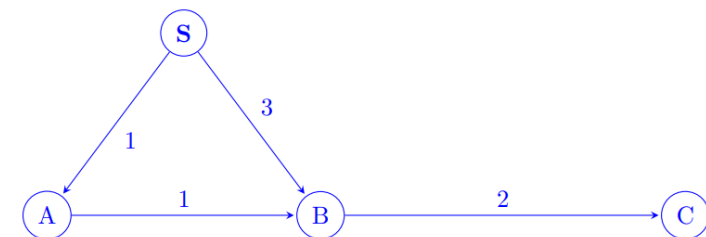
State space graph



Search tree



$$h(S)=4$$



$$h(A)=3$$

$$h(B)=0$$

Brief Summary

- Tree Search:
 - No **closed set** needed.
 - **Heuristic** must be **admissible**.
 - Guaranteed to find the **optimal path**.
- Graph search:
 - **Closed set** is essential to prevent redundant expansions.
 - **Heuristic** must be both **admissible** and **consistent**.
 - **Consistent heuristic** guarantees **optimal path** even **without closed set**, but reprocessing nodes can be **inefficient**.
 - **Inconsistent heuristic** can lead to **excessive node expansions**.
 - Using a **closed set** with an **inconsistent heuristic** can lead to **suboptimal solutions**.
 - **A*** with a **consistent heuristic** is **optimally efficient**.

** Using a **consistent heuristic** with a **closed set** is typically preferred in graph searches to ensure both **optimality** and **efficiency**!*

Admissible Heuristics

- E.g., for the 8-puzzle:
 - $h_1(n)$ = number of misplaced tiles (*Hamming distance*)
 - $h_2(n)$ = total **Manhattan** distance
(i.e., no. of squares from desired location of each tile)

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

- $h_1(S) = ?$
- $h_2(S) = ?$

Admissible Heuristics

- E.g., for the 8-puzzle:
 - $h_1(n)$ = number of misplaced tiles (*Hamming* distance)
 - $h_2(n)$ = total **Manhattan** distance
(i.e., no. of squares from desired location of each tile)

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

- $h_1(S) = ?$ $1+0+1+1+1+0+1+1 = 6$
- $h_2(S) = ?$

Admissible Heuristics

- E.g., for the 8-puzzle:
 - $h_1(n)$ = number of misplaced tiles (*Hamming* distance)
 - $h_2(n)$ = total **Manhattan** distance
(i.e., no. of squares from desired location of each tile)

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

- $h_1(S) = ?$ $1+0+1+1+1+0+1+1 = 6$
- $h_2(S) = ?$ $4+0+3+3+1+0+2+1 = 14$

Admissible Heuristics

- E.g., for the 8-puzzle:
 - $h_1(n)$ = number of misplaced tiles (*Hamming* distance)
 - $h_2(n)$ = total **Manhattan** distance
(i.e., no. of squares from desired location of each tile)

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

- $h_1(S) = ?$ $1+0+1+1+1+0+1+1 = 6$
- $h_2(S) = ?$ $4+0+3+3+1+0+2+1 = 14$

* $cellDist = |x - x'| + |y - y'|$

Manhattan Distance

$y' \backslash x'$	0	1	2
0	7	2	4
1	5		6
2	8	3	1

Start State

$y \backslash x$	0	1	2
0	1	2	3
1	4	5	6
2	7	8	

Goal State

$$cellDist = |x - x'| + |y - y'| = |0 - 2| + |0 - 2| = 2 + 2 = 4$$

* The **Hamming** and **Manhattan** heuristics are both **admissible** and **consistent**.

Dominance

- If $h_2(n) \geq h_1(n)$ for all n (both admissible)
then h_2 dominates h_1 and is better for search
- Typical search costs:
 - $d = 14$ IDS = 3,473,941 nodes
 - $A^*(h_1) = 539$ nodes
 - $A^*(h_2) = 113$ nodes
 - $d = 24$ IDS $\approx 54,000,000,000$ nodes
 - $A^*(h_1) = 39,135$ nodes
 - $A^*(h_2) = 1,641$ nodes
- Given any admissible heuristics h_a, h_b ,
$$h(n) = \max(h_a(n), h_b(n))$$

is also admissible and dominates h_a, h_b

Relaxed Problems

- A problem with fewer restrictions on the actions is called a **relaxed problem**.
- The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem.
- Admissible heuristic can be derived from the **exact** solution cost of a **relaxed** version of the problem.
- If the rules of the 8-puzzle are relaxed so that a tile can move **anywhere**, then $h_1(n)$ gives the shortest solution.
- If the rules are relaxed so that a tile can move to **any adjacent square**, then $h_2(n)$ gives the shortest solution.
- Key point: the optimal solution cost of a relaxed problem is no greater than the optimal solution cost of the real problem.

Heuristics from Relaxed Problems

In general, most natural admissible heuristics tend to be consistent, especially if from relaxed problems.

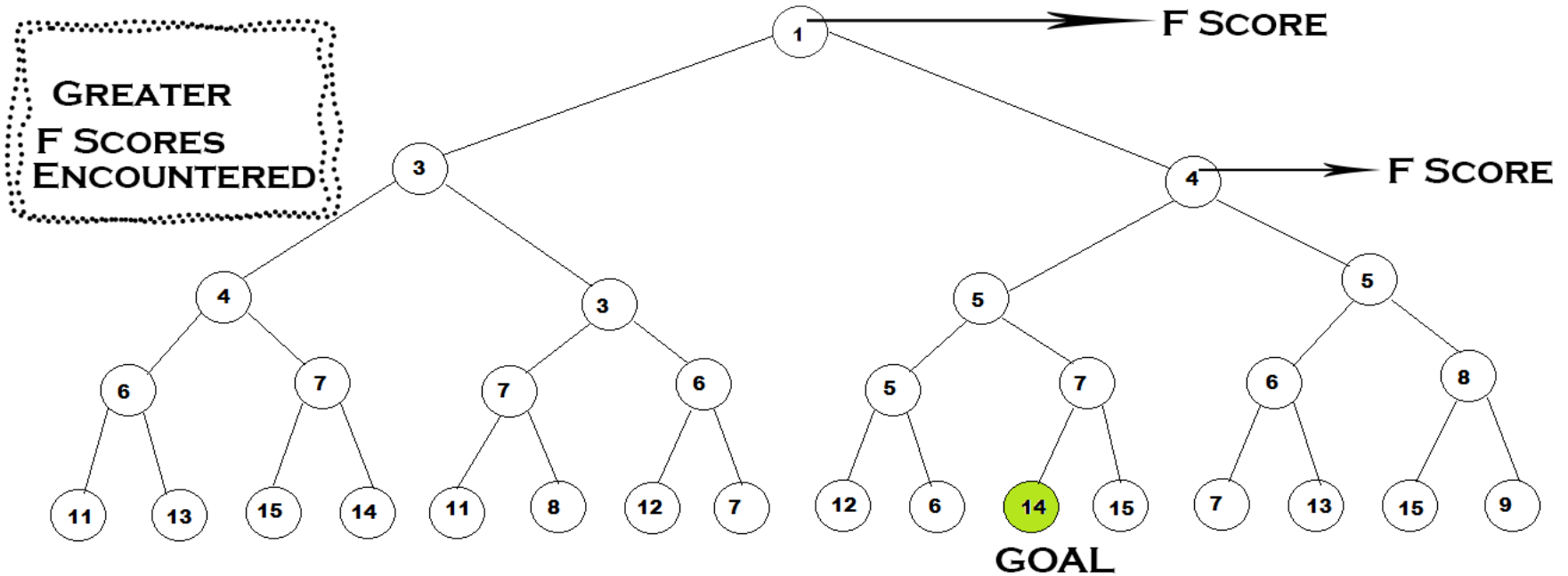
Memory-Based A^*

- = combination between A^* and *beam-search*,
i.e., keeps l nodes in the queue

Iterative Deepening A^*

- = combination between A^* and *iterative-deepening-search*,
i.e., uses A^* evaluation function as a *threshold* and runs *DLS*

Iterative Deepening A*



Copied from:

<https://algorithmsinsight.files.wordpress.com/2016/03/ida-star.gif?w=1326>

Summary of Algorithms

Criterion	Greedy	Beam Search	Hill Climbing	A*	IDA*
Complete	No*	No	No	Yes*	Yes
Time	b^m	blm	bm	b^d	b^d
Space	b^m	bl	b	b^d	bd
Optimal	No	No	No	Yes	Yes
Data Structure	Priority Queue	Limited Priority Queue	„Limited Priority Queue“	Priority Queue	Stack

Homework: “*Sliding Blocks* (*N Puzzle*)”

- [The game](#) starts with a board consisting of blocks numbered 1 through N and one blank block represented by the number 0. The goal is to arrange the tiles according to their numbers. Moving is done by moving the blocks on top, bottom, left and right in place of the empty block.
- At the input is given the number N - the number of blocks with numbers (8, 15, 24, etc.), the number I - the index of the position of zero (the empty block) in the decision (using -1 the default zero index position is set at the bottom right) and then the layout of the board is introduced. Using the **A*** (or **IDA***) algorithm and the *Manhattan distance heuristics* (or *Hemming distance*), derive:
 - In the first line, the length of the "**optimal**" path from start to destination.
 - The appropriate steps (in a new line for each one) that are taken to reach the final state. The steps are **left**, **right**, **up** and **down**
 - Keep in mind that not every puzzle is solvable. You can [check whether the puzzle is solvable](#) or directly use valid examples.

Homework: “*Sliding Blocks* (*N Puzzle*)”

- **Sample input:**

8

-1

1 2 3

4 5 6

0 7 8

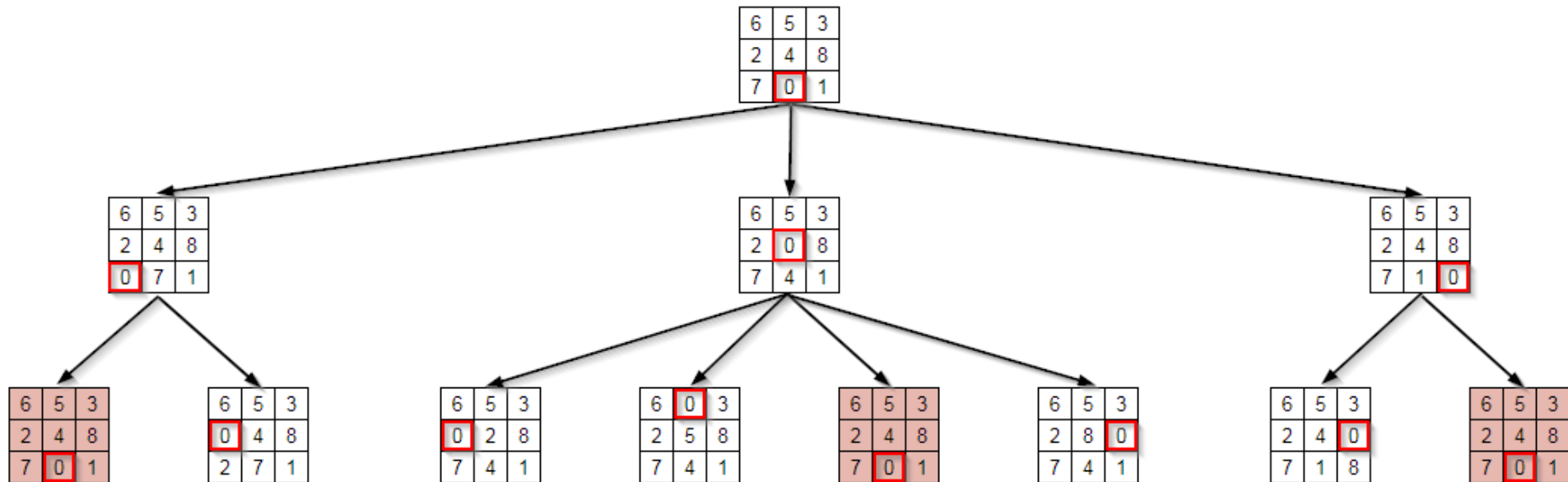
- **Sample output:**

2

left

left

Sliding Blocks (N Puzzle)



Sliding Blocks (N Puzzle)

```
class Node {  
    int[][] matrix = new int[MATRIX_DIMENSION][MATRIX_DIMENSION];  
    int distance = 0;  
    List<String> pathToTheNode; //Or reference to the parent  
}
```

Sliding Blocks (N Puzzle)

A*: Pseudocode

Iterative Deepening A*: Pseudocode

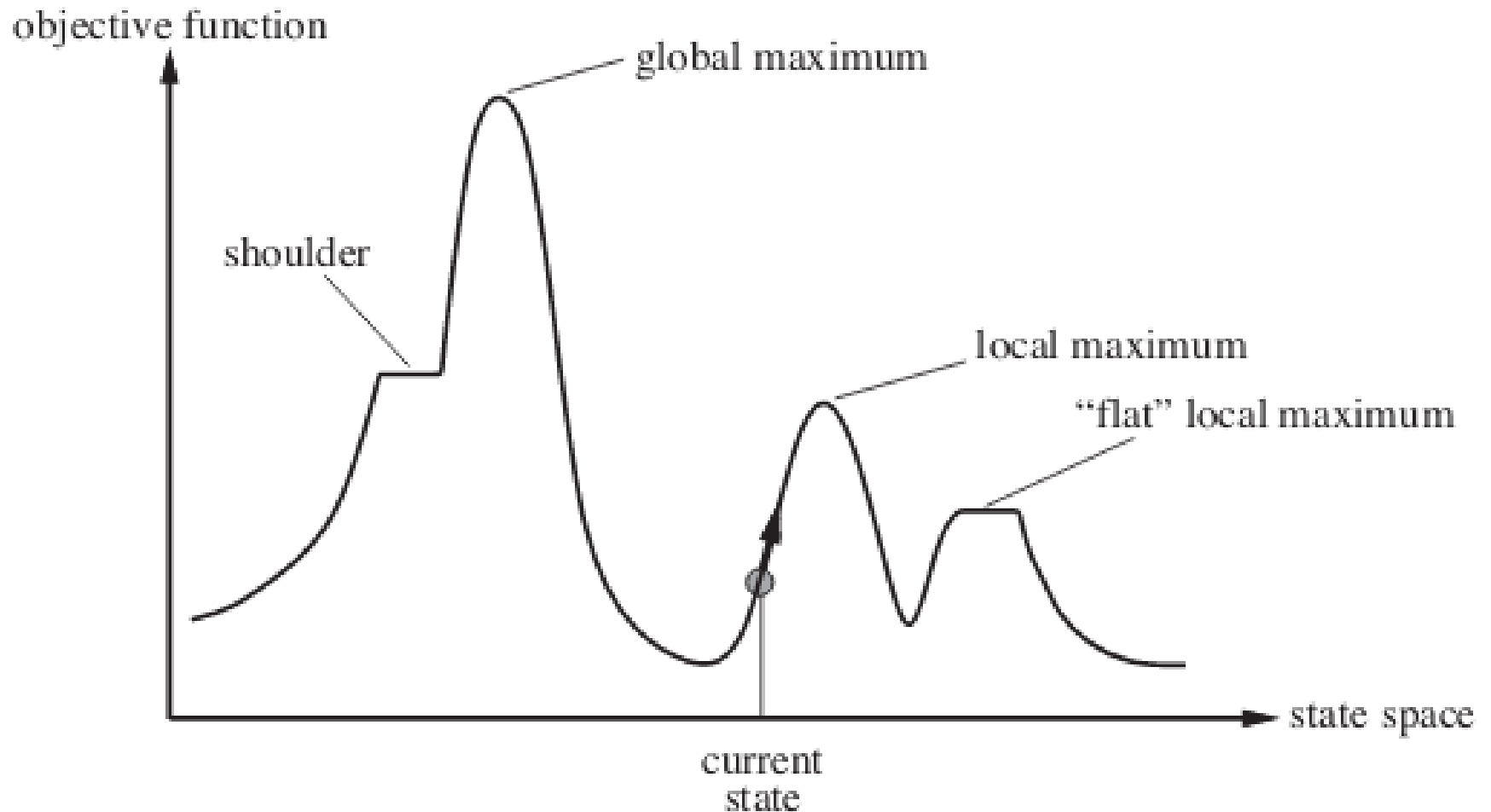
Local Search

- In many optimization problems, **path** is irrelevant; the goal state itself is the solution
- Then state space = set of "complete" configurations
find **optimal** configuration, e.g., TSP
or, find configuration satisfying constraints, e.g., timetable
- In such cases, can use **iterative improvement** (local search) algorithms;
keep a single "current" state, try to improve it
- Constant space, suitable for online as well as offline search

Local Search

- Local Beam Search
- Hill Climbing
- Variants of Hill Climbing:
 - Simulated Annealing: Allows “downhill” moves to escape local optima, but still not guaranteed to be optimal.
 - Stochastic Hill Climbing: Introduces randomness, improving exploration, but no guaranteed optimality.

Local Search



Hill Climbing (or Gradient Ascent/Descent)

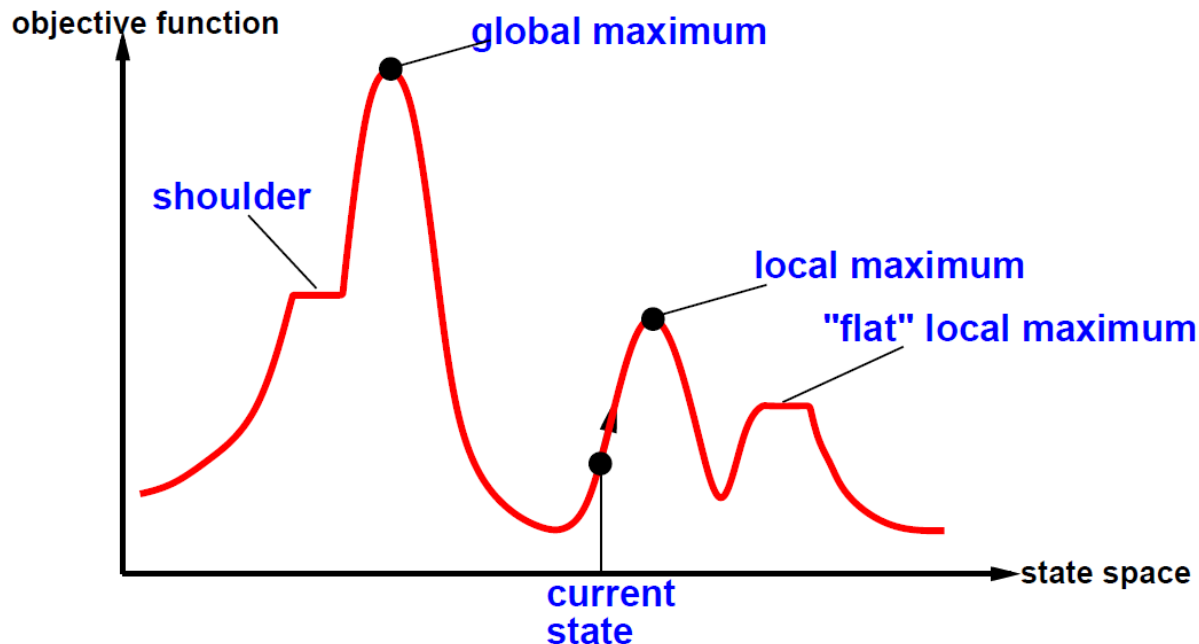
- “Like climbing Everest in thick fog with amnesia”

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                  neighbor, a node

  current ← MAKE-NODE(INITIAL-STATE[problem])
  loop do
    neighbor ← a highest-valued successor of current
    if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
    current ← neighbor
```

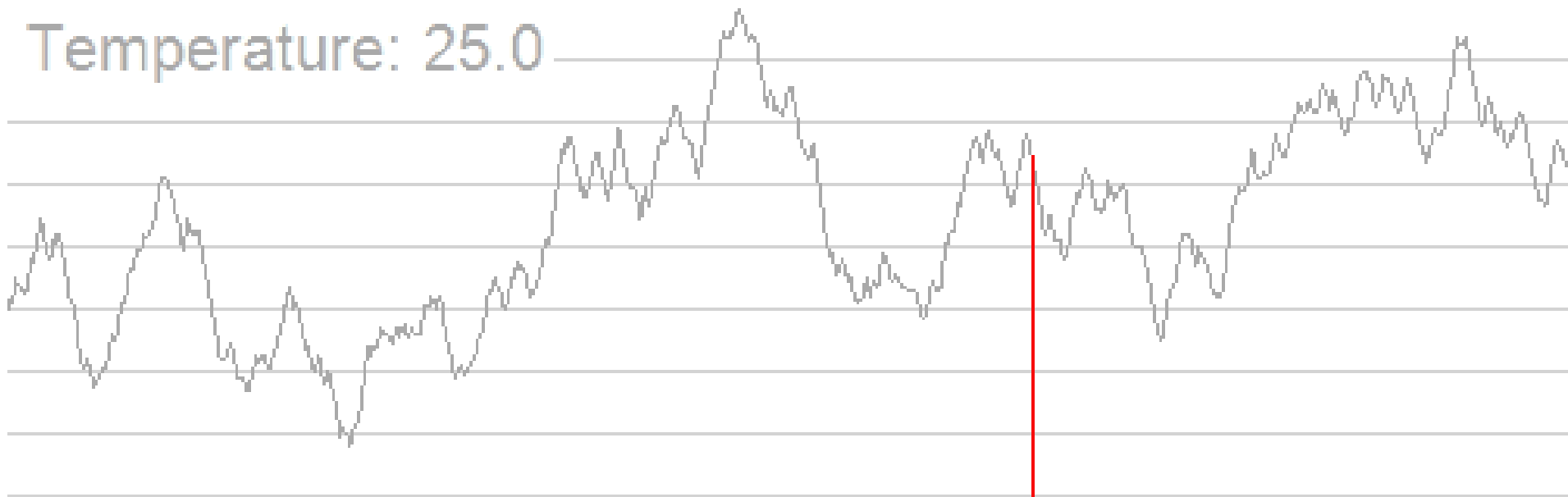
Hill Climbing

- Useful to consider state space landscape



- Random-restart hill climbing** overcomes local maxima – trivially complete
- Random sideways moves** 😊 escape from shoulders 🙄 loop on flat maxima

Simulated Annealing



[Simulated Annealing Video](#)

Simulated Annealing

- **Idea:** escape local maxima by allowing some "bad" moves **but gradually decrease their size and frequency.**

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to "temperature"
  local variables: current, a node
                   next, a node
                   T, a "temperature" controlling prob. of downward steps

  current ← MAKE-NODE(INITIAL-STATE[problem])
  for t ← 1 to ∞ do
    T ← schedule[t]
    if T = 0 then return current
    next ← a randomly selected successor of current
     $\Delta E \leftarrow \text{VALUE}[\textit{next}] - \text{VALUE}[\textit{current}]$ 
    if  $\Delta E > 0$  then current ← next
    else current ← next only with probability  $e^{\Delta E/T}$ 
```